



DEGREE PROJECT IN Electrical Engineering,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2025

Safe Weighted Federated Learning: Accelerated Multi-Party Computation Architectures based Intel SGX

Jin Xin

Authors

Jin Xin <xiji@kth.se>

Information and Communication Technology Innovation, Electrical Engineering
KTH Royal Institute of Technology

Place for Thesis

Stockholm, Sweden

Examiner

Johan Håstad

KTH Royal Institute of Technology

Supervisor

Ming Xiao

KTH Royal Institute of Technology

Tianxiang Dai, Yong Li

Huawei Munich Research Center, Trustworthy Technology and Engineering Lab

Abstract

Federated Learning (FL) allows training Machine Learning (ML) models without sharing private data, which is crucial for domains like finance and healthcare. However, FL is vulnerable to inference and membership attacks. To enhance security, Multi-Party Computation (MPC) are introduced in FL, but they face challenges such as high computational costs, memory requirements, and communication overhead, leading to prolonged training times.

This thesis introduces a Weighted Federated Learning (wFL) approach using secret-sharing scheme MPC, and accelerated by plain-text computation with Intel Software Guard Extensions (SGX). We propose and evaluate two secure wFL computation infrastructures, SGXDL and HybridSGXDL, for their efficiency, feasibility, and practicality in Convolutional Neural Networks (CNNs).

Because of utilizing Intel SGX enabling plain-text computation, these infrastructures achieve significant efficiency improvements, reducing training time by at least $5\times$ compared to Piranha(Graphics processing units (GPUs)-accelerated MPC platform). For moderate tasks, improvements can reach $164\times$, and efficiency gains continue with larger models. Intel SGX limit available libraries to C standard libraries and very less C++ libraries, consequently requiring MPC developers' expertise in both applied mathematics and C implementation. SGXDL and HybridSGXDL demonstrate the practicality of securely training CNNs such as ResNet18 and VGG16, allowing developers to access and use the models for secure training without extensive mathematics expertise.

Keywords

Federated Learning, Intel SGX, Multi-Party Computation

Abstract

FL möjliggör träning av maskininlärningsmodeller (ML) utan att dela privat data, vilket är avgörande för områden som finans och sjukvård. FL är dock sårbart för inferens- och medlemskapsattacker. För att öka säkerheten används Multi-Party Computation (MPC), men dessa står inför utmaningar som höga beräkningskostnader, minneskrav och kommunikationsbelastning, vilket leder till förlängda träningstider.

Denna avhandling introducerar en viktad federerad inlärningsmetod med användning av MPC baserat på en hemlighetsdelningsschema, accelererat genom beräkningar i klartext med Intel Software Guard Extensions (SGX). Vi föreslår och utvärderar två säkra viktade FL-beräkningsinfrastrukturer, SGXDL och HybridSGXDL, för deras effektivitet, genomförbarhet och praktiska tillämpning i konvolutionella neurala nätverk (CNN).

Genom att använda Intel SGX för klartextberäkningar uppnår dessa infrastrukturer betydande effektivitetsförbättringar, vilket minskar träningstiden med minst 5× jämfört med Piranha (GPUs-accelererad MPC-plattform). För måttliga uppgifter kan förbättringarna nå 164×, och effektivitetsvinsterna fortsätter med större modeller. Intel SGX begränsar tillgängliga bibliotek till C-standardbibliotek, vilket därmed kräver MPC-utvecklarens expertis inom både tillämpad matematik och C-implementering. SGXDL och HybridSGXDL demonstrerar praktikaliteten i att säkert träna CNNs som ResNet18 och VGG16, vilket möjliggör för utvecklare att använda modellerna för säker träning utan omfattande matematisk expertis.

Nyckelord

Federerad Inläring, Intel SGX, Flerpartiberegningsprotokoll

Acknowledgements

I would like to express my deepest gratitude to my company supervisor, Dr. Tianxiang Dai for his academic guidance and support during my thesis study. I am also immensely grateful to my colleagues: Fei Mei, for assisting with my thesis plan; Jonas Fierlings, for helping with the code implementation; and Yufan Jiang, for swiftly helping me understand the MPC mechanism.

I would like to extend my sincere thanks to my examiner, Prof.Dr. Johan Håstad at KTH, for his insightful guidance and academic suggestions very very important for the foundation of my work, which greatly contributed to the smooth progress of my thesis. Johan's assistance during my writing was particularly vital—without his help, I would not have been able to complete my master's thesis. Johan Håstad embodies the rigor, brilliance, and scholarly dedication that define a true academic. He is one of the most meticulous and talented individuals I have had the privilege to learn from, and his mentorship has been both inspiring and hands-on.

My gratitude also goes to my supervisor, Prof.Dr. Ming Xiao, at KTH.

A special acknowledgment goes to Prof.Dr. Markus Flierl at KTH, whose invaluable guidance and support were crucial during my thesis internship. His assistance with my time plan and visa arrangements enabled me to start my internship on time. I deeply appreciate his kindness, enthusiasm, and unwavering commitment to illuminating the path for students and researchers.

I aspire to one day follow in their footsteps—to help, inspire, and guide others, while making meaningful contributions in my field.

Finally, I would like to extend my heartfelt gratitude to my parents, whose unwavering support and encouragement have been a constant source of strength and inspiration throughout my life.

Acronyms

CPU	Central Processing Unit
CNNs	Convolutional Neural Networks
LLMs	Large Language Models
ReLU	Rectified Linear Unit
GPUs	Graphics processing units
FL	Federated Learning
wFL	Weighted Federated Learning
MPC	Multi-Party Computation
TEEs	Trusted Execution Environments
ML	Machine Learning
DL	Deep Learning
SGX	Software Guard Extensions
RSA	Rivest–Shamir–Adleman
PKI	Public Key Infrastructure
DP	Differential Privacy
LDP	Local Differential Privacy
HE	Homomorphic Encryption
AES	Advanced Encryption Standard
OCalls	Out Calls
MSE	Mean Squared Error
ASS	Additive Secret-Sharing

Contents

1	Introduction	1
1.1	Background	1
1.1.1	Piranha: a secure GPUs computing platform	2
1.2	Problems	3
1.2.1	Privacy Challenges in FL	3
1.2.2	Overhead Challenges in Secure FL techniques	3
1.2.3	Expanded Memory Requirements and Workload Growth Challenges	4
1.2.4	Non-linear Operation Overhead Challenge in MPC	4
1.3	Methodology	5
1.3.1	Multi-Party Computation based FL	5
1.3.2	Trusted Execution Environments(TEEs) - Intel SGX	5
1.4	Solutions	5
1.4.1	Intel-SGX Plain-text Secure Deep Learning(SGXDL)	6
1.4.2	Hybrid Intel SGX-GPUs Secure Deep Learning(HybridSGXDL)	6
1.5	Purpose	7
1.6	Dataset Description	8
1.6.1	MNIST Dataset	8
1.6.2	Cifar10 Dataset	8
1.7	Hypothesis, Environment and Threat Model	8
1.7.1	Hypothesis	8
1.7.2	Environment	10
1.7.3	Threat Model	11
2	Theoretical Background	13
2.1	Secret-Sharing based MPC	13
2.1.1	Additive Secret Sharing	14
2.1.2	Multi-Party Computation(MPC)	14

2.2	Federated Learning	15
2.2.1	History of Secret-Sharing Federated Learning	15
2.2.2	Secure techniques for Federated Learning	16
2.2.3	Data Splitting in Federated Learning	16
2.2.4	Weighted Federated Learning	17
2.3	Data shares representation in SGXDL and HybridSGXDL	18
2.3.1	The random data shares generation	19
2.3.2	SGXDL data shares generation and distribution	19
2.3.3	HybridSGXDL data shares generation and distribution	21
2.3.4	Lack of Perfect Security for the shares generation	23
2.4	MPC protocols in Piranha	23
2.5	Deep Learning	24
2.5.1	Overview of Deep Learning	25
2.5.2	Training phase of Deep Learning	26
2.5.3	Arithmetic	26
2.5.4	Non-Linear Function	31
2.6	Intel SGX	32
2.7	Hybrid AES-RSA Secure Channel	34
3	SGXDL	36
3.1	Motivation for SGXDL	36
3.2	The structure of SGXDL	37
3.2.1	Data shares distribution	37
3.2.2	Secure Channel and Data Transfer initialization	37
3.3	Task order and Synchronization	41
3.4	Summary	42
4	HybridSGXDL	43
4.1	Model Weight Share Distribution	44
4.1.1	Keys Exchange of secure channel for model weight initialization	45
4.1.2	Secure Data Transmission and Integration	46
4.2	Training Dataset Share Distribution	47
4.3	Arithmetic and Non-Linear Computation for CNNs	47
4.3.1	Arithmetic Operations on local party's GPUs	48
4.3.2	Non-Linear Operations in enclave	51

4.4	Gradient Update	52
4.5	Task order and Synchronization	53
4.6	Summary	53
5	The Deep Learning Models in the Implementation	55
5.1	The input and output of CNNs in SGXDL and HybridSGXDL	55
5.1.1	MNIST	56
5.1.2	Cifar-10	57
5.2	CNNs model structure details	58
6	Results and Analysis	62
6.1	Non-linear operation in Intel SGX	62
6.2	Efficiency Comparison SGXDL vs. HybridSGXDL vs. Piranha	63
6.3	SGXDL performance analysis	66
6.3.1	SGXDL overhead analysis	66
6.4	HybridSGXDL performance analysis	69
6.4.1	Non-linear and Arithmetic Computation	69
6.4.2	Communication overhead	69
6.4.3	Exchange overhead between CPU and GPUs	73
6.4.4	Accuracy	78
6.5	Security Analysis	79
6.5.1	Side-Channel Attack for Intel SGX	79
6.5.2	Gradients attack for Federated Learning	80
7	Conclusion	82
7.1	Future Work	83
	References	85

Chapter 1

Introduction

1.1 Background

The success of ML in artificial intelligence applications, such as recommendation systems, image recognition, and natural language processing, has led to a significant increase in the volume of available datasets. In the current landscape, expansive dataset repositories and the pervasive influence of ML applications across various domains—such as healthcare [88], autonomous vehicle systems [58], and financial markets [62, 85]—pose considerable privacy threats when datasets are disclosed. FL facilitates collaborative model training by transmitting the gradients of local datasets, rather than raw data, to a central server for updating Deep Learning (DL) model weights. This approach preserves data privacy and distributes computational load across decentralized devices.

Despite its promise, FL is vulnerable to gradient-related adversarial attacks, potentially exposing sensitive information [11, 42, 105]. To mitigate these vulnerabilities, several defense mechanisms have been proposed, including MPC, Differential Privacy (DP), and Trusted Execution Environments (TEEs). While MPC can mitigate threats through secret-sharing schemes without revealing the inputs of computations, it is burdened by significant communication overhead and computational complexity, particularly for non-linear operations involving comparison operations such as activation functions in DL [50, 100]. The secure MPC-based training of large models can be up to $10^4 \times$ slower than standard plain-text computation [100]. Other secure computation techniques, such as Homomorphic Encryption (HE), provides varying degrees of

security and privacy but also suffer from computational overhead, impeding their practical deployment for real-world DL applications' workloads.

Given the significant overhead associated with cryptographic computations, a crucial question emerges: Can these computations be performed in plain-text to eliminate the overhead while preserving security? This thesis addresses this question by utilizing Intel SGX to enable secure plain-text computation, thereby overcoming the efficiency and practicality challenges inherent in existing secure computation methods.

1.1.1 Piranha: a secure GPUs computing platform

Training large-scale ML models using secure MPC typically requires substantial time and computational resources. To address this challenge, Piranha [103] introduces a general-purpose, GPUs-accelerated framework designed to enhance the efficiency of MPC computations, enabling the training of realistic neural networks within roughly one day. However, despite these advancements, the computational efficiency of secret-sharing-based approaches remains significantly lower than that of plain-text computations due to the inherent complexity of secure protocols. Moreover, Piranha continues to face considerable communication overhead, which constrains its scalability. As the number of computing clients increases, memory requirements escalate rapidly, further intensifying the challenges of deploying MPC in real-world, large-scale applications.

In contrast, plain-text ML leverages direct floating-point arithmetic while maintaining manageable memory usage, aligning well with the architecture of GPUs. However, MPC frameworks such as Piranha primarily operate over integer-based data types to ensure security, underutilizing the floating-point acceleration capabilities of GPUs. This disparity underscores a fundamental inefficiency in secure ML training, as GPUs are inherently optimized for floating-point computations.

The objective of this master's thesis is to develop secure computation infrastructures capable of preserving floating-point arithmetic operations within MPC frameworks and to benchmark its performance against Piranha's existing architecture.

1.2 Problems

Deploying and supporting real-world secure MPC ML applications pose overhead challenges. Traditional FL is susceptible to security breaches, while MPC incurs substantial communication overhead, particularly for non-linear operations. Our proposed SGXDL and HybridSGXDL frameworks aim to address these issues by providing efficient and secure solutions, leveraging Intel SGX for plain-text computation.

1.2.1 Privacy Challenges in FL

Although FL promotes a privacy-aware framework, it remains vulnerable to inference attacks both inside and outside the system, compromising data privacy and system integrity [68, 70, 79]. The gradients sent to the server during the FL process can unintentionally reveal sensitive information [38, 97, 104], allowing attackers to infer private data through modified inputs. This presents the privacy risk in scenarios where malicious actors intentionally misrepresent data. For instance, during adversarial CNNs training, if the attacker intends to declare an image belonging to Alice as belonging to Eve, the victim device may continuously contribute Alice’s information to correct the model, leading to unintended data leakage. This underscores the critical need for advanced privacy-preserving techniques in FL to overcome such unauthorized access and protect individual user privacy. Techniques such as MPC, DP, and HE are vital to enhancing the privacy and integrity of the FL ecosystem. Effective privacy-preserving strategies are crucial to protect individual users’ data and maintain the overall trustworthiness of the FL framework [11, 83, 116].

1.2.2 Overhead Challenges in Secure FL techniques

A lot of work has been done to integrate the encrypted privacy-preserving methods into FL to enhance privacy. However, HE and MPC are not applicable for large-scale FL in real-world applications, because the encrypted computation without revealing the insights of data incur heavy communication and computation overhead. While DP provides a solution with reasonable overhead, it requires the aggregated value to contain noise up to a certain magnitude at aggregation-based tasks, which is not ideal for FL [68]. Additionally, DP is also facing the challenge of explainability, trackability, accuracy drop, and vulnerability to attacks, for example, complicating the

ability to monitor specific information such as fraudulent activities and adjust models accordingly [20]. Thus, developing effective and secure FL solutions which address these limitations is imperative.

1.2.3 Expanded Memory Requirements and Workload Growth Challenges

MPC protocols typically accommodate between one to four participants. As the number of computing parties and the complexity of the CNN model increase, the memory requirements and computational workloads of shares grow quickly. This escalation poses significant challenges to the feasibility of deploying MPC protocols in practical, large-scale applications. The Piranha platform serves as a notable example of these scalability challenges. During a 3-party setup for training AlexNet on the MNIST dataset with a batch size of 300, the platform experienced a memory overflow on a 24GB GPUs. This unavailability of modest task for most DL applications, underscores the substantial scalability issues faced by current MPC protocols in managing large-scale operations.

To address the scalability challenges in MPC, integrating TEEs is crucial. TEEs can replace segments of the encrypted computation workload with plain-text computations while preserving the security benefits of MPC. This integration can significantly reduce memory requirements to a reasonable level, making large-scale secure computation deployments more feasible.

1.2.4 Non-linear Operation Overhead Challenge in MPC

MPC is an effective technique to bolster privacy and security in FL, and enable secure computation between multiple parties without disclosing inputs and outputs, which inspire the research of applying MPC in FL [36]. However, MPC's encrypted computation methods incur significant communication overhead for non-linear operations involving comparisons because of more data share exchanges [64]. This overhead can further ruin processing speed due to network bandwidth constraints and synchronization, causing GPUs to remain underutilized as they await required data tokens from other devices. To improve the performance of MPC-based FL in real-world scenarios while maintaining security, optimizing non-linear operations in CNNs is the important target in this thesis to address.

1.3 Methodology

1.3.1 Multi-Party Computation based FL

MPC have emerged as a promising technique to enhance security in FL [12, 33, 48, 49, 93, 113]. MPC enable participants to perform joint computations and gain insights from data based on secret shares without requiring decrypting process. Crucially, the shares each party hold are random and meaningless. To enhance this architecture, we aim to integrate Intel SGX to reduce the computational overhead of a MPC-based FL framework. This approach will streamline computations, ensuring efficient, secure processing without compromising the confidentiality integral to MPC.

1.3.2 Trusted Execution Environments(TEEs) - Intel SGX

Despite the benefits of MPC, it often suffers from significant communication overhead, particularly with large datasets and complex ML models. TEEs, such as Intel SGX, offer a robust solution for ensuring privacy-preserving execution of plain-text computations [17]. Intel SGX provide a secure enclave for executing safety-critical operations and sensitive functions without exposure to external entities [30]. Intel SGX enable physical isolation on the Central Processing Unit (CPU) to securely process privacy data and code within an enclave [45]. The data and functions within the SGX enclave remain inaccessible to any external entity including the server that holds the TEEs, ensuring that computations can be performed securely and confidentially. By utilizing TEEs, it is feasible to replace the computationally intensive overhead operations in MPC with plain-text computations, maintaining accuracy and efficiency compared to other privacy-preserving techniques such as DP.

1.4 Solutions

To address the significant overhead issues associated with MPC, we propose two secure computing infrastructures SGXDL and HybridSGXDL. These infrastructures facilitate the deployment of secure and efficient DL applications among clients in real-world scenarios without compromising accuracy. By balancing the trade-off between efficiency and security, our approach aims to significantly reduce computational workload while maintaining the security of the system. The core idea

is to replace secret-sharing computation with plain-text computation into Intel SGX enclaves, eliminating the need for encrypted computation and reducing computational workload.

While Intel SGX provides a secure environment for computation, it is not sufficient for a fully secure collaborative computing framework. Secure channel is required at initialization to ensure data security outside of Intel SGX and during the communication process.

1.4.1 Intel-SGX Plain-text Secure Deep Learning(SGXDL)

The size of SGX enclaves has significantly increased from the previous 257MB to 64GB and beyond. This expansion allows current server enclaves to accommodate the large DL models and training in plain-text, effectively reducing communication overhead and operation complexity in MPC. The primary idea of Intel-SGX Plain-text Secure Deep Learning (SGXDL) is to perform all computation operations within the enclaves. SGXDL infrastructure conducts the entire plain-text training process inside the enclave. This approach eliminates the requirement of communication between enclave and outside environment, without Out Calls (OCalls) during training, thereby mitigating the risk of side-channel attacks.

1.4.2 Hybrid Intel SGX-GPUs Secure Deep Learning(HybridSGXDL)

While the proposed SGXDL infrastructure is effective, performing the entire training process within the enclave on a CPU limits the use of GPUs acceleration. Inspired by Piranha, which utilizes GPUs to accelerate MPC DL applications, we propose a heterogeneous Intel SGX-GPUs computing infrastructure to further improve the efficiency of secure computing by leveraging GPUs. This infrastructure distributes computation workloads between the server's Intel SGX and local GPUs.

We introduce and implement the privacy-preserving infrastructure Hybrid Intel SGX-GPUs Secure DL Framework (HybridSGXDL) to accelerate secure computing by using GPUs. This framework incorporates efficient secure DL schemes where data samples are distributed among parties using a secret-sharing scheme [6] and GPUs to accelerate the computations.

In HybridSGXDL, computing operations are divided into arithmetic and non-linear modules. Arithmetic operations, which are less computationally intensive for MPC (e.g. addition and multiplication), are executed locally on clients' GPUs in secret-sharing. Intel SGX serves as a trusted third party to integrate these secret-sharing arithmetic computations results from edge parties and compute non-linear operations such as Rectified Linear Unit (ReLU) and MaxPool in plain-text within Intel SGX.

The secret-sharing non-linear intermediate results are then returned from Intel SGX to the edge devices for the next set of secret-sharing arithmetic operations in a recursive manner. The backward propagation process also follows the same non-linear and arithmetic operation separation rules. The updated gradients are also separated into random shares and distributed back to the corresponding devices for the update, maintaining the integrity and privacy of the data throughout the process.

Table 1.4.1: Target Efficiency Benchmark Analysis

Computing Schemes	Platforms
Proposed secure computing infrastructures	SGXDL, HybridSGXDL
Cutting edge secure computation platform	Piranha
Plain-text computation	CPU, GPU

1.5 Purpose

The purposes of the research are to:

- Efficient and secure DL infrastructures that replace encrypted computation with secure plain-text computation in Intel SGX(HybridSGXDL and SGXDL).
- Explore the feasibility of conducting the entire training process of plain-text DL models within an enclave(SGXDL).
- Design a scheme that separates arithmetic operations on local devices' GPUs using a secret-sharing approach while performing non-linear operations in TEEs.
- Implement secure and efficient DL platforms capable of running LeNet, AlexNet, ResNet18, and VGG16. These platforms will be implemented in pure C style, making them compatible with most types of servers.

1.6 Dataset Description

In the thesis MNIST and Cifar10 are used for experiment to demonstrate the efficiency performance of SGXDL and HybridSGXDL.

1.6.1 MNIST Dataset

The MNIST (Modified National Institute of Standards and Technology) dataset is a classic benchmark used for image classification, ML, and DL research. It consists of 60,000 grayscale images of handwritten digits, each sized at 28×28 pixels, with labels ranging from 0 to 9, representing 10 classes in total. Due to its simplicity and well-defined structure, the MNIST dataset is widely employed to evaluate the performance of classification algorithms and the convergence speed of DL models. Its moderate size and minimal preprocessing requirements make it an essential standard dataset in the field [60].

1.6.2 Cifar10 Dataset

The CIFAR-10 (Canadian Institute For Advanced Research) dataset is a more complex collection of color images that includes 10 different object classes. Each class contains 6,000 images, leading to a total of 60,000 images with a resolution of 32×32 pixels. Of these, 50,000 images are designated for training, and 10,000 for testing. The dataset encompasses a diverse set of object categories, such as airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Due to its diversity and the presence of colored images, CIFAR-10 is often used as a challenging benchmark for image classification tasks [54].

1.7 Hypothesis, Environment and Threat Model

1.7.1 Hypothesis

- The non-linear operations in the thesis refer to the operation involving comparison operations, e.g. MaxPool and ReLU. The arithmetic operations in the thesis are addition and multiplication.
- The number of computing parties $n \in \mathbf{N}$ is not theoretically limited and can be

1,2,3,4,5,6,7....etc in SGXDL. For HybridSGXDL, we only implement the 3 party scenarios to demonstrate the feasibility of separating non-linear operations into enclave.

- All information is in a secret-sharing format outside the Intel SGX enclave.
- There will be only one server holding the SGX, and other parties will send the secret shares to the SGX server. SGX is the only trusted entity, and all parties including the server are assumed to be semi-honest.
- There will be only communications between the SGX server and clients, with no communication between clients in SGXDL.
- In HybridSGXDL there are communications among servers and clients.
- Network setting: LAN 2GB/s with 2ms latency.
- The FL in the thesis will apply the weighted FL format as described in [117].

1.7.2 Environment

Development Environment

Operating System: Ubuntu 22.04

Default Maximum Enclave Page Cache (EPC) Size for Intel SGX:64 GB

GPUs: Nvidia RTX A5000, memory of 24GB

Architecture: x86_64

CPU op-mode(s): 32-bit, 64-bit

Address sizes: 46 bits physical, 57 bits virtual

Byte Order: Little Endian

CPU(s): 144

On-line CPU(s) list: 0-143

Vendor ID: GenuineIntel

Model name: Intel(R) Xeon(R) Platinum 8360Y CPU @ 2.40GHz

CPU family: 6

Model: 106

Thread(s) per core: 2

Core(s) per socket: 36

Socket(s): 2

Stepping: 6

CPU max MHz: 3500,0000

CPU min MHz: 800,0000

BogoMIPS: 4800.00

Build tools

Intel® SGX Linux 2.9 release

CUDA 11.7

GCC 13.1

C++ 17

1.7.3 Threat Model

The thesis considers a semi-honest threat model (i.e. honest-but-curious), where attackers may corrupt parties but still obey the protocol. In other words, the corrupted parties participate in FL honestly but attempt to extract as much information as possible from the messages they receive from other parties and the SGX. The semi-honest adversaries may try to learn information from the honest parties and exploit the established communication channels of the protocol.

In SGXDL, the communication is exclusively between the server's SGX and the clients. The key point is to establish a secure channel between the SGX and the clients, ensuring that the server party, which mainly handles the shares, cannot learn or retrieve the information exchanged. For HybridSGXDL, there are communications among clients and server, additional secure channels are required to protect the communication among clients. For more rigorous scenarios involving malicious environments, additional monitoring and detection mechanisms are required, but these are beyond the scope of this thesis.

SGXDL security targets

The purpose of SGXDL is to safeguard the model structures, parameters, and raw data, including intermediate products, during the training process. To extract useful information by integrating all shares, adversaries would need to simultaneously compromise all participating parties. Consequently, as the number of participating computing parties increases, the overall security is strengthened.

HybridSGXDL security targets

The primary security objective of HybridSGXDL is to ensure the confidentiality of raw data and model parameters across all participating parties, effectively preventing the server from accessing plaintext information. This is in contrast to traditional FL models, where the server typically receives gradient values from all participants.

Even in the most adverse scenarios—such as when secure communication channels are entirely compromised and the server is capable of reconstructing the shares—only intermediate results might be exposed. The raw data remains protected, as these intermediate results are composite products of both addition and multiplication during

local convolution, making it difficult to reconstruct the original raw data. This level of security is considered sufficient to meet the objectives of HybridSGXDL.

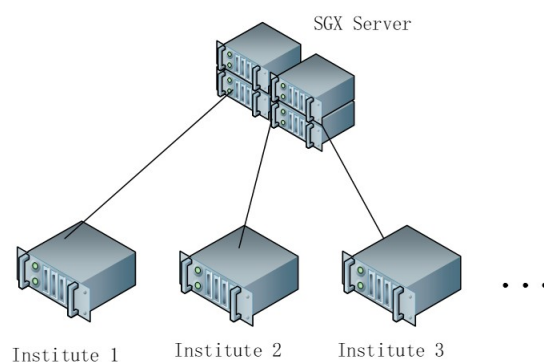


Figure 1.7.1: Communication Connection Model(SGXDL)

Chapter 2

Theoretical Background

This chapter provides a comprehensive overview of FL, focusing on wFL which is a MPC secret-sharing-based FL frame, addresses the security challenges in traditional FL. Furthermore, the chapter elaborates the details of the DL operations used in the thesis, such as convolution and ReLU, which are essential for the deployment within Intel SGX using limited C standard libraries. Understanding these operations is crucial for grasping how secret-sharing computations for convolution, accelerated by GPUs on local devices, can be integrated into the HybridSGXDL framework.

Intel SGX functionalities are explored, including interface design and implementation. Even though data is protected through secret random shares in this thesis, there remains a risk of the server accessing all shares. To mitigate this risk, a primary secure communication channel between SGX and client parties is introduced in the chapter to prevent the shares from server's access.

2.1 Secret-Sharing based MPC

Secret-sharing scheme is a method to safely share data between numerous gatherings, provides superior performance for arithmetic operations such as matrix addition and multiplication over other cryptographic tools, and has been extensively used for privacy-preserving CNNs' inference and training [13, 14, 99].

2.1.1 Additive Secret Sharing

Additive Secret-Sharing (ASS) and Shamir secret-sharing [87] are two widely used schemes. Shamir [87] proposed a (n,t) -threshold scheme based polynomials for the secret share that the raw data is divided into n shares, and (1) any t shares can retrieve the raw data (2) any $t-1$ shares can not retrieve information. And ASS is a (n,n) -threshold secret-sharing scheme that raw data is divided into n shares, and can only be retrieved with n shares. In ASS, the raw data x will be randomly divided into $[x_0, x_1, x_2, \dots, x_{n-1}]$ on a finite field such that $\sum_{i=0}^{n-1} x_i = x$. Many ASS MPC protocols have been widely used for privacy-preserving computation in recent days [8, 9, 21, 37, 63, 110]. This thesis also applies ASS MPC learning frameworks over the real number field $\mathbf{R}$ for floating-point arithmetic operations.

2.1.2 Multi-Party Computation(MPC)

The security community has a long accepted common sense that any security systems are born with their corresponding attack surface, it is nearly impossible to design a fully secure complex system [27], adversaries will be able to penetrate and stealthily take control over some of the network nodes. To enhance security, it is crucial to minimize the attack surface and maximize the cost of attacks.

MPC is introduced by Andrew Yao [111] in 1982. Yao gave a precise formulation for MPC problems (e.g. Two millionaires wish to know who is richer without disclosing the information of wealth values). The problem can be modified as m parties wish to jointly compute a function $f(x_0, x_1, \dots, x_{m-1})$ where x_i is the i^{th} party's private input of integers with bounded range. Several well-known MPC protocols have been developed, including Yao's Garbled Circuits, GMW [35, 37], BGW [7], BMR [5], and GESS [52].

MPC enhances privacy in ML by allowing joint data processing without exposing individual inputs. However, the significant communication and computation overhead makes it impractical for large-scale data and models. Hybrid solutions combining MPC with HE have been proposed by researchers [32, 84]. To improve efficiency, TEEs like Intel SGX are used to assist MPC, balancing encrypted computation and hardware security [22, 107]. Given the substantial computational cost of non-linear operations in MPC, HybridSGXDL leverages SGX to execute non-linear operations on plain-text, thereby minimizing overhead.

2.2 Federated Learning

2.2.1 History of Secret-Sharing Federated Learning

The success of CNNs [59], advancements in Transformer attention mechanisms [98], and the emergence of Large Language Models (LLMs) like ChatGPT [1] have highlighted the potential of DL in artificial intelligence. The availability of big data has accelerated the adoption of DL in various fields such as healthcare, finance, autonomous systems, and e-commerce. However, individual institutions often lack sufficient data, necessitating collaborative training with datasets from other parties. Directly sharing and integrating datasets poses significant privacy risks.

FL introduced by Google in 2016 [75], addresses the privacy concerns by allowing clients to train models locally and send model updates to a server for integration. However, FL frameworks are vulnerable to gradient-based attacks [69]. Consequently, secret-sharing-based FL emerged to enhance security by distributing data in secret shares, ensuring data remains encrypted and random, revealing very least information unless all shares are integrated [29]. This makes it significantly challenging for attackers, as they must compromise multiple parties simultaneously to obtain useful information.

There is a substantial body of work focusing on using secret-sharing schemes to assist FL. Bonawitz et al. [10] presented a privacy-preserving MPC protocol using Shamir's t -out-of- n secret-sharing [87] for secure gradient aggregation in FL. Zhang et al. [114] combined threshold secret sharing with HE to address similar challenges. Y. Dong et al. [23] applied secret-sharing for distributed learning. Jinhyun So et al. [92] proposed Turbo, which masks models at local clients with randomness using additive secret-sharing schemes, canceling out noise during aggregation. Secret-sharing is widely used in FL [44, 66, 80, 89, 91, 109].

Typically, centralized traditional FL trusts the server is honest to receive everything. However, for stronger security, this thesis has assumed a semi-honest threat model where the server is also semi-honest and can be monitored by an attacker capable of observing its behaviors. In such cases, the Intel SGX hardware enclave ensures that all parties, including the server, cannot access the code and data within it. Even if an attacker alters the server's behavior and introduces disturbances, the FL performance may be affected, but the information accessible to the attacker remains unavailable in

the enclave.

2.2.2 Secure techniques for Federated Learning

Homomorphic Encryption (HE) [3, 66] allows FL to execute the gradients aggregation over cipher-text on server without decrypting or revealing information to the server in advance [16, 28, 40, 65, 112], which provides a strong guarantee for data privacy. However, HE supports a limited number of arithmetic operations and incurs substantial computational overhead due to complex cryptographic operations [81]. Encryption and decryption on local clients can consume up to 80% of training time in local plain-text ML model updates [112].

Local Differential Privacy (LDP) adds noise to local datasets, protecting individual data privacy [25, 51]. However, LDP introduces uncertainty, potentially reducing accuracy, thus necessitating a balance between privacy and accuracy [79].

MPC protocols provide strong security for decentralized FL by distributing different random shares to clients as introduced in Section 2.1.2. MPC incurs significant communication and computation overhead, particularly for non-linear computations like ReLU and MaxPool, which involve comparisons and conversions between arithmetic and binary secret sharing [34, 99]. ReLU latency can be 10,000 times slower than convolution operations [34].

Intel SGX is an encrypted area on the CPU that protects the code and data within the enclave from external access [15, 18, 76, 77]. Introduced in 2015, Intel SGX allows running code and storing data securely on an untrusted device [18]. It provides the option of secure floating-point plain-text computation without accuracy loss, thereby eliminating the overhead associated with encrypted computation.

2.2.3 Data Splitting in Federated Learning

Classical FL frameworks are typically categorized as horizontal, vertical, and transfer learning, based on how datasets are split. In vertical FL, the dataset is split by features with the same objects, while in horizontal FL, the dataset is split by objects with the same feature space. In this thesis, data is distributed in secret shares, and the corresponding FL approach is wFL [117], which will be introduced in the following

section.

Wagh et al. [109] have demonstrated that secret-sharing schemes offer the highest efficiency in secure DL scenarios. When individual clients have stringent privacy requirements, they may be reluctant to expose personal data to any institution, complicating data generation and training processes. Via secret-sharing, access to private data for model training or inference can be certified and agreed by individual client to solve the concern. For instance, if Alice wants to take a genetic disease test and investigate the results herself, she can send her data in three secret-sharing values to three independent institutions that have databases for DNA diagnosis [24].

2.2.4 Weighted Federated Learning

In this section, we introduce wFL and provide the description for the data splitting in wFL, under a setup of a 3-party scenario. Zhu et al. [117] proposed a wFL scheme that utilizes secret-sharing to split private data into random shares. Building on this approach, this thesis applies the wFL to private data to meet higher privacy demands. This secret-sharing MPC-based wFL method has the potential to unlock a wide range of ML applications that were previously inaccessible due to data privacy concerns.

Data Splitting in weighted federated learning

Consider a 3-party wFL scenarios with three MPC parties: P_0, P_1, P_2 , the private data x . In Zhu's work x is defined in a finite integer field Z_p^* (where p is a suitable large prime number e.g. $|p| = 512$). State-of-the-art secret-sharing schemes are also defined on finite fields and rings of integers to improve security that attackers can not learn information from the magnitude of shares.

Assume there is a secure channel between Alice and the MPC servers, implemented under the standard Public Key Infrastructure (PKI) assumption [2], ensuring that the secret shares are securely distributed during initialization. The splitting procedure is defined below

- Alice selects $x_0, x_1 \in Z_p^*$ uniformly at random, and then sends x_0 to P_0 , x_1 to P_1 ;
- Alice computes $x_2 = x - x_0 - x_1 \bmod p$ and sends x_2 to P_2 .

Given the differences in addition and multiplication operations in our wFL implementation within the Intel SGX-assisted computing structures, we have omitted

the resharing, addition, and multiplication of wFL in this section. The details of data share splitting and arithmetic computations for HybridSGXDL and SGXDL will be presented in the following sections.

2.3 Data shares representation in SGXDL and HybridSGXDL

Building on the ideas from wFL, we incorporate secret-sharing into SGXDL and HybridSGXDL. However, to fully leverage Intel SGX’s ability to perform plain-text computations, we also aim to apply floating-point real number secret shares in our proposed computing structures, whereas most secret-sharing protocols typically rely on uniform distributions to generate discrete integer shares. For example under a 3 party setup scenario, data x is defined in a floating arithmetic field such that x is divided into 3 secret shares: $x_0, x_1, x_2 \in \mathbf{R}$. We do not set a bounded range in theory because such bounds can lead to information leakage. For example, if $x_0, x_1, x_2 \in (-100, 100)$, $x_0 = -56$, $x_1 = -43$, it can be inferred that $x < 1$. Applying state-of-the-art cryptographic integer computation would involve overhead of conversions between integers and floating-point numbers. Therefore, a real number secret-sharing scheme is needed for secure plain-text computation in the thesis.

K. Tjell et al. [96] introduced a real-number secret-sharing scheme utilizing Gaussian distribution, which eliminates the need for integer shares and modular arithmetic, allowing for direct arithmetic computations on the shares. Inspired by K. Tjell’s approach, we also employ Gaussian distribution to provide randomness and add noise to real number shares. Gaussian noise is widely used to obfuscate sensitive information in many privacy-preserving mechanisms, such as DP. Applying Gaussian distribution to generate shares in secret sharing can provide similar protective effects, enhancing resistance to attacks and reducing the risk of secrets being inferred.

However, generating secret shares using Gaussian distribution introduces challenges, including floating-point precision issues and vulnerability to statistical attacks. Given that this thesis focuses on exploring the feasibility and efficiency of Intel SGX in enabling secure and efficient plain-text ML, additional security mechanisms are primarily aimed at providing moderate security protection. While we acknowledge the limitations and security risks associated with this approach, we do not delve into

the theoretical underpinnings of Gaussian-distributed secret sharing in detail here. Future research could explore optimizing the share generation mechanism, such as adopting discrete Gaussian distributions or hybrid distribution schemes, to improve both security and robustness.

2.3.1 The random data shares generation

SGXDL and HybridSGXDL use the same data shares generation schemes. Under the n party scenario setup, assume the data $x \in \mathbf{R}$ held by Alice and she wants to share the data to n parties $P_0, P_1, \dots, P_{n-1}$. The procedure of generating data shares by Alice locally is:

- Alice selects $x_0, x_1, x_2, \dots, x_{n-2} \in \mathbf{R}$ from Gaussian distributions with zero mean value at random. Such that $x_0 \sim \mathcal{N}(0, \sigma_{x_0}^2), x_1 \sim \mathcal{N}(0, \sigma_{x_1}^2), \dots, x_{n-2} \sim \mathcal{N}(0, \sigma_{x_{n-2}}^2)$.
- Alice computes $x_{n-1} = x - \sum_{i=0}^{n-2} x_i$.

The splitting of data x is defined by $[x] = (x_0, x_1, \dots, x_{n-1}), x = \sum_{i=0}^{n-1} x_i$. The choice of variance in Gaussian noise does not compromise the accuracy of reconstructing shares to the original data in HybridSGXDL and SGXDL. Thus, from a privacy protection standpoint, a larger variance is generally advantageous. If variance tends to infinity, the information of x from x_0, x_1 and x_2 tends to 0. However, when determining the appropriate level of variance, it is crucial to account for the sensitivity of the data, the desired level of privacy protection, and the accuracy requirements of the application. We assume that Alice has selected sufficiently large $\sigma_{x_i}^2$ to safeguard data privacy, while carefully balancing the trade-off between privacy preservation and data accuracy.

2.3.2 SGXDL data shares generation and distribution

Since communication can occur concurrently during training, the number of parties has minimal impact on the overhead in SGXDL. Adding additional parties to SGXDL is straightforward, allowing us to consider configurations with more than three parties. In SGXDL, data shares are generated as described in Section 2.3.1. The holding condition for the data shares of a data sample x and the computation operand $w \in \mathbf{R}$ (in CNNs w is always the model weights stored in the enclave):

P_i holds x_i . The Intel SGX enclave of P_0 holds w

e.g. under a 3 party set-up, the data shares distribution is :

- P_0 holds x_0 .
- P_1 holds x_1 .
- P_2 holds x_2 .
- The Intel SGX enclave of P_0 holds w .

P_0 is the server having Intel SGX.

Random shares generation in SGXDL

- Random numbers $x_0, x_1 \in \mathbf{R}$ are selected from Gaussian distributed with mean value zero. Such that $x_0 \sim \mathcal{N}(0, \sigma_{x_0}^2), x_1 \sim \mathcal{N}(0, \sigma_{x_1}^2)$.
- $x_2 = x - x_0 - x_1$.

Addition of x and w in SGXDL In SGXDL the data shares are distributed among the clients and the SGX server, the procedure is:

- P_1, P_2 sends x_1, x_2 to the Intel SGX enclave of P_0
- In the Intel SGX enclave of P_0 , retrieve $x = x_0 + x_1 + x_2$.
- In the Intel SGX enclave of P_0 , compute $x + w$

Multiplication of x and w in SGXDL In SGXDL the data shares are distributed among the clients and the SGX server, the procedure is:

- P_1, P_2 sends x_1, x_2 to the Intel SGX enclave of P_0
- In the Intel SGX enclave of P_0 , retrieve $x = x_0 + x_1 + x_2$.
- In the Intel SGX enclave of P_0 , compute $x * w$

When the computation involving $x + w$, $x * w$ and non-linear operations need to be performed, the independent shares x_0, x_1, x_2 are securely transmitted to the SGX enclave. Within the enclave, these distributed shares are combined to reconstruct the complete secret value x . This marks a transition in the representation of x from

distributed shares outside the enclave to a reconstructed global secret inside the enclave. Once x has been reconstructed inside the SGX enclave, it can be securely combined with w to perform the complete CNNs training computation without extra communication during the training, including addition and multiplication, and non-linear operations

2.3.3 HybridSGXDL data shares generation and distribution

In CNNs, the fundamental arithmetic operations are addition and multiplication, with no division involved. Non-linear operations involving comparison operations, such as activation functions, are executed in plain-text within the enclave. As a result, non-linear functions like ReLU and MaxPool are not covered in the context of secret-sharing schemes in the subsequent modules. The computation of addition and multiplication does not engage the Intel SGX enclave. In the thesis HybridSGXDL only consider and implement 3 party scenarios. In the future, parties > 3 can be explored for HybridSGXDL based on corresponding secret-sharing schemes.

In HybridSGXDL, initially, a secret data sample value x is divided into multiple shares x_1, x_2, x_3 , computation operand $w \in \mathbf{R}$ is divided into shares w_0, w_1, w_2 , with each share being held and processed by different local computation nodes. Each node performs local multiplications on these shares to compute the corresponding local multiplication results z_0, z_1, z_2 . At this stage, the data remains in the form of a locally shared representation, ensuring that each node only holds partial information about the original secret value.

Subsequently, these local multiplication results z_0, z_1, z_2 are transmitted to the SGX. A significant change in data representation occurs at this point: within the SGX, the local shares are securely combined and reconstructed to form the global secret value z . Here, the data representation transitions from being "locally independent shares" at each node to a "fully reconstructed secret" within the SGX, thus preserving the privacy of the data while ensuring the correctness of the computed results. Under a 3 party set-up, the shares of x are distributed as follows in HybridSGXDL:

- P_0 holds $(x_0, x_2), (w_0, w_2)$.
- P_1 holds $(x_0, x_1), (w_0, w_1)$.
- P_2 holds $(x_1, x_2), (w_1, w_2)$.

Here, we describe the shares distribution in HybridSGXDL.

Random shares generation in HybridSGXDL The values of $x_0, x_1, x_2, w_0, w_1, w_2$ are set as:

- Select random numbers $x_0, x_1, w_0, w_1 \in \mathbf{R}$ where each is Gaussian distributed with mean value zero. Such that $x_0 \sim \mathcal{N}(0, \sigma_{x_0}^2), x_1 \sim \mathcal{N}(0, \sigma_{x_1}^2), w_0 \sim \mathcal{N}(0, \sigma_{w_0}^2), w_1 \sim \mathcal{N}(0, \sigma_{w_1}^2)$.
- $x_2 = x - x_0 - x_1, w_2 = w - w_0 - w_1$.

Addition of x and w in HybridSGXDL, $z=x+w$

- P_0 make the computation $z_0 = x_0 + w_0$
- P_1 make the computation $z_1 = x_1 + w_1$
- P_2 make the computation $z_2 = x_2 + w_2$
- Finally, z_0, z_1 and z_2 can be sent to SGX to get the result: $z = z_0 + z_1 + z_2 = x + w = (x_0 + x_1 + x_2) + (w_0 + w_1 + w_2)$.

Multiplication of x and w in HybridSGXDL, $z=x*w$

- P_0 make the computation $z_0 = x_0 * w_0 + x_0 * w_2 + x_2 * w_0$
- P_1 make the computation $z_1 = x_1 * w_1 + x_1 * w_0 + x_0 * w_1$
- P_2 make the computation $z_2 = x_2 * w_2 + x_2 * w_1 + x_1 * w_2$
- Finally, z_0, z_1 , and z_2 can be sent to SGX to get the result: $z = z_0 + z_1 + z_2 = x * w = (x_0 + x_1 + x_2) * (w_0 + w_1 + w_2)$.

In the worst-case scenario where an attacker fully compromises the secure channels, they might be able to observe z_0, z_1 and z_2 . And if $z_0 + z_1 + z_2 = 0 = x * w$ holds, then attacker could deduce x or w is 0. However, in the thesis, such leakage is considered to pose no significant risk because the intermediate products of HybridSGXDL exchanged during communication are not the result of single multiplication or addition operations. Instead, they are composite results of a convolution process, which inherently involves both multiplication and addition. Although there is a risk that information about these intermediate products could be exposed during training once the secure channels were compromised, it remains insufficient for reconstructing

the original raw data involved in the computation. The primary goal of the secure computing infrastructure outlined in the thesis is to protect this raw data from being compromised by external parties or adversaries.

2.3.4 Lack of Perfect Security for the shares generation

In large sample tests, the shared values generated using a Gaussian distribution may exhibit distribution characteristics that could enable attackers to infer partial secret information. Consequently, this scheme does not fully satisfy the strict criteria for 'perfect secret sharing'.

Despite these limitations under rigorous security standards, the Gaussian-based secret sharing scheme offers notable benefits in terms of computational efficiency and ease of implementation. Therefore, it is well-suited for applications with moderate security requirements. Future research could focus on integrating alternative distribution models or refining algorithmic designs to strengthen the scheme's security.

2.4 MPC protocols in Piranha

Due to the differing security mechanisms employed by HybridSGXDL and SGXDL compared to Piranha, a direct efficiency comparison may raise concerns regarding the validity of HybridSGXDL and SGXDL's results. Nevertheless, HybridSGXDL and SGXDL provide a degree of security while establishing the feasibility of running and accelerating DL models within TEEs. A more detailed analysis of the underlying security aspects in future work could help identify appropriate scenarios for making informed trade-offs between security and efficiency.

In this study, we benchmarked the performance of HybridSGXDL and SGXDL against the Piranha framework, which implements MPC protocols tailored to different numbers of participating parties:

Piranha (2PC): This two-party computation (2PC) protocol is based on the SecureML framework [78], employing a 2-out-of-2 arithmetic secret-sharing scheme. Linear layers are executed using Beaver triples, while non-linear layers are computed with garbled circuits. Specifically, Piranha (2PC) leverages the edaBits technique for evaluating ReLU activations [26, 72].

Piranha (3PC): This three-party computation (3PC) protocol builds upon the Falcon framework [100], using a 2-out-of-3 replicated secret-sharing scheme. Linear layers are processed through local multiplications and resharing, while non-linear layers requiring comparison operations are implemented using the techniques proposed by Makri et al. [72].

Piranha (4PC): The four-party computation (4PC) implementation follows the methodology presented by Dalskov et al. [19], utilizing a 3-out-of-4 replicated secret-sharing scheme. Linear layers are computed using a combination of local multiplications and resharing, while non-linear operations involving comparisons rely on protocols combining the approaches developed by Barak et al. [4] and Koti et al. [53].

In all Piranha protocols, secret-sharing is carried out over finite fields, ensuring robust security for privacy-preserving computation. In contrast, HybridSGXDL and SGXDL generate secret shares using floating-point values sampled from Gaussian distributions, operating in a floating-point real field. This distinction in the underlying secret-sharing mechanism could imply a different, and potentially weaker, level of security compared to Piranha’s finite-field-based methods.

Therefore, in this thesis, Piranha serves as an efficiency reference to demonstrate the performance capabilities of HybridSGXDL and SGXDL while emphasizing the implications of their differing security models. Future research can explore these trade-offs more thoroughly to identify suitable contexts for the effective deployment of each approach.

2.5 Deep Learning

In this chapter, we introduce the history of DL, specifically focusing on CNNs. We then delve into the detailed implementation of DL models for the SGXDL and HybridSGXDL frameworks. Given the absence of existing CNNs C projects suitable for Intel SGX computation, we undertook the significant task of manually writing the code using standard C libraries, involving extensive manual work and mathematical induction. This chapter aims to highlight the implementation details essential for reproducing the results and understanding the code presented in the Appendix of this thesis.

2.5.1 Overview of Deep Learning

The idea of artificial neurons was introduced by Warren McCulloch and Walter Pitts in 1943, based on electrical circuits[73]. Since then, many researchers have advanced the field of neural networks, including Frank Rosenblatt[86], who developed practical applications like the Perceptron[86]. Research on theoretical aspects, including the backpropagation (invented in the 1970s and popularized in the 1980s)[106], enabled the training of multi-layer neural networks[95].

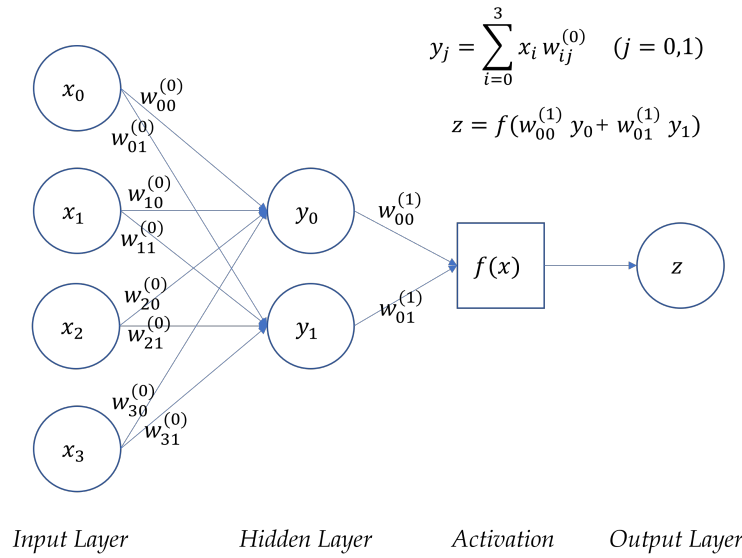


Figure 2.5.1: Simple Neural Network Structure

During the 1990s and early 2000s, AI experienced a "winter" period due to data scarcity, overfitting, gradient vanishing in deeper layers, and limited computational power. However, from 2015 onwards, the availability of GPUs[71] and large datasets facilitated more practical applications, leading to significant breakthroughs in DL structures.

One of the important structures that improved the efficiency and normalization ability of neural networks is the convolutional layer, proposed by LeCun et al.[59] and successfully applied in LeNet. The convolutional layer became a fundamental component of modern DL, contributing to major successes in image recognition competitions, such as AlexNet [55], which won the ImageNet Competition in 2012.

The emergences of advanced techniques like ResNet[41], DenseNet[43], and Transformer[98] models brought significant breakthroughs in DL performance.

Especially Transformer advancements have enabled multi-modal applications and impressive applications in natural language processing(NLP) like ChatGPT[1]. Today, DL is recognized as one of the most powerful artificial intelligence paradigms, driving a wide range of applications in healthcare, finance, customer service, autonomous systems, and more.

2.5.2 Training phase of Deep Learning

CNNs involve a series of repeated operations, such as convolutions followed by non-linear operations. The training phase in DL is computationally intensive, requiring the model to process and learn all the samples in the large datasets comprehensively, and identify the optimal optimization extreme for model parameters' values. We implemented the pure C-style DL models based on Intel SGX's standard C libraries, and the learning rate sets to fix the vanish gradients can see Appendix A.2.

2.5.3 Arithmetic

The addition and multiplication computations are arithmetic operations. In the context of CNNs, these operations are equivalent to the matrix multiplication and accumulation processes.

2D Convolution

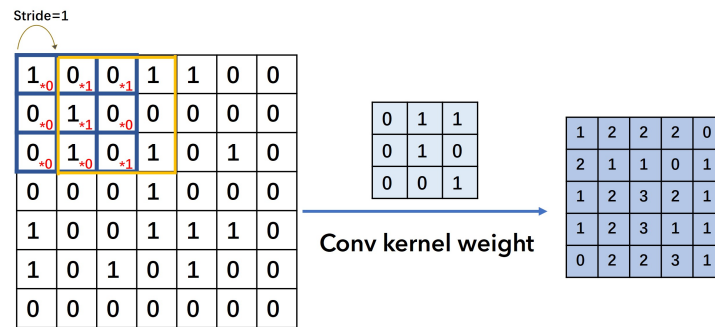


Figure 2.5.2: Convolution

Forward function, the convolution $Conv(w, x)$ is:

$$Conv[i_1, i_2, \dots, i_D, c_{out}] = \sum_{c_{in}=1}^{C_{in}} \sum_{k_1=1}^{K_1} \sum_{k_2=1}^{K_2} \dots \sum_{k_D=1}^{K_D} \mathbf{w}[k_1, k_2, \dots, k_D, c_{in}, c_{out}] \cdot \mathbf{x}[i_1 - k_1, \dots, i_D - k_D, c_{in}] \quad (2.1)$$

- $\mathbf{x}[i_1, i_2, \dots, i_D, c_{\text{in}}]$ is the element of the input tensor $\mathbf{x}$ at channel c_{in} , located at position $(i_1, i_2, \dots, i_D)$.
- $\mathbf{w}[k_1, k_2, \dots, k_D, c_{\text{in}}, c_{\text{out}}]$ is the weight of the convolution kernel $\mathbf{w}$ at position $(k_1, k_2, \dots, k_D)$, which connects the input channel c_{in} and the output channel c_{out} .
- D is the number of dimensions of the input tensor (e.g., 2D, 3D, 4D, etc.),
- $K_1, K_2, \dots, K_D$ are the sizes of the convolution kernel along each dimension.
- C_{in} is the number of input channels (e.g., 3 channels for an RGB image), and C_{out} is the number of output channels.

Assume the input matrix $x^{C \times I \times I} \in \mathbf{R}$, the output $y^{O \times N \times N} \in \mathbf{R}$, and a weight $w^{C \times O \times M \times M} \in \mathbf{R}$. The Stride $S = 1$, Padding $P = 0$. C is the channels magnitude of input matrix x , and O is the channels magnitude of output matrix y . The weights w are the convolution kernels of the CNN. Padding is a technique that adds empty borders to the input image to prevent a significant reduction in the output size. The stride is a parameter that specifies the number of pixels by which the filter matrix moves across the input matrix during the convolution operation.

$$N = \frac{I - M + 2 * P}{S} + 1 \quad (2.2)$$

N is the width and height of the output matrix y

$$y = \text{Conv}(w, x)$$

$$y(o, i, j) = \sum_{c=0}^{C-1} \sum_{p_1, p_2=0}^{M-1} w(c, o, p_1, p_2) * x(c, o, i + p_1, j + p_2) \quad (2.3)$$

Backward function. Assume the loss function L . Loss functions in DL are used to measure how well a neural network model performs. L is the error between the final output of a CNNs model and the given expected output value. The goal of training is to minimize the value of the loss function during the backpropagation step to make the neural network better. While L in this thesis is the softmax function [46], which is effective for multi-class classification problems. The details of softmax L in the thesis are as shown below:

Assume the n-output of CNNs: $\{o_0, o_1, \dots, o_{n-1}\}$, thesis's DL apply softmax and entropy loss as the loss function. Then the probability distribution of outputs is modified

as:

$$p_i = \frac{e^{o_i}}{\sum_{k=0}^{n-1} e^{o_k}} \quad (2.4)$$

Assume the one-hot label: $\{l_0, l_1, \dots, l_{n-1}\}$, where $l_j = 1, l_i = 0 (i \neq j)$. The Entropy Loss L is:

$$L = - \sum_{i=0}^{n-1} l_i \ln p_i \quad (2.5)$$

And the gradients L versus output o are:

$$\frac{\partial L}{\partial o_i} = \begin{cases} p_i, i \neq j \\ p_j - 1, i = j \end{cases}$$

The gradient update equations (Equation 2.6, 2.7) apply universally to other loss function such as Mean Squared Error (MSE), Logloss etc[101]. Exploring alternative loss functions in future work may enhance performance. Then the gradients of L versus w for update is:

$$\begin{aligned} \frac{\partial L}{\partial w}(c, o, :, :) &= Conv\left(\frac{\partial L}{\partial y}(o, :, :), x(c, :, :)\right) \\ \frac{\partial L}{\partial w(c, o, p_1, p_2)} &= \sum_{i=0, j=0}^{N-1} \frac{\partial L}{\partial y(o, i, j)} \frac{\partial y(o, i, j)}{\partial w(c, o, p_1, p_2)} \\ &= \sum_{i=0, j=0}^{N-1} \frac{\partial L}{\partial y(o, i, j)} x(c, i + p_1, j + p_2) \end{aligned} \quad (2.6)$$

where $\frac{\partial y(o, i, j)}{\partial w(c, o, p_1, p_2)} = x(c, i + p_1, j + p_2)$ is the input matrix value, $\frac{\partial L}{\partial y(o, i, j)}$ is the loss gradients versus convolution's estimated output $y(o, i, j)$, which can be transmitted in the back propagation process from the final layer to the previous layers. $\frac{\partial L}{\partial w}(c, o, :, :)$ denote that the gradient is calculated for the weight associated with input channel c , output channel o . $\frac{\partial L}{\partial y}(o, :, :)$ represents the gradient of the loss versus the output y of the convolution for the $o - th$ channel. $x(c, :, :)$ is the input to the convolution layer for the $c - th$ channel. We can also find that the backward of convolution for CNNs weight is also a linear convolution computation indeed. These gradients are propagated backward through the network, enabling the adjustment of weights from the final layer to the preceding layers. In a CNN, the output of one layer serves as

the input to the next layer, making it essential to compute the output at each layer in order to effectively propagate the gradients back and update the model parameters accordingly:

$$\begin{aligned} \frac{\partial L}{\partial x(c, i', j')} &= \sum_{o=0}^{O-1} \sum_{i=0, j=0}^{N-1} \frac{\partial L}{\partial y(o, i, j)} \frac{\partial y(o, i, j)}{\partial x(c, i', j')} \\ &= \sum_{o=0}^{O-1} \sum_{i=0, j=0}^{N-1} \frac{\partial L}{\partial y(o, i, j)} w(c, o, i' - i, j' - j) \end{aligned} \quad (2.7)$$

where $\frac{\partial y(o, i, j)}{\partial x(c, i', j')} = w(c, o, i' - i, j' - j)$ is the value of CNNs model's current weight w . For the input tensor x , the backpropagation of the convolution calculates the gradient of the loss function with respect to the input. The formula is similar to convolution forward, using the output gradient and the convolution kernel.

Algorithm 1 ConvolutionForward Appendix: A.1.1.

weight.width is the width of kernel, weight.height is the height of kernel, InputLayer.channels is the number of channels for the input matrix. The same applies to other parameters.

Input: 4-dimensional **input** with dimensions of [batchsize, image.channels, image.width, image.height],
4-dimensional **weight** with dimensions of [InputLayer.channels, OutputLayer.channels, weight.width, weight.height],
1-dimensional **bias**,
activation $f(x)$.

Output: **output** with dimensions of [batchsize, OutputLayer.channels, output.Width, output.Height]

Conv Computation:

```
for  $b = 1 : \text{batchsize}$  do
  for  $i = 1 : \text{InputLayer.channels}$  do
    for  $j = 1 : \text{OutputLayer.channels}$  do
      for  $o_0 = 1 : \text{output.width}, o_1 = 1 : \text{output.height}$  do
        for  $w_0 = 1 : \text{weight.width}, w_1 = 1 : \text{weight.height}$  do
          output[ $b$ ][ $j$ ][ $o_0$ ][ $o_1$ ] += input[ $b$ ][ $i$ ][ $o_0 + w_0$ ][ $o_1 + w_1$ ] * weight[ $i$ ][ $j$ ][ $w_0$ ][ $w_1$ ]
        end for
      end for
    end for
  end for
end for
```

Bias Computation:

```
for  $b=1:\text{batchsize}$  do
  for  $c=1:\text{output.count}$  do
    output[ $b$ ][ $c$ ] =  $f(\text{output}[b][c]) + \text{bias}[c]$  /* output.count is the number of elements  

    for each sample in a batch* /
  end for
end for
```

2.5.4 Non-Linear Function

ReLU

Forward Equation, where $x \in \mathbf{R}$

$$ReLU(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (2.8)$$

Backward Equation, where $x \in \mathbf{R}$ the gradient of ReLU versus the input x is:

$$\frac{\partial ReLU(x)}{\partial x} = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (2.9)$$

MaxPool

Forward Equation Assume the input matrix $x^{C \times I \times I}$ for MaxPool, and the MaxPool output of the input $y^{C \times N \times N}$, and the kernel size of MaxPool $k_{maxpool} \in \mathbf{N}$, stride $S = k_{maxpool}$ set in the thesis, Padding is 0. The MaxPool operation is applied to the spatial dimensions (height and width) for each channel. In the forward pass, the layer slides a fixed-size window over the input and selects the maximum value within each window. This reduces the spatial resolution of the feature maps while retaining the strongest activations. The MaxPool forward can be described as:

$$y(c, i, j) = \max_{p_1, p_2} \{x(c, i \cdot S + p_1, j \cdot S + p_2)\} \quad (2.10)$$

where p_1 and p_2 are the dimensions of the pooling window.

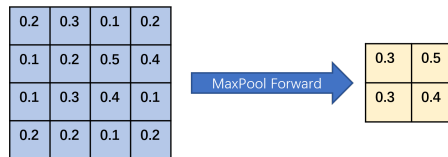


Figure 2.5.3: MaxPool Forward, Stride=2, Padding=0, $k_{maxpool} = 2$

Backward Equation In the backward pass, the gradient is propagated only through the locations in the input matrix x where the maximum values were selected during the forward pass. The MaxPool layer itself does not have any learnable parameters, but the backward operation ensures that only the relevant positions (those

with the maximum values in the forward pass) receive the gradient. The backward of MaxPool can be described as:

$$\frac{\partial y(c, i, j)}{\partial x(c, i', j')} = \begin{cases} 1, & \text{if } x(c, i', j') \text{ was the maximum in its pooling window} \\ 0, & \text{otherwise} \end{cases} \quad (2.11)$$

- $\frac{\partial y(c, i, j)}{\partial x(c, i', j')}$ is the gradient of the MaxPool output y with respect to the input x .
- i', j' represent the spatial location in the input matrix.
- The gradient is non-zero only if $x(c, i', j')$ corresponds to the maximum value in the pooling window during the forward pass.

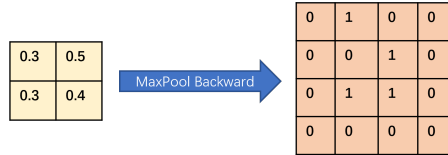


Figure 2.5.4: MaxPool Backward

MaxPool is a typical non-linear operation that is expensive in MPC, unlike average pooling. This thesis applies a "MaxPool-ReLU" block throughout the network to demonstrate efficiency improvements more clearly. Comparison operations in MPC require two rounds of communication, making MaxPool and ReLU quite costly[102].

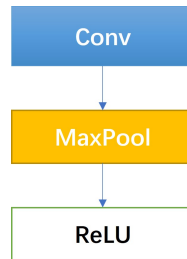


Figure 2.5.5: "MaxPool-ReLU" Block

2.6 Intel SGX

Intel SGX is a set of security-related instruction codes built into modern Intel processors. Intel proposed SGX and published at the HASP conference in 2013[74], introduced a physical CPU(Skylake) supporting SGX at 2016[115]. SGX provides an encrypted execution environment which is a physical area called enclave on CPU, that

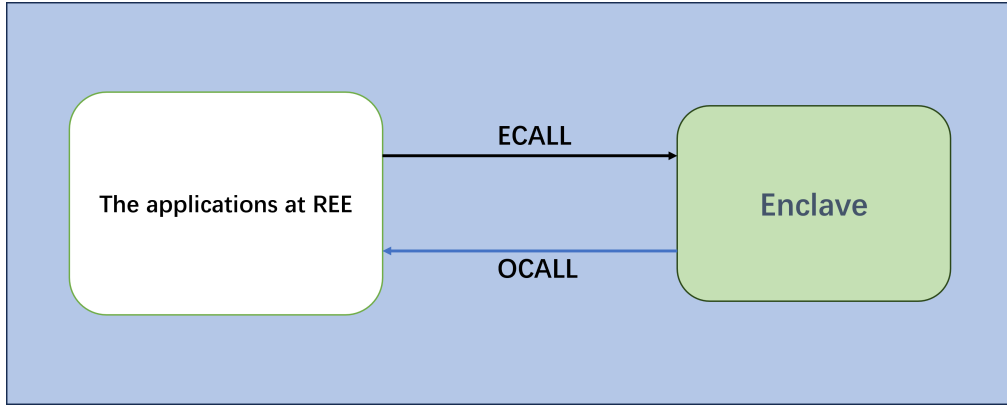


Figure 2.6.1: Intel SGX Interface Calls

protects and prevents the code and data from the outside environment. These enclaves are designed to protect sensitive data and computations from unauthorized access, even if the operating system or other system software is compromised.

- **ECall:** ECALLs are interface functions running within the enclave, defined in .edl files, and called by untrusted applications from the Rich Execution Environment (REE), which is untrusted. Data transfer is implemented by passing pointers and reading bytes; the enclave creates a copy of the data inside. When the execution of an ECALL is finished, the code continues to run the next REE ECALL functions, but the enclave remains unchanged until the next ECALL or the enclave is closed.
- **OCall:** OCALLs are untrusted interfaces used to call applications running in the REE from within the enclave. The usage of OCALLs requires interactions between SGX and the untrusted environment, exposing the system to potential side-channel attacks by observing the interactions. It is crucial to minimize the usage of OCALLs as much as possible to enhance security.

In this thesis, we utilize Intel SGX hardware security to execute plain-text computations for DL instead of cryptographic or shared computations, reducing communication and computation overhead. A practical and important detail of Intel SGX libraries is that they forbid most C++ functions and some C libraries, including the timing library, for security protection. Consequently, all functions executed are basically in pure C style. Implementing the desired functionality from scratch using C standard libraries is crucial for using Intel SGX.

The DL models in this thesis are implemented based on C standard libraries. This

requires knowledge of linear algebra, mathematical analysis, differential matrices, extreme points, derivative chain induction, and optimization theory, as well as the entire computational process of CNNs. These are complex topics for engineering programmers who want to use Intel SGX directly. The infrastructure provided in this thesis implements CNN models that programmers can use for secure training with Intel SGX without the requirement to understand the intricate details of CNN implementation, which involves sophisticated mathematical induction and implementation.

2.7 Hybrid AES-RSA Secure Channel

Asymmetric encryption algorithms, such as RSA, are inefficient for encrypting large data due to their low performance. Conversely, symmetric encryption algorithms like Advanced Encryption Standard (AES), while efficient, present significant key management challenges because the key must be securely shared between parties [39]. Research on hybrid AES-RSA algorithms validates their practicality and effectiveness in resolving these issues [47, 57, 67]. This thesis employs the AES-RSA hybrid encryption scheme to establish a secure communication channel. Future work can explore newer and more efficient methods to further enhance security of secure channels.

Advanced Encryption Standard (AES) AES is a symmetric encryption algorithm established by NIST in 2001, widely used for securing sensitive data due to its efficiency and robustness.

Rivest–Shamir–Adleman (RSA) RSA, developed by Rivest, Shamir, and Adleman in 1977, is an asymmetric encryption technique. While RSA is computationally intensive and impractical for large data encryption, it is ideal for securely transmitting symmetric keys.

The AES-RSA hybrid channel schedule is as follows:

1. Generate a random AES key.
2. Use AES key to encrypt data.
3. Sender encrypts the AES key using RSA public key

4. Receiver decrypts the AES key using RSA private key.
5. Decrypt the data using the AES key.

In this hybrid encryption scheme, AES encrypts the data, and RSA encrypts the AES symmetric key. This combination ensures efficient data encryption and secure key exchange, providing a robust solution for secure communication channels.

Chapter 3

SGXDL

3.1 Motivation for SGXDL

The core idea of SGXDL is to accommodate the entire CNN training process within the SGX enclave, ensuring that training is completed inside the enclave without gradients being transferred outside. Consequently, CNN models remain protected and inaccessible to external entities. Initially, SGX’s enclave capacity was limited to 128 MB, insufficient for the entire CNN model and training process. Today, the enclave capacity has increased to 512 GB, sufficient for even LLMs. Inspired by this progress, we propose executing all CNNs computations in the SGX, presenting a potential path for leveraging hardware security and GPUs-TEEs to address privacy concerns.

While DL computations have seen significant efficiency improvements with GPUs, plain-text calculations within the enclave on the CPU offer three key advantages:

- Compared to other secure and cryptographic techniques, the SGX enclave’s plain-text DL calculation efficiency outperforms GPUs-assisted encrypted calculations, as examined with models like LeNet, AlexNet, ResNet18, and VGG16. The efficiency gap increases with larger model sizes. In this thesis, we benchmark the computing performance together with the known GPUs-assisted MPC platforms Piranha (efficiency winner among SecureML [78], Falcon [100], and FantasticFour [19]).
- Wu et al. [108] proposed GPUs-TEEs, introducing GPUs acceleration for Intel SGX. Our methodology embeds the entire CNN training within the SGX

enclave, leveraging hardware security for privacy computations. SGX plain-text computation already outperforms other secure techniques, and its efficiency can further improve with GPUs-TEEs, making plain-text DL execution within the enclave practical. SGXDL explores running all deep computations within SGX and prepares for the potential future use of GPUs-enabled SGX.

- State-of-the-art MPC protocols, typically involving 2 to 4 parties, suffer from increased communication overhead, computation workload, and memory requirements as the number of parties increases, making them impractical on current servers. For instance, Piranha’s 3-party memory requirement for ResNet18 exceeds the capacity of 24GB GPUs. SGXDL handles the complete process of training, with communication only at the beginning of each training batch when all parties simultaneously send shares securely to the SGX, then the enclave integrates these shares into plain-text within the enclave. Consequently, execution time and performance are minimally influenced by the number of participating parties, unlike state-of-the-art MPC protocols.

3.2 The structure of SGXDL

3.2.1 Data shares distribution

The data are stored in n parties distributedly in a secret-sharing scheme as follows(Figure 3.2.2). Assume the original data sample x . Generate the random numbers as the data shares $x_0, x_1, x_2, \dots, x_{n-2}$ (Section 2.3), the share x_{n-1} satisfies $x_{n-1} = x - \sum_{k=0}^{n-2} x_k$. The dataset and labels used for training would be sent in this way to the SGX enclave. The CNNs model structure and parameters w are generated and stored securely in the Intel SGX enclave.

3.2.2 Secure Channel and Data Transfer initialization

For the server hosting the SGX enclave, although the method of shares generation is unknown, it receives all the shares from the participating parties. This poses a risk because the server could potentially reconstruct the original data sample from these shares. To mitigate this risk, additional protection of the shares is necessary. A secure transmission channel is established to prevent this as described in Section 2.7.

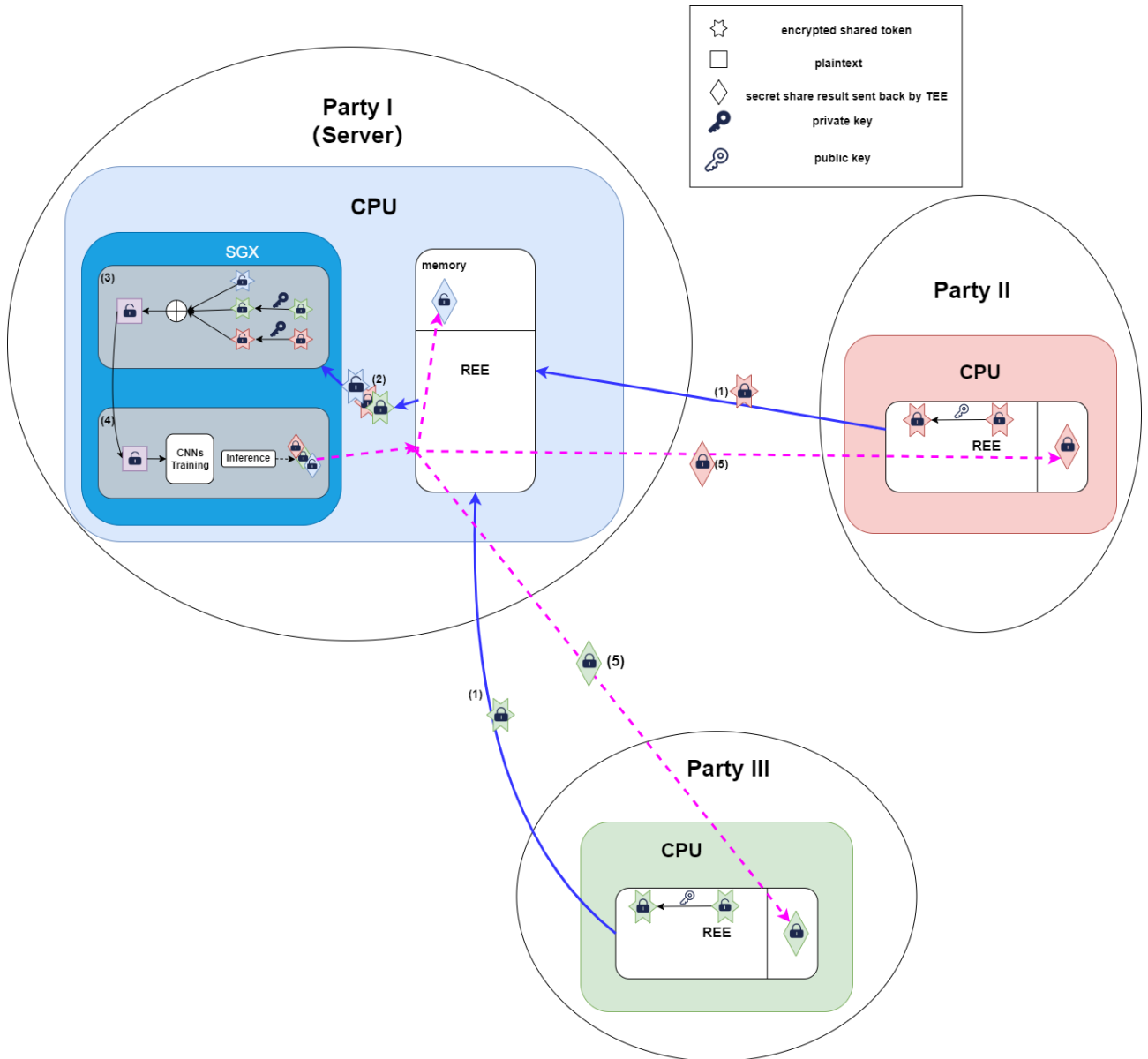


Figure 3.2.1: SGXDL secure computing structure. The blue connection indicates the shares transmission process from the parties to SGX for completing the training, and the hexagram symbol denotes the data shares transmitted from the parties to the SGX. The pink connection represents the computing results in a secret-sharing format back to the parties. The shares would be retrieved to the plain-text format in the SGX enclave.

Locally, shares are encrypted using the AES key and this AES key is then encrypted using the RSA public key. Once the shares are transmitted to the enclave, the AES key is decrypted using the RSA private key which is stored in the enclave to recover the share values. This method ensures that the shares are securely integrated into plaintext values and remain inaccessible to the server. The demonstrations in the following modules apply a three-party scenario.

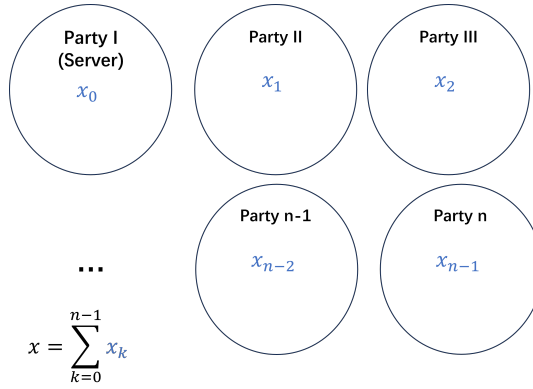


Figure 3.2.2: Data shares distribution(SGXDL)

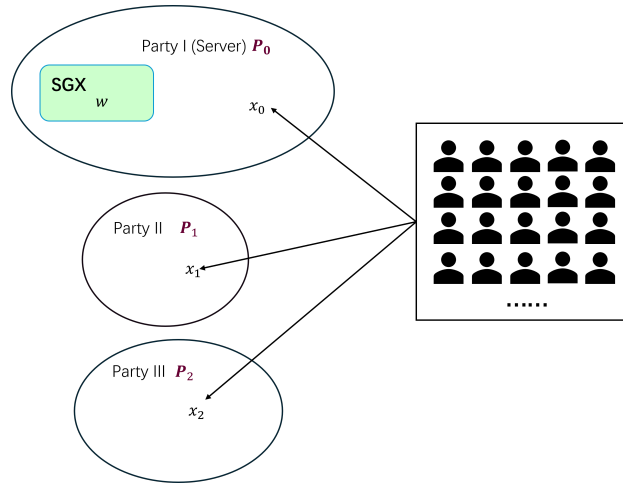


Figure 3.2.3: Data shares and model weight distribution(SGXDL)

Public Key and Private Key distribution

At initialization, the enclave generates the RSA key pairs for the secure transmission channel. Since the server holds and knows the value of shares distributed to it, the enclave doesn't need to encrypt the share that is held by the server. The enclave sends the public key to all the client parties and keeps the private key securely within the enclave.

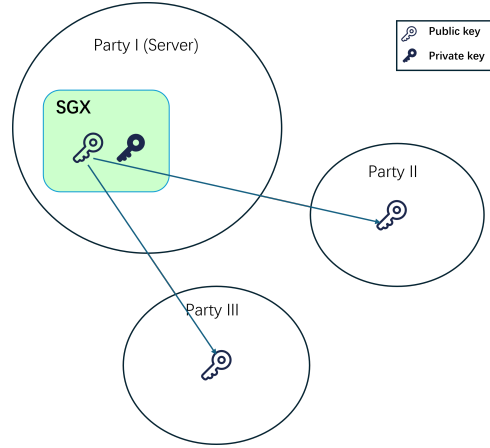


Figure 3.2.4: RSA Public Key Exchange at Initialization(SGXDL, 3 party)

Secure Data Transmit and Integration

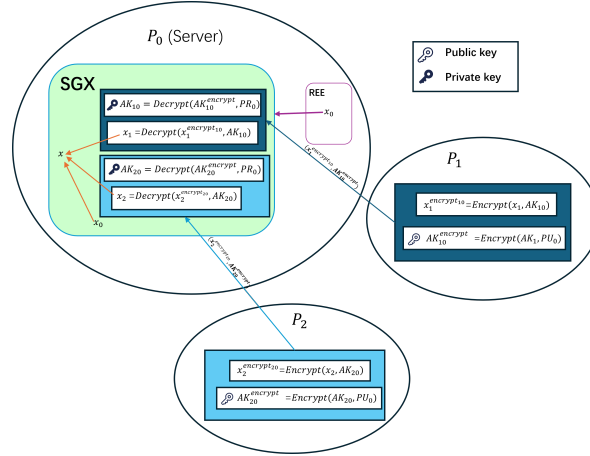


Figure 3.2.5: SGXDL secure data transmission channel

Assume the RSA public key PU , and the private key PR . $Encrypt(C, K)$ represents using the key K to encrypt the context C , and the $Decrypt(C, K)$ represents using the key K to decrypt the context C . Assume the share x_k of data sample held by the party P_i , the random AES keys $AK_{ij} \in \mathbf{R}$ are generated by party i and encrypt data sent to party j . Define the RSA key pairs PU_i and PR_i are generated by party P_i .

In the local party i

$$x_k^{encrypt_{ij}} = Encrypt(x_k, AK_{ij})$$

where $x_k^{encrypt_{ij}}$ is the encryption value for k -th share of x with AES key AK_{ij} , and will be sent from party P_i to P_j .

$$AK_{ij}^{encrypt} = Encrypt(AK_{ij}, PU_j)$$

where $AK_{ij}^{encrypt}$ is the encryption of symmetric key AK_{ij} by public key P_i . In the enclave, the symmetric key and the share would be decrypted that:

$$AK_{ij} = Decrypt(AK_{ij}^{encrypt}, PR_j)$$

$$x_k = Decrypt(x_k^{encrypt_{ij}}, AK_{ij})$$

The party we send the $(x_k^{encrypt_{ij}}, AK_{ij}^{encrypt})$ to the SGX, and in the sgx enclave, the encrypted symmetric key $AK_{ij}^{encrypt}$ is decrypted to AK_{ij} using the private key P_j . And the encrypted shares would be decrypted to the original shares using the AK_{ij} .

$$x_k = Decrypt(x_k^{encrypt_{ij}}, Decrypt(AK_{ij}^{encrypt}, PR_j))$$

The data used for training in the enclave can be integrated into the enclave:

$$x = \sum_{k=0}^{n-1} x_k$$

Then the plain-text training can be executed on the plain-text dataset $\{x\}$, with the CNNs model parameters w stored in the secure enclave.

3.3 Task order and Synchronization

Figure 3.3.1 illustrates the SGXDL workflow throughout the entire training process. Since the whole training process occurs within the enclave, the training can begin as soon as the SGX receives the shares of a batch of samples. The SGX has the flexibility to start training either after receiving a batch of data sample shares or after receiving all data shares.

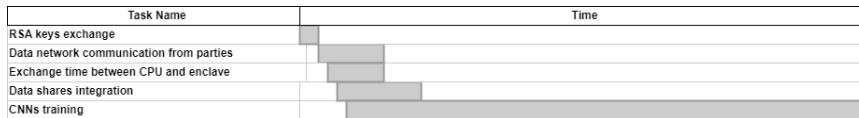


Figure 3.3.1: SGXDL task order and synchronization. The diagram describing SGXDL synchronization illustrates the workflow of SGXDL. Section 6.3.1 shows that during training, the exchange time between the CPU and the enclave, as well as the data sharing integration overhead, is almost zero. The rectangular task time blocks only represent a schedule of tasks' orders and do not reflect accurate time.

3.4 Summary

- The core idea of SGXDL is to perform plain-text CNN training entirely within the enclave, using a secret-sharing scheme to ensure that the attacker can not extract useful information without compromising all parties.
- The model and gradients remain within the enclave throughout the whole training process, protecting them from common FL attacks such as membership inference and gradient attacks.
- During training, SGX communicates with the outside environment only at the beginning of training for dataset shares transmission, minimizing the risk of side-channel attacks by limiting observable communication.
- Secure training in the enclave relies on CPU capability; hardware-secured plain-text computation has been shown to outperform other cryptographic techniques in efficiency, as demonstrated in Chapter 6. In the future, GPUs-TEEs research could further enhance the efficiency of training complete plain-text models within the enclave.

Chapter 4

HybridSGXDL

To address privacy and efficiency challenges and leverage GPUs advantages for secure DL in real-world scenarios, HybridSGXDL is proposed as a secure computing architecture. The operations would be separated into arithmetic and non-linear operations with the assistance of Intel SGX. This architecture uses GPUs to accelerate the local arithmetic computation in secret-sharing format. Non-linear operations are executed in the enclave in plain-text. The communication between parties and the server would happen at the end of each arithmetic and non-linear computation, to share the results for the next operations and complete the training together.

Unlike SGXDL, which keeps all computations within the enclave, HybridSGXDL expects devices to handle arithmetic distributed computation with GPUs, secret-sharing the model weights and data among parties. Training of CNNs begins with randomly initialized weights. In this process, the initial model weights are generated collectively by all parties in such a way that no single party has knowledge of the actual initial weight values. The server can only receive the shares of intermediate computing results during the training.

CNNs modules typically consist of arithmetic operations followed by non-linear operations. Arithmetic operations, such as addition and multiplication, are performed locally with GPUs acceleration, while non-linear operations, such as ReLU and MaxPool, are executed in plain-text within the Intel SGX enclave. The enclave acts as a trusted third party, integrating intermediate results from the arithmetic operations. After completing the non-linear computations, the server returns the results in data shares format to the parties for the next arithmetic convolution computation.

The HybridSGXDL implementation in this thesis is under a 3-party scenario.

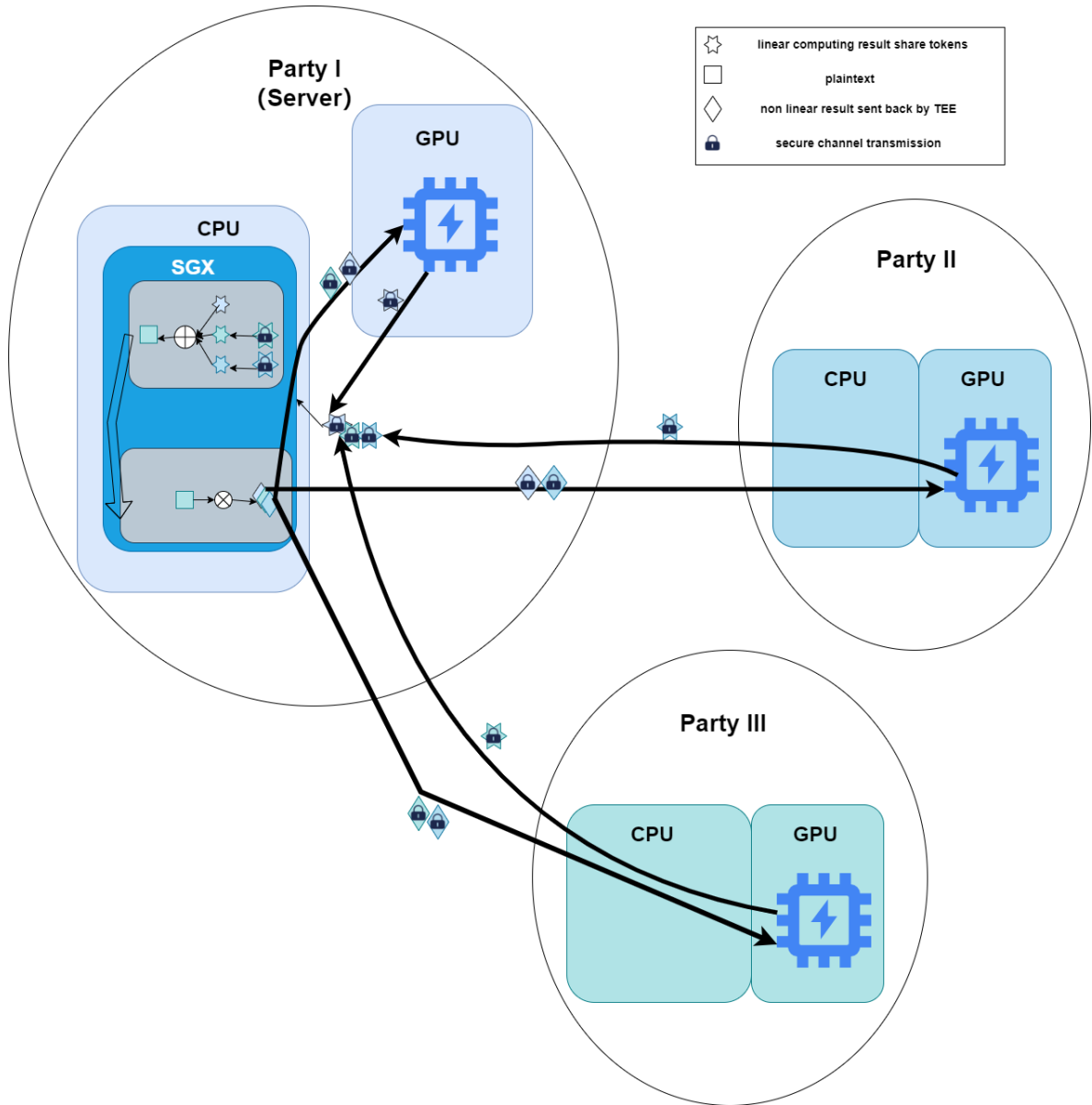


Figure 4.0.1: HybridSGXDL safe computing structure

4.1 Model Weight Share Distribution

In HybridSGXDL, the model weights and dataset are distributed in a secret-sharing format. The weight updates are sent to the parties by the enclave, meaning the server has the potential to generate all the secret shares. The initialized weights are unknown to the parties, including the server, so even if in the worst case an attacker hacks all weight update shares, the value of the weights remains unknown. Additionally, adversaries must attack multiple participating parties to retrieve any

potentially useful information, distributing the risks and significantly increasing the difficulty for attackers.

Since the server has the potential to generate all the secret shares, we apply the secure channel scheme described in Section 3.2.2 to protect these shares from the server. In the thesis we only implement the HybridSGXDL under a 3 party setup:

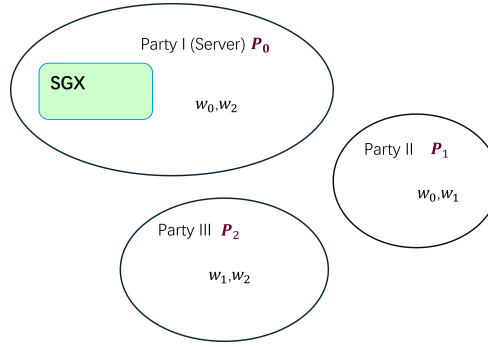


Figure 4.1.1: Weight share distribution for HybridSGXDL(3 Party)

The 3 party CNNs model weight initialization in HybridSGXDL To randomly generate the model weight shares together by all the parties, and no party has useful information of the initialization weight real values. The 3-party condition would follow the steps:

- P_0 generates the random values w_0, w_2 , P_1 generates the random value w_1 at the same time.
- P_0 sends w_0 secret share to P_1 , and w_2 to P_2 . P_1 sends w_1 to P_2 at the same time.

Then the real value of model initialization weights is: $w = w_0 + w_1 + w_2$, but no party knows this value, and in the following computation the model's weight would consent to be unknown for all the parties including the SGX enclave.

4.1.1 Keys Exchange of secure channel for model weight initialization

There are requirements of establishing secure channels for the shares resharing in model weight initialization. Since the model weight initialization and transmission only require the server P_0 sending shares to other parties P_1, P_2 , and $P_1 \rightarrow P_2$. Only P_1 and P_2 need create the RSA key pairs, the process is:

- P_1 and P_2 create the RSA public and private key pairs locally.

- P_1 and P_2 send their public keys to the server P_0 , and P_2 sends out its public key to P_1 at the same time.

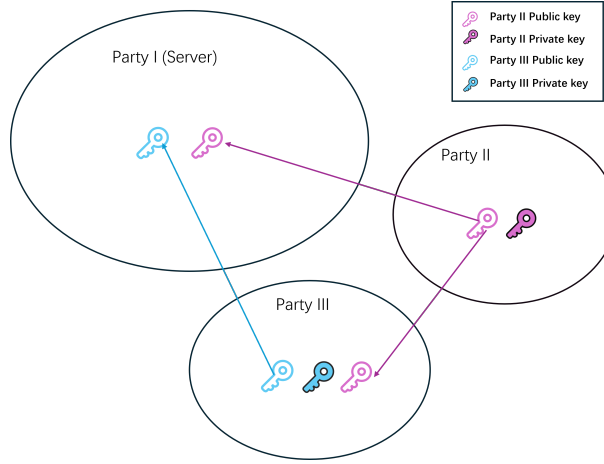


Figure 4.1.2: Public Key Exchange for model weight shares initialization(HybridSGXDL, 3 party)

4.1.2 Secure Data Transmission and Integration

After defining the transmission flow in HybridSGXDL, the secure channel to prevent the server from potentially receiving all shares of intermediate products directly is the same as with SGXDL (Section 3.2.2, Figure 3.2.4). The secure transmission workflows for HybridSGXDL weights initialization, and the shares of non-linear operations are illustrated in Figure 4.1.3 and Figure 4.3.4.

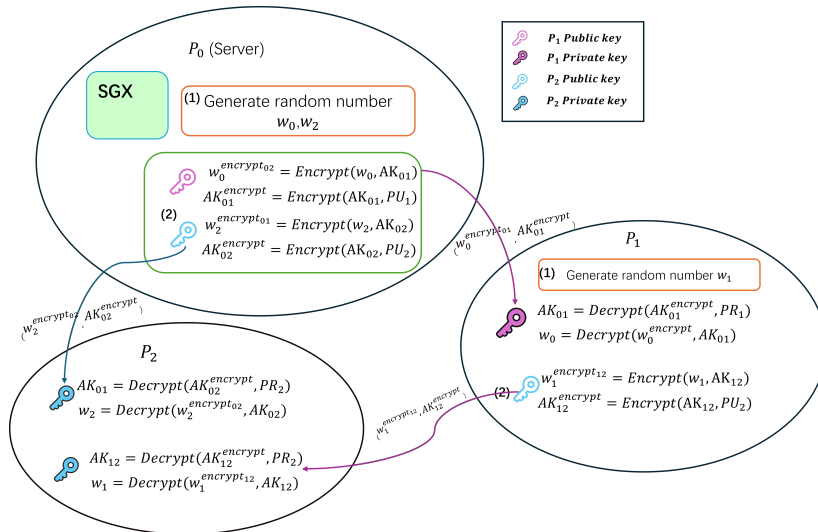


Figure 4.1.3: Model weight random initialization(HybridSGXDL, 3 party)

4.2 Training Dataset Share Distribution

The training dataset X is distributed to the parties in the same secret-sharing scheme as the model weight w . Assume the dataset shares $X = \{x_0, x_1, \dots, x_{n-1}\} (x = \sum_{k=0}^{n-1} x_k)$.

For this thesis, the dataset shares are initially distributed to the parties by the individual data owners(Figure 4.2.1).

The 3 party dataset share distribution in HybridSGXDL as described in Section 2.3.3. x is the data sample.

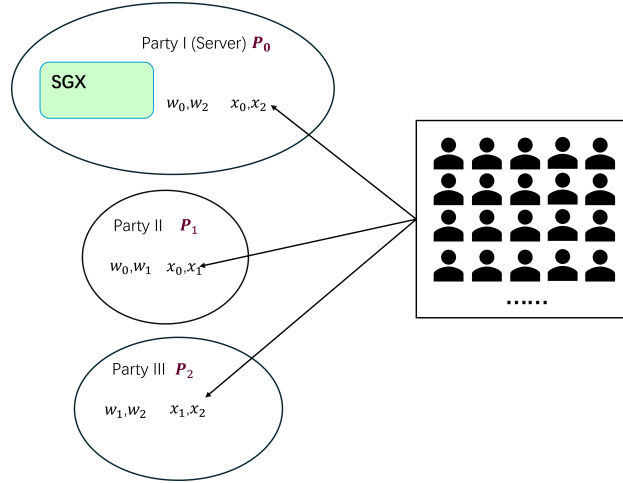


Figure 4.2.1: Dataset and model weight shares distribution(HybridSGXDL, 3 party)

4.3 Arithmetic and Non-Linear Computation for CNNs

CNNs typically consist of repeating modules, such as convolution followed by ReLU, and convolution followed by MaxPool. ReLU and MaxPool are non-linear operations in MPC protocols that involve significant communication overhead, and common methods using polynomials to approximate these non-linear operations are also computationally intensive. HybridSGXDL separates the computation of CNNs into arithmetic and non-linear operations.

Arithmetic operations, such as addition and multiplication, are executed on local parties' GPUs, leveraging GPUs acceleration. These operations follow the same flow

as plain-text arithmetic computations and the same structure as arithmetic operations. Non-linear operations, such as ReLU and MaxPool, are executed in the SGX enclave, where plain-text computation is protected by SGX's hardware security.

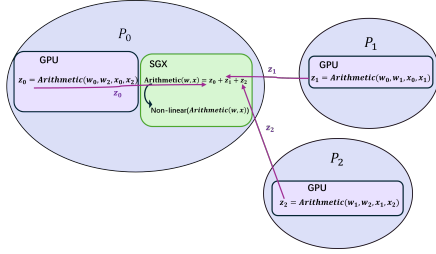


Figure 4.3.1: Non-Linear and Arithmetic operations Separation Computation(HybridSGXDL, 3 party)

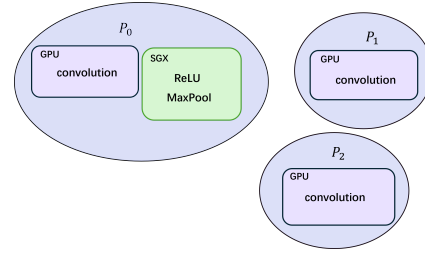


Figure 4.3.2: Non-Linear and Arithmetic Operations Distribution(HybridSGXDL, 3 party)

In the thesis, the repeating modules consist of convolution followed by MaxPool and ReLU . In these modules, the arithmetic operations are convolutions, and the non-linear operations are MaxPool and ReLU .

4.3.1 Arithmetic Operations on local party's GPUs

The arithmetic computation method in HybridSGXDL would not ruin the results of arithmetic operations(which only consist of multiplication and addition), in the following context we demonstrate the correctness of convolution forward result as an example. In the thesis we implemented and experimented the 3-party scenarios. All the following proofs are under the 3-party setup. Assume the input image 3-dimensional matrix $x^{C \times I \times I}$, the convolution 3-dimensional output $y^{O \times N \times N}$ (padding=0, stride=1), and the 4-dimensional convolution kernel $w^{C \times O \times M \times M}$. C is the channel number of the image, e.g. the number of channels for RGB image is 3. I is the width and height of the image matrix. O is the channels number of the output matrix. The magnitude of the first dimension of w is the same with input image x 's channels C . The convolution is:

$$Conv(w, x) : y(o, i, j) = \sum_{c=0}^{C-1} \sum_{p_1, p_2=0}^{M-1} w(c, o, p_1, p_2) * x(c, i + p_1, j + p_2) \quad (4.1)$$

Assume $w(c, o, p_1, p_2)_{u_0}$ is the weight share of party P_{u_0} for the matrix element $w(c, o, p_1, p_2)$. And the model weight shares $w = w_0 + w_1 + w_2$ that

$$w(c, o, p_1, p_2) = w(c, o, p_1, p_2)_0 + w(c, o, p_1, p_2)_1 + w(c, o, p_1, p_2)_2 \quad (4.2)$$

$$x(c, i + p_1, j + p_2) = x(c, o, p_1, p_2)_0 + x(c, i + p_1, j + p_2)_1 + x(c, i + p_1, j + p_2)_2 \quad (4.3)$$

Where $x(c, i + p_1, j + p_2)_{u_0}$ is the weight share of party P_{u_0} for input image's element $x(c, i + p_1, j + p_2)$. Then the convolution would be:

$$y(o, i, j) = \sum_{c=0}^{C-1} \sum_{p_1, p_2=0}^{M-1} (w(c, o, p_1, p_2)_0 + w(c, o, p_1, p_2)_1 + w(c, o, p_1, p_2)_2) * x(c, i + p_1, j + p_2) \quad (4.4)$$

$$\begin{aligned} y(o, i, j) &= \sum_{c=0}^{C-1} \sum_{p_1, p_2=0}^{M-1} w(c, o, p_1, p_2) * x(c, i + p_1, j + p_2) \\ &= \sum_{c=0}^{C-1} \sum_{p_1, p_2=0}^{M-1} \left(\left(\sum_{u_0=0}^2 w(c, o, p_1, p_2)_{u_0} \right) * \left(\sum_{u_1=0}^2 x(c, i + p_1, j + p_2)_{u_1} \right) \right) \\ &= \sum_{c=0}^{C-1} \left(\sum_{\substack{p_1, p_2=0 \\ (u_0, u_1) \in \{(0,0), (0,2), (2,0)\}}}^{M-1} w(c, o, p_1, p_2)_{u_0} x(c, i + p_1, j + p_2)_{u_1} \right. \\ &\quad + \sum_{\substack{p_1, p_2=0 \\ (u_0, u_1) \in \{(1,1), (1,0), (0,1)\}}}^{M-1} w(c, o, p_1, p_2)_{u_0} x(c, i + p_1, j + p_2)_{u_1} \\ &\quad \left. + \sum_{\substack{p_1, p_2=0 \\ (u_0, u_1) \in \{(2,2), (2,1), (1,2)\}}}^{M-1} w(c, o, p_1, p_2)_{u_0} x(c, i + p_1, j + p_2)_{u_1} \right) \\ &= z_0 + z_1 + z_2 \end{aligned} \quad (4.5)$$

such that for $z = y(o, i, j)$

$$\begin{aligned} z_0 &= \sum_{c=0}^{C-1} \sum_{\substack{p_1, p_2=0 \\ (u_0, u_1) \in \{(0,0), (0,2), (2,0)\}}}^{M-1} w(c, o, p_1, p_2)_{u_0} x(c, i + p_1, j + p_2)_{u_1} \\ &= Conv(w_0, x_0) + Conv(w_0, x_2) + Conv(w_2, x_0) \end{aligned} \quad (4.6)$$

$$\begin{aligned}
 z_1 &= \sum_{c=0}^{C-1} \sum_{\substack{p_1, p_2=0 \\ (u_0, u_1) \in \{(1,1), (1,0), (0,1)\}}}^{M-1} w(c, o, p_1, p_2)_{u_0} x(c, i + p_1, j + p_2)_{u_1} \\
 &= Conv(w_1, x_1) + Conv(w_1, x_0) + Conv(w_0, x_1)
 \end{aligned} \tag{4.7}$$

$$\begin{aligned}
 z_2 &= \sum_{c=0}^{C-1} \sum_{\substack{p_1, p_2=0 \\ (u_0, u_1) \in \{(2,2), (2,1), (1,2)\}}}^{M-1} w(c, o, p_1, p_2)_{u_0} x(c, i + p_1, j + p_2)_{u_1} \\
 &= Conv(w_2, x_2) + Conv(w_2, x_1) + Conv(w_1, x_2)
 \end{aligned} \tag{4.8}$$

Parties execute the computation locally basing the holding shares of w, x , and calculate the shares of convolution products $z = y(o, i, j)$ that:

- P_0 computes z_0
- P_1 computes z_1
- P_2 computes z_2

on their local GPUs. In the process, the local parties calculate z_0, z_1, z_2 independently, do not need to communicate with parties.

and send this arithmetic results to the SGX enclave can calculate the output $z = y(o, i, j)$ as shown in Figure 4.3.1. Then the SGX enclave calculates:

$$\begin{aligned}
 z &= z_0 + z_1 + z_2 = y(o, i, j) && \text{retrieve to plain-text convolution product } z \\
 ReLU(z) &&& \text{execute the activation computation (Equation 2.8)}
 \end{aligned}$$

w or x can not be retrieved from these intermediate products, and the w weights and x data do not leave the party during the entire training process. The above proof is an illustration of how an element in a convolution result matrix is calculated in HybridSGXDL, demonstrating that this arithmetic operation ensures the accuracy of the final result in comparison with plain-text computation. In actual situations, the matrix is calculated and transmitted in batches.

We have demonstrated that the correctness of the forward convolution operation remains intact in HybridSGXDL. The backward propagation process adheres to the same separation rules for arithmetic and non-linear operations. Backward's

arithmetic operations are computed locally on clients' GPUs, while backward's non-linear operations are performed within the Intel SGX enclave. Because the backward computation of arithmetic operations is also arithmetic. The backward propagation of non-linear operations remains non-linear and is executed secretly in plain-text within the enclave. The backward propagation of convolution is essentially a forward convolution, except that it operates different operands in comparison with convolution forward. For example, the backward convolution computes the gradients for updating the weights, its operands are the input matrix of forward computation and the gradient of output matrix versus loss function(as shown in Equation 2.6). Therefore, the correctness of the convolution's backward pass also has been mathematically validated through the forward as shown in Equation 4.5, 4.6, 4.7, 4.8.

Secure Transmission for Arithmetic Results Since the RSA keys from clients to the server have been distributed at the initialization as shown in Figure 4.1.2, we want to send the arithmetic results securely to the enclave using this secure channel to prevent the server from receiving all the shares for the arithmetic intermediate computation results. We always use secure channels for communications.

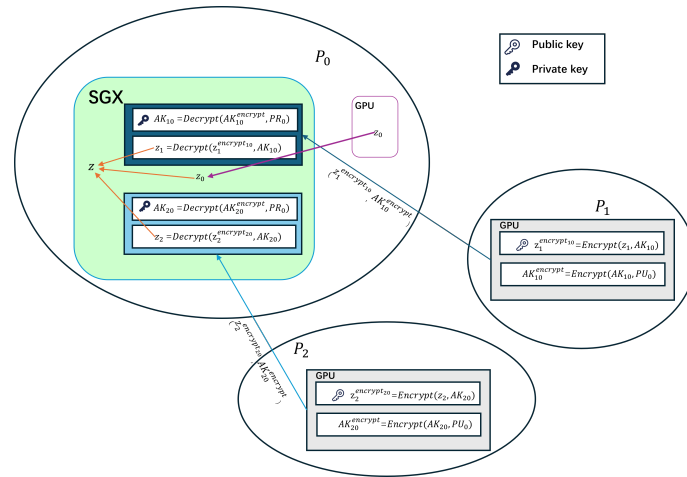


Figure 4.3.3: HybridSGXDL Secure Arithmetic Results sent to SGX for next non-linear operations. z is the convolution result retrieved by z_0, z_1, z_2 , where z_0, z_1, z_2 are the shares of arithmetic result computed by local parties on GPUs.

4.3.2 Non-Linear Operations in enclave

In the enclave, the arithmetic result shares are integrated into plain-text values. SGX hardware security prevents access from the outside environment. This keeps the computation process protected and secret within the SGX enclave. After completing

the non-linear computation, the results of non-linear operations would be separated into secret shares $\{r_0, r_1, r_2\}$ ($ReLU(z) = r_0 + r_1 + r_2$), z is the integrated input which is also the result of convolution in CNNs, and sent back to parties' GPUs via the secure channel. These shares participate in the next convolution computation operations, continuing until the forward and backward computations of the CNNs are completed.

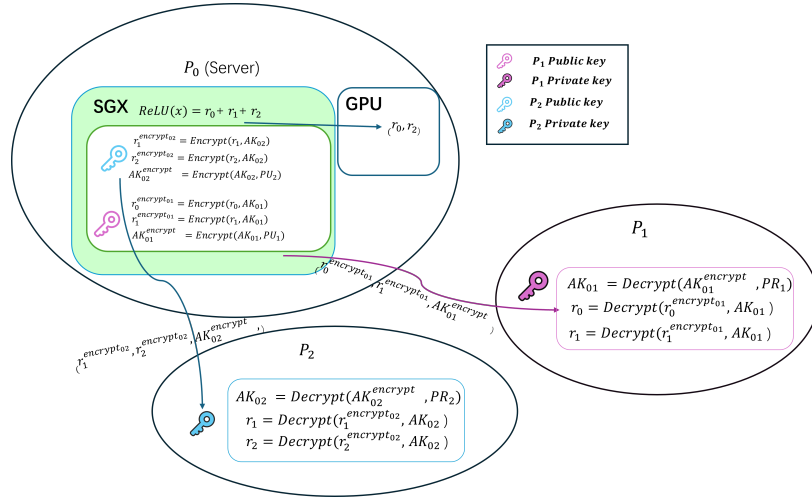


Figure 4.3.4: HybridSGXDL Secure Non-Linear Results Transmission from Enclave. r here denotes the result of non-linear operation in the enclave, and r_0, r_1, r_2 are the shares of r .

Though the server has known the real values of holding shares r_0, r_2 , it is still necessary using secure channels for all communications to prevent external adversaries.

4.4 Gradient Update

The model's weights w are updated at the end of each training. Assume the update is ∂w , separated into shares $\partial w = \partial w_0 + \partial w_1 + \partial w_2$ for updating the w_0, w_1, w_2 , the secure transmission for the weight update workflow is the same with non-linear results transmission as shown in Figure 4.4.1.

The new model weight w' and the updated shares w'_0, w'_1, w'_2 would be

$$w' = w + \partial w \quad (4.9)$$

$$w'_0 = w_0 + \partial w_0, w'_1 = w_1 + \partial w_1, w'_2 = w_2 + \partial w_2 \quad (4.10)$$

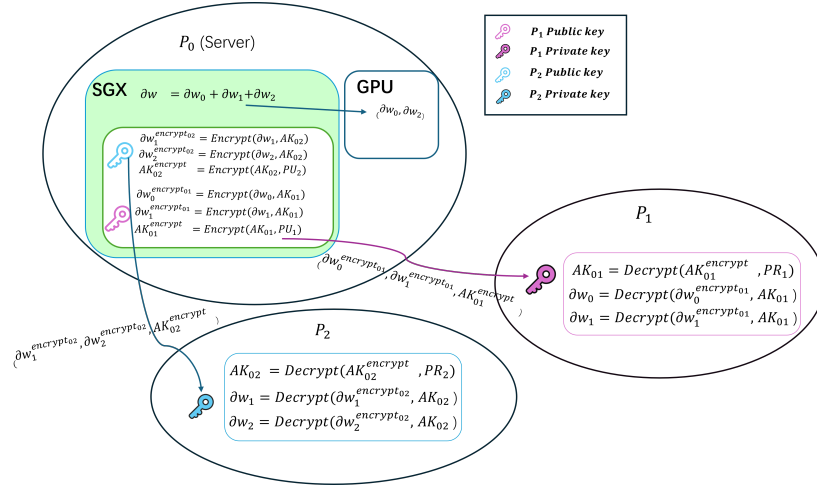


Figure 4.4.1: HybridSGXDL Gradient Shares Safe Transmission, AK_{ij} is the symmetric key used for encrypting the transmitted shares from party P_i to P_j .

4.5 Task order and Synchronization

Figure 4.5.1 depicts the task order of a training batch for CNNs, we can notice that each computation layer in the CNNs waiting for the results from the last layer computation, and that there is almost no synchronization for the further time optimization.

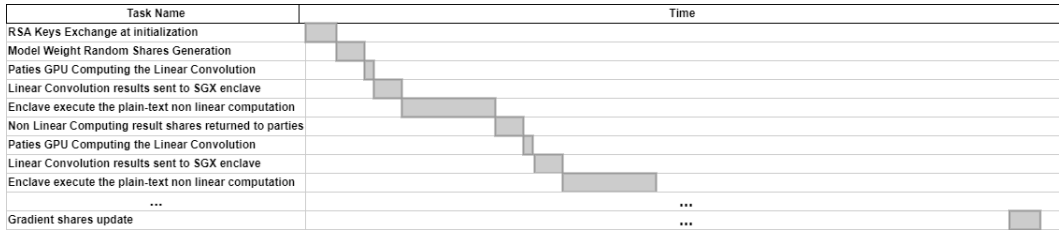


Figure 4.5.1: Task order and synchronization, the image depicts the task orders in a training batch and the synchronization condition. The rectangular task time blocks only represent a rough relative description of the task, and do not reflect the accurate condition, in some cases, the plain-text non-linear computation time can be very short and less than GPUs arithmetic computation time. Usually, the network communication time is constant.

4.6 Summary

- HybridSGXDL separates CNNs computations into arithmetic and non-linear operations. Arithmetic operations are computed distributedly in secret shares using the same logic as plain-text arithmetic computations on local parties, enabling the utilization of GPUs. Non-linear operations, which are computationally intensive in MPC cryptography protocols, are executed and

protected in plain-text within the SGX enclave, ensuring secure and efficient computation.

- Unlike state-of-the-art FL HybridSGXDL does not require sharing model weights. Initialized weight shares are generated randomly by each party locally, ensuring that the integrated model weights are unknown to any party. Data shares and weights remain local during training convolution, and only the random multiplication intermediate product shares are sent out, protecting the data and weights from being revealed. Without access to model weights, common gradient attacks in FL, such as membership inference, are not applicable.
- The share exchanges occur through a secure transmission channel, using a hybrid AES-RSA secure channel for fast encryption and decryption.
- Data sample shares are sent directly by individuals to the computing parties, ensuring that no party knows the data sample values. This increases security for individuals and enables training on private datasets that they are unwilling to share with institutions. Additionally, the discrete and unpredictable communication from individuals makes the condition more complex for attackers and reduces the risk of data leakage, as discrete intercepted message contains minimal information.

Chapter 5

The Deep Learning Models in the Implementation

This section provides details of the DL models enabled by the secure computing infrastructures, which are essential for understanding the execution efficiency of these models. The arithmetic operations in the thesis are convolutions and fully connected neural layers, while non-linear operations are MaxPool and ReLU (Eq 2.10, 2.8). The data type used in the implementation is double (8 bytes).

Though other practical CNN models typically use the float data type (4 bytes), the double data type VGG16 model in this thesis, with tens of millions of parameters and a size of 297 MB, offers a practical reference for the secure computation of LLMs, which are often hundreds of MBs in size (e.g., BERT is 110 MB [61]).

5.1 The input and output of CNNs in SGXDL and HybridSGXDL

The section describes the input and output details of CNNs in SGXDL and HybridSGXDL.

Network	Dataset	Model parameter number(double)	Model memory size(MB)
LeNet	MNIST	51,902	0.396
	Cifar10	52,202	0.398
AlexNet	MNIST	3,868,170	29.51
	Cifar10	3,868,746	29.51
ResNet18	MNIST	11,167,114	85.2
	Cifar10	11,168,266	85.21
VGG16	MNIST	38,946,762	297.14
	Cifar10	38,947,914	297.15

Table 5.0.1: The number and size of model parameters. Note that this reflects the parameters of plain-text CNN models; the parameter count may be much higher in cryptographic computations due to the specific protocol’s cryptographic format. In SGXDL, the enclave stores the plain-text CNN model and performs plain-text computations. This table demonstrates the computation workload and CNN model size in SGXDL. Since the CNN models share the same structure, the computation workload for MNIST and CIFAR-10 for the same number of batches is almost identical, so the time required to execute a batch for MNIST and CIFAR-10 should be similar.

5.1.1 MNIST

Input:

The MNIST dataset consists of grayscale images of handwritten digits, each with a size of 28×28 pixels. For neural network input, these images are typically represented as a three-dimensional tensor of shape $1 \times 28 \times 28$ (where 1 represents the single grayscale channel). The specific representation depends on the network architecture. For CNNs, the $1 \times 28 \times 28$ format is usually preserved to maintain spatial structure.



Figure 5.1.1: MNIST Data Input of "3" [60]



Figure 5.1.2: MNIST Dataset [60]

Output:

The output for MNIST is a probability distribution over 10 classes, corresponding to the digits 0 to 9. The neural network generates a 1×10 vector using a softmax activation function, where each element represents the probability of the input image being classified as a particular digit (0 to 9). The final prediction of the network is the class with the highest probability.

5.1.2 Cifar-10**Input:**

The CIFAR-10 dataset contains color images, each with a size of 32×32 pixels and 3 color channels (RGB). Thus, the input to the neural network is typically represented as a three-dimensional tensor of shape $3 \times 32 \times 32$ (where 3 indicates the color channels). This representation retains the spatial and color information of the image, which is crucial for CNNs.

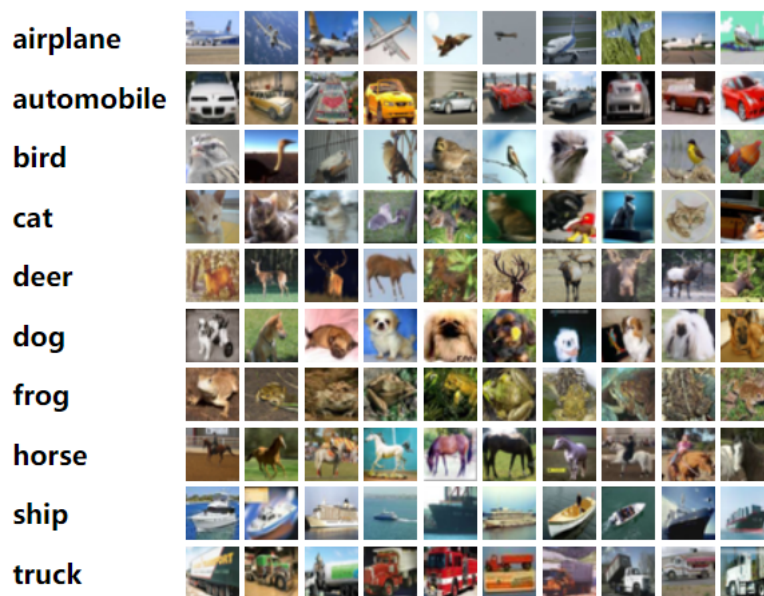


Figure 5.1.3: Cifar10 Dataset [54]

Output:

The output for CIFAR-10 is also a probability distribution over 10 classes, corresponding to 10 different object categories (such as airplanes, automobiles, birds, cats, etc.). The neural network generates a 1×10 vector using a softmax activation

function, where each element represents the probability of the input image being classified into a specific category. Similarly, the final prediction of the network is the class with the highest probability.

5.2 CNNs model structure details

The following tables demonstrate the models' details for the MNIST dataset implemented in the thesis. The structures for cifar10 are the same with MNIST except the input layers at the beginning to make them applicable for cifar10 images' sizes.

layer name	kernel size	output size
conv1	$1 \times 6 \times 5 \times 5$, stride=1, padding=2	$6 \times 28 \times 28$
relu	-	$6 \times 28 \times 28$
maxpool	2×2 , stride=2	$6 \times 14 \times 14$
conv2	$6 \times 16 \times 5 \times 5$, stride=1	$16 \times 10 \times 10$
relu	-	$16 \times 10 \times 10$
maxpool	2×2 , stride=2	$16 \times 5 \times 5$
conv3	$16 \times 120 \times 5 \times 5$, stride=1	$120 \times 1 \times 1$
relu	-	$120 \times 1 \times 1$
fc	120×10	10
relu	-	10

Table 5.2.1: LeNet Structure(MNIST) [59], fc is the fully-connected layer, conv is the convolution layer. conv1 is the first convolution layer. The C code defined the structure in implementation see Appendix A.2.1

layer name	kernel size	output size
conv1	$1 \times 32 \times 3 \times 3$, stride=1, padding=1	$32 \times 28 \times 28$
relu	-	$32 \times 28 \times 28$
maxpool	2×2 , stride=2	$32 \times 14 \times 14$
conv2	$32 \times 64 \times 3 \times 3$, stride=1, padding=1	$64 \times 14 \times 14$
relu	-	$64 \times 14 \times 14$
maxpool	2×2 , stride=2	$64 \times 7 \times 7$
conv3	$64 \times 128 \times 3 \times 3$, stride=1, padding=1	$128 \times 7 \times 7$
relu	-	$128 \times 7 \times 7$
conv4	$128 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 7 \times 7$
relu	-	$256 \times 7 \times 7$
conv5	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 7 \times 7$
relu	-	$256 \times 7 \times 7$
maxpool	3×3 , stride=2	$256 \times 3 \times 3$
fc1	$(256 \times 3 \times 3) \times 1024$	1024
relu	-	1024
fc2	1024×512	512
relu	-	512
fc3	512×10	10
relu	-	10

Table 5.2.2: AlexNet Structure(MNIST)[56]. Appendix A.2.2

layer name		kernel size	output size
conv1		$1 \times 64 \times 3 \times 3$, stride=1, padding=3	$64 \times 32 \times 32$
relu		-	$64 \times 32 \times 32$
maxpool		2×2 , stride=2	$64 \times 16 \times 16$
Res Block1	conv1	$64 \times 64 \times 3 \times 3$, stride=1, padding=1	$64 \times 16 \times 16$
	relu	-	$64 \times 16 \times 16$
	conv2	$64 \times 64 \times 3 \times 3$, stride=1, padding=1	$64 \times 16 \times 16$
	relu	-	$64 \times 16 \times 16$
	conv3	$64 \times 64 \times 3 \times 3$, stride=1, padding=1	$64 \times 16 \times 16$
	relu	-	$64 \times 16 \times 16$
	conv4	$64 \times 64 \times 3 \times 3$, stride=1, padding=1	$64 \times 16 \times 16$
	relu	-	$64 \times 16 \times 16$
Res Block2	conv1	$64 \times 128 \times 3 \times 3$, stride=2, padding=1	$128 \times 8 \times 8$
	relu	-	$128 \times 8 \times 8$
	conv2	$128 \times 128 \times 3 \times 3$, stride=1, padding=1	$128 \times 8 \times 8$
	relu	-	$128 \times 8 \times 8$
	conv3	$128 \times 128 \times 3 \times 3$, stride=1, padding=1	$128 \times 8 \times 8$
	relu	-	$128 \times 8 \times 8$
	conv4	$128 \times 128 \times 3 \times 3$, stride=1, padding=1	$128 \times 8 \times 8$
	relu	-	$128 \times 8 \times 8$
Res Block3	conv1	$128 \times 256 \times 3 \times 3$, stride=2, padding=1	$256 \times 4 \times 4$
	relu	-	$256 \times 4 \times 4$
	conv2	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 4 \times 4$
	relu	-	$256 \times 4 \times 4$
	conv3	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 4 \times 4$
	relu	-	$256 \times 4 \times 4$
	conv4	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 4 \times 4$
	relu	-	$256 \times 4 \times 4$
Res Block4	conv1	$256 \times 512 \times 3 \times 3$, stride=2, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
	conv2	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
	conv3	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
	conv4	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
maxpool		2×2 , stride=2	$256 \times 1 \times 1$
fc		512×10	10
relu		-	10

Table 5.2.3: ResNet18 Structure(MNIST) [41]. ResBlock is the repeating convolution blocks with res connections. Appendix A.2.3

layer name		kernel size	output size
Conv Block1	conv1	$1 \times 64 \times 3 \times 3$, stride=1, padding=3	$64 \times 32 \times 32$
	relu	-	$64 \times 32 \times 32$
	conv2	$64 \times 64 \times 3 \times 3$, stride=1, padding=1	$64 \times 32 \times 32$
	relu	-	$64 \times 32 \times 32$
	maxpool	2×2 , stride=2	$64 \times 16 \times 16$
Conv Block2	conv1	$64 \times 128 \times 3 \times 3$, stride=1, padding=1	$128 \times 16 \times 16$
	relu	-	$128 \times 16 \times 16$
	conv2	$128 \times 128 \times 3 \times 3$, stride=1, padding=1	$128 \times 16 \times 16$
	relu	-	$128 \times 16 \times 16$
	maxpool	2×2 , stride=2	$128 \times 8 \times 8$
Conv Block3	conv1	$128 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 8 \times 8$
	relu	-	$256 \times 8 \times 8$
	conv2	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 8 \times 8$
	relu	-	$256 \times 8 \times 8$
	conv3	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 8 \times 8$
	relu	-	$256 \times 8 \times 8$
	conv4	$256 \times 256 \times 3 \times 3$, stride=1, padding=1	$256 \times 8 \times 8$
	relu	-	$256 \times 8 \times 8$
Conv Block4	conv1	$256 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 4 \times 4$
	relu	-	$512 \times 4 \times 4$
	conv2	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 4 \times 4$
	relu	-	$512 \times 4 \times 4$
	conv3	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 4 \times 4$
	relu	-	$512 \times 4 \times 4$
	conv4	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 4 \times 4$
	relu	-	$512 \times 4 \times 4$
Conv Block5	conv1	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
	conv2	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
	conv3	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
	conv4	$512 \times 512 \times 3 \times 3$, stride=1, padding=1	$512 \times 2 \times 2$
	relu	-	$512 \times 2 \times 2$
maxpool		2×2 , stride=2	$512 \times 1 \times 1$
fc1		512×4096	4096
relu		-	4096
fc2		4096×4096	4096
relu		-	4096
fc3		4096×10	10
relu		-	10

Table 5.2.4: VGG16 Structure(MNIST) [90]. Conv Block is the repeating convolution blocks with repeating convolution-maxpool structures inside. Appendix A.2.4

Chapter 6

Results and Analysis

This chapter presents the performance evaluation of SGXDL and HybridSGXDL in real-world DL applications, analyzing both macro and micro-level performance aspects. It examines the unique features of SGXDL and HybridSGXDL, identifying optimal scenarios for each framework. A comparison with Piranha further highlights the superior efficiency of SGXDL and HybridSGXDL in secure computing environments. All data in this thesis were generated under a three-party LAN setup with 2 GB/s bandwidth and 2 ms latency.

6.1 Non-linear operation in Intel SGX

Non-linear operations in cryptographic computation, including MPC protocols, incur significant communication overhead and memory demands. While Piranha applies MPC protocols with GPU acceleration, ReLU execution remains slower than SGX's plaintext computation. A key challenge in GPU-accelerated MPC is that GPUs are optimized for floating-point arithmetic with reasonable memory size, whereas MPC require high memory and primarily integer-based operations(as described in Section 1.1.1).

As indicated by Figure 6.4.1, the ReLU plain-text computation in SGX is far more efficient and quick than Piranha's secure computation of MPC protocols with the acceleration of GPUs, which provides the idea to put the computation especially the non-linear computation of plain-text computation format in SGX enclave with the protection of hardware security techniques. This observation provides the evidence

and support to propose the non-linear operation into SGX enclave for plain-text computation.

It can also be observed that, the execution time from the beginning of a small computation workload does not increase until the ReLU computation workload size becomes larger than 1 MB, which indicates that the beginning lowest of 2ms is mainly the time to call SGX interface and transform the data to enclave.

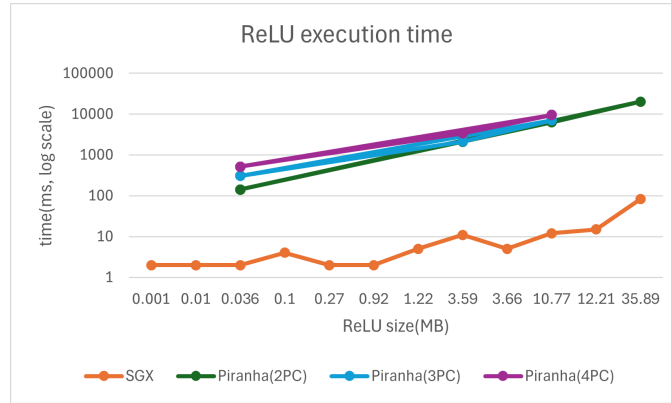


Figure 6.1.1: Comparison of ReLU Execution Time between SGX and Piranha with Different Protocols(ms). The x-axis represents the data size for ReLU in megabytes (MB), while the y-axis shows the execution time in milliseconds on a logarithmic scale. The results indicate that the SGX implementation achieves significantly lower execution times compared to Piranha across all protocols, with a notable increase in execution time for Piranha as the number of parties increases from 2PC to 4PC. This trend highlights the trade-off between security and computational efficiency in multi-party computation frameworks.

6.2 Efficiency Comparison SGXDL vs. HybridSGXDL vs. Piranha

From Table 6.2.1, we can conclude that SGXDL is the most efficient secure computation infrastructure for moderate computation workloads, which have been useful for most CNN industry applications [31, 82, 94]. On the other hand, HybridSGXDL demonstrates superior performance for large models and high-batch training computations, making it a feasible solution for secure training of LLMs. Both SGXDL and HybridSGXDL outperform Piranha in terms of computation efficiency. Piranha faces limitations due to GPUs memory overflow caused by high memory requirements, while SGXDL and HybridSGXDL, which use plain-text computation with reasonable memory requirements, do not suffer from this disadvantage.

			SGXDL	HybridSGXDL (3PC)	Piranha (2PC)	Piranha (3PC)	Piranha (4PC)
batchsize=1	LeNet	MNIST	15.00	12,272.41	588.35	1,618.52	2,462.94
		Cifar10	29.50	11,845.01	1,011.81	1,618.52	2,161.96
	AlexNet	MNIST	1,415.70	24,965.14	13,013.80	2,886.52	7,013.60
		Cifar10	1,812.55	23,977.07	3,431.93	2,765.13	6,996.19
	ResNet18	MNIST	3,621.14	63,932.61	73,486.00	22,878.86	57,304.80
		Cifar10	3,642.14	63,130.10	27,838.00	23,484.04	57,467.20
	VGG16	MNIST	11,645.89	76,775.55	16,905.30	13,959.71	29,298.70
		Cifar10	13,000.70	64,503.51	16,462.60	27,897.08	29,152.80
	batchsize=100	LeNet	MNIST	232.40	12,298.98	7,805.07	1,337.70
			Cifar10	451.40	12,624.20	7,991.68	6,019.23
		AlexNet	MNIST	17,032.80	25,952.27	100,063.00	107,841.10
			Cifar10	21,599.30	28,661.03	99,370.00	63,969.34
		ResNet18	MNIST	43,236.20	67,048.59	567,553.00	N/A
			Cifar10	43,933.70	65,331.23	499,207.00	N/A
		VGG16	MNIST	168,929.90	81,493.17	N/A	N/A
			Cifar10	131,468.90	67,897.09	N/A	N/A
batchsize=300	LeNet	MNIST	567.00	13,065.24	22,753.20	17,391.69	38,193.00
		Cifar10	1,083.60	13,141.89	22,174.70	17,758.10	38,951.50
	AlexNet	MNIST	48,122.90	27,641.74	295,326.00	N/A	N/A
		Cifar10	62,005.30	28,240.25	294,758.00	N/A	N/A
	ResNet18	MNIST	122,788.40	70,640.84	N/A	N/A	N/A
		Cifar10	125,122.80	66,859.13	N/A	N/A	N/A
	VGG16	MNIST	519,591.67	82,328.94	N/A	N/A	N/A
		Cifar10	358,065.00	75,625.55	N/A	N/A	N/A
batchsize=1000	LeNet	MNIST	1,916.33	13,853.92	75,242.90	N/A	N/A
		Cifar10	3,595.83	16,421.78	75,242.90	N/A	N/A
	AlexNet	MNIST	160,078.83	35,939.76	N/A	N/A	N/A
		Cifar10	203,220.67	28,807.49	N/A	N/A	N/A
	ResNet18	MNIST	405,699.17	91,033.73	N/A	N/A	N/A
		Cifar10	407,061.33	106,064.55	N/A	N/A	N/A
	VGG16	MNIST	1,774,915.67	137,688.56	N/A	N/A	N/A
		Cifar10	1,136,187.33	138,133.41	N/A	N/A	N/A

Table 6.2.1: Overall execution time (ms) for training a single batch of images using different batch sizes (batchsize=1, 100, 300, 1000 for SGXDL, HybridSGXDL, Piranha). Each value represents the total time taken to process a batch of images and subsequently update the model parameters. For instance, batchsize=1 indicates training on a single image followed by a model update, while batchsize=100 refers to training on 100 images in a batch before updating the model. 'N/A' denotes the memory overflow resulting in not available to execute the computation. The bold font denotes the efficiency winner of the same computation workload. The overall execution time can consist of computation time, communication time and exchange time etc.

HybridSGXDL starts with a high 12,272.41 ms execution time for the smallest computation workloads due to the exchange overhead between the CPU and GPUs. In contrast, SGXDL directly calculates the plain-text data with only one initial communication overhead in the enclave, and the exchange time between the enclave and CPU is nearly zero for normal data sizes. However, as the data computation size increases, HybridSGXDL shows its advantage with GPUs acceleration for floating-point computations.

The datasets CIFAR-10 and MNIST use the same CNN structure, resulting in almost identical computation workloads for the same number of batches. SGXDL and HybridSGXDL exhibit similar execution times when the computation workload is consistent. However, Piranha's execution time can be highly unstable. For instance, The case of Piranha (2PC) with batch size 1 and AlexNet shows significantly different execution times on MNIST and CIFAR-10, at 13,013.80 ms and 3,431.93 ms, respectively, despite their almost the same computation workloads. Several other pairs with similar workloads also demonstrate this instability, as shown in Table 6.2.1.

This instability in Piranha using MPC protocols with GPUs acceleration can be attributed to several factors. Firstly, Piranha is heavily impacted by communication overheads, causing the CPU to wait for communication and the GPUs to operate below full capacity. Piranha needs more communication rounds because it has to handle the synchronization between communication and GPUs computation, in comparison with SGXDL and HybridSGXDL's single workflow. The frequent data exchanges between the CPU and GPUs, along with the constant communication requirement, further exacerbate the issue. Although GPUs are capable of handling large workloads in parallel, their efficiency can be hindered by limited communication speed. This constraint forces GPUs to either wait for all data to arrive, incurring communication overhead, or proceed with computations despite incomplete data, leading to additional CPU-GPUs exchanges to handle the remaining workloads. Both scenarios result in suboptimal operation. Even when all data is eventually received, the CPU may still need to wait for the GPUs to complete its current tasks. In contrast, with HybridSGXDL, GPUs at local parties only begin processing after receiving all data shares for the current iteration, ensuring more efficient synchronization and operation. The complex synchronization of Piranha leads to unpredictable waiting periods during training, resulting in highly variable and unstable execution times, as illustrated in Table 6.2.1. In contrast, SGXDL and HybridSGXDL, as shown in Figure 3.3.1, follow a

Number of images	100	300	1000	60000
MNIST	4	15	50	3,055
Cifar10	19	58	197	12,062

Table 6.3.1: Communication time required to transfer images of various sizes (ms) in a 3-party and LAN 2GB/s setup. It shows the time taken by the parties to send data shares for a specific number of samples to the SGX server. This communication time can remain consistent regardless of the number of parties, as all parties send their data shares simultaneously. However, when a large number of parties are sending samples concurrently, the server’s receiving capacity needs to be further evaluated for the communication time.

clear and single workflow without requiring such complex synchronization, leading to stable execution efficiency.

6.3 SGXDL performance analysis

6.3.1 SGXDL overhead analysis

This section demonstrates the overhead in SGXDL for a single batch of training, which includes communication overhead, exchange time between the CPU and enclave, and data shares integration time. The analysis concludes that these overheads are nearly zero in practical training scenarios. This implies that SGXDL primarily spends time on plain-text computation, which is the main reason why SGXDL outperforms Piranha’s cryptographic computation, which suffers heavily from communication overhead and CPU-GPUs data exchange overhead.

Communication Time

Since each training session only requires a batch size of data samples, training can begin immediately after receiving the first batch of samples. During training, the server continues to receive additional data sample shares. As indicated in Figure 4.5.1, the communication overhead only involves the transmission of the first batch of data sample shares. Table 6.3.1 demonstrates the communication time for different sizes of shares. For example, the 50 ms communication overhead for a batch size of 1000 MNIST images in SGXDL constitutes only a small part of the overall batch training time.

Batchsize	CNNs model	Dataset	Communication overhead
batchsize=100	LeNet	MNIST	1.72%
		Cifar10	4.21%
	AlexNet	MNIST	0.02%
		Cifar10	0.09%
	ResNet18	MNIST	0.01%
		Cifar10	0.04%
	VGG16	MNIST	0.00%
		Cifar10	0.01%
batchsize=300	LeNet	MNIST	2.65%
		Cifar10	5.35%
	AlexNet	MNIST	0.03%
		Cifar10	0.09%
	ResNet18	MNIST	0.01%
		Cifar10	0.05%
	VGG16	MNIST	0.00%
		Cifar10	0.00%
batchsize=1000	LeNet	MNIST	2.61%
		Cifar10	5.48%
	AlexNet	MNIST	0.03%
		Cifar10	0.10%
	ResNet18	MNIST	0.01%
		Cifar10	0.05%
	VGG16	MNIST	0.00%
		Cifar10	0.02%

Table 6.3.2: Communication overhead for a batch of training of SGXDL(3PC). It shows the communication overhead as a percentage of total execution time for training a batch in SGXDL (3PC) across different batch sizes (batchsize=100, 300, 1000). Each value represents the proportion of total time consumed for inter-party communication relative to the overall training time for processing a batch of images and updating the model parameters. For example, batchsize=100 indicates training on 100 images in a batch before a model update, whereas batchsize=300 indicates training on 300 images before updating the model. The overhead values demonstrate that under certain conditions, the communication overhead remains relatively low and can approach zero as the batch size or model size increases.

Number of images(double)	1000	10000	20000	30000	40000	60000	800000	180000	240000
MNIST	~ 0	1	2	3	4	7	23	104	148
Cifar10	~ 0	7	9	14	56	100	150	N/A	N/A

Table 6.3.3: Exchange time (ms) between CPU and enclave for different data sizes. The table illustrates the time required for data exchange between the CPU and the secure enclave, measured for varying numbers of images processed from the MNIST and Cifar10 datasets. Each value represents the latency of data transfer when using double-precision data types. The ~ 0 indicates an exchange time so minimal to zero that it cannot be accurately measured by timing functions, effectively making it negligible, while "N/A" denotes cases where the data exchange size exceeds the processing capacity.

Number of parties	2 PC	3 PC	4 PC	10 PC	15 PC	20 PC	40 PC	50 PC
Integration time(ms)	4	4	5	5	5	5	6	9

Table 6.3.4: The integration time (ms) of data shares in the enclave for 60,000 MNIST images. The table demonstrates the integration time required to retrieve the original data in enclave under different party conditions. The data type is double.

The exchange time between CPU and enclave

Table 6.3.3 demonstrates that in SGXDL and HybridSGXDL, the data exchange time between the enclave and CPU is nearly zero. This is because each training session processes a batch of images, with a maximum batch size of 1000. Therefore, in SGXDL and HybridSGXDL, the data size for each exchange is not larger than this 1000-image batch, resulting in minimal exchange time. The enclave, being a physical area on the CPU, allows for this minimal exchange time. This is in stark contrast to the exchange time between the CPU and GPUs, where data exchange often constitutes a major overhead in GPUs computations.

The integration time for data shares

As we know, data shares need to be integrated in the enclave before executing plain-text computation. Table 6.3.4 shows that the integration overhead would be nearly zero in applications because, in SGXDL, each training and update typically processes a batch of data samples' shares, with a maximum batch size of 1000. This is much smaller than the data sizes in the Table 6.3.4. In extreme cases with many participating parties, there would be some integration time in account, but it would still represent a very small fraction of the total computation training time for SGXDL.

Since SGXDL executes the entire plain-text training process within the enclave, the number of parties only adds overhead related to the data size for communication, exchange, and integration, which overheads being nearly zero in most cases. Therefore, we can conclude that the number of participating parties has a very limited influence on SGXDL's performance, making it a viable solution for scenarios involving a large number of participating parties.

SGXDL computation ability analysis

As evidenced in Table 6.3.5, the computational capability of Intel SGX for CNNs is inferior even to that of the CPU. The Intel SGX environment supports only 9 threads

for parallel computing, in contrast to the CPU, which can utilize up to 144 threads. This disparity results in significantly slower computation times for SGX compared to the CPU. Additionally, the CPU itself underperforms compared to GPUs acceleration for CNNs. Consequently, SGXDL is less efficient than HybridSGXDL for handling large data computation workloads.

6.4 HybridSGXDL performance analysis

6.4.1 Non-linear and Arithmetic Computation

From Tables 6.4.1, 6.4.2, and 6.4.3, it is evident that non-linear operations, such as ReLU computation in Piranha, are significantly more expensive than secure plain-text ReLU computation in HybridSGXDL. This discrepancy primarily affects Piranha's performance, while HybridSGXDL benefits from it. For arithmetic operations in HybridSGXDL, convolution and fully connected layers, the execution time for small and moderate tasks is similar, with overall time mainly consisting of CPU-GPUs exchange time. Although Piranha outperforms HybridSGXDL in arithmetic computations for small and moderate workloads, its higher memory requirement leads to slower execution compared to HybridSGXDL and memory overflow for larger batch sizes and models, making it unfeasible.

6.4.2 Communication overhead

From Figure 6.4.2, we can infer several critical insights regarding the communication overhead of the evaluated protocols. Piranha with 2-party and 3-party protocols suffers significantly from communication overhead. Although the 4-party Piranha protocol appears to exhibit a lower communication overhead, it encounters numerous conditions it cannot execute due to large memory requirements, leading to memory overflow.

HybridSGXDL consistently maintains the lowest communication overhead compared to Piranha across all data points. Although its communication time increases with larger computational workloads, it is not significantly impacted by the workload size. This is because the primary computation time is dominated by the efficient data exchange between the GPUs and CPU, where GPUs computation remains fast regardless of data size. In contrast, Piranha's communication time increases

Batchsize	CNNs model	Dataset	CPU	Intel SGX
batchsize=1	LeNet	MNIST	7.33	15.00
		Cifar10	10.02	29.50
	AlexNet	MNIST	929.47	1,415.70
		Cifar10	3,833.64	1,812.55
	ResNet18	MNIST	2,815.83	3,621.13
		Cifar10	4,156.36	3,642.14
	VGG16	MNIST	4,665.35	11,645.89
		Cifar10	10,342.32	13,000.70
batchsize=100	LeNet	MNIST	31.51	232.40
		Cifar10	41.86	451.40
	AlexNet	MNIST	2,445.56	17,032.80
		Cifar10	7,061.73	21,599.30
	ResNet18	MNIST	10,977.70	43,236.20
		Cifar10	11,289.12	43,933.70
	VGG16	MNIST	20,770.98	168,929.90
		Cifar10	20,312.35	131,468.90
batchsize=300	LeNet	MNIST	88.43	567.00
		Cifar10	95.26	1,083.60
	AlexNet	MNIST	7,377.75	48,122.90
		Cifar10	8,695.30	62,005.30
	ResNet18	MNIST	35,786.55	122,788.40
		Cifar10	21,117.2	125,122.8
	VGG16	MNIST	68,704.3	519,591.6667
		Cifar10	94,676.65	358,065.00
batchsize=1000	LeNet	MNIST	244.93	1,916.33
		Cifar10	268.43	3,595.83
	AlexNet	MNIST	20,192.00	160,078.83
		Cifar10	25,138.83	203,220.67
	ResNet18	MNIST	97,536.67	405,699.17
		Cifar10	119,430.17	407,061.33
	VGG16	MNIST	238,136.00	1,774,915.67
		Cifar10	271,556.67	1,136,187.33

Table 6.3.5: Computation time comparison between CPU and Intel SGX (ms) for different batch sizes and CNN models. The table shows the time required to train a single batch of images using either a standard CPU or Intel SGX, followed by a model parameter update. The comparison is conducted for four batch sizes (batchsize=1, 100, 300, 1000), using four CNN models (LeNet, AlexNet, ResNet18, VGG16) on two datasets (MNIST and Cifar10). Each entry indicates the execution time required in milliseconds. The results demonstrate that Intel SGX, despite providing secure computation, incurs a noticeable performance overhead compared to standard CPU processing. In most cases, the time taken using Intel SGX is significantly higher than that of a regular CPU due to the additional layers of security and processing inside the SGX enclave. This comparison highlights the trade-off between security and computational efficiency when using secure enclaves like Intel SGX.

CNNs model	Dataset	Secure computation	ReLU	Conv	Fc
LeNet	MNIST	HybridSGXDL	20.42	9,788.42	2,454.76
		Piranha(2PC)	449.01	108.29	24.31
		Piranha(4PC)	1,663.87	672.95	97.75
	Cifar10	HybridSGXDL	22.70	10,104.54	6,437.34
		Piranha(2PC)	872.90	104.51	26.46
		Piranha(4PC)	1,676.86	348.28	136.82
AlexNet	MNIST	HybridSGXDL	40.11	17,294.70	7,613.41
		Piranha(2PC)	6,123.99	3,483.95	3,971.81
		Piranha(4PC)	4,162.24	1,432.48	1,390.19
	Cifar10	HybridSGXDL	42.36	17,663.05	8,572.06
		Piranha(2PC)	1,692.40	795.23	934.24
		Piranha(4PC)	4,070.43	1,475.19	1,450.55
ResNet18	MNIST	HybridSGXDL	148.27	60,316.00	3,427.16
		Piranha(2PC)	20,307.40	45,882.49	26.33
		Piranha(4PC)	18,854.71	25,714.94	142.07
	Cifar10	HybridSGXDL	85.01	59,677.30	3,347.17
		Piranha(2PC)	6,720.78	17,682.75	23.52
		Piranha(4PC)	18,483.81	26,096.50	114.66
VGG16	MNIST	HybridSGXDL	138.25	64,749.39	11,844.76
		Piranha(2PC)	5,470.29	11,251.64	176.01
		Piranha(4PC)	12,715.48	16,150.69	404.17
	Cifar10	HybridSGXDL	148.00	53,938.77	10,365.97
		Piranha(2PC)	5,234.41	11,062.23	157.72
		Piranha(4PC)	12,705.31	16,015.11	432.37

Table 6.4.1: Computation time comparison of arithmetic and non-linear operations for different CNN models during the training of a single batch (**batch size = 1**) with model updates in HybridSGXDL(3PC). This table presents the execution time (ms) for ReLU and arithmetic operations (Conv: convolution, and Fc: fully connected neural layer) across four CNN models (LeNet, AlexNet, ResNet18, and VGG16) on two datasets (MNIST and Cifar10). The comparison is conducted for three secure computation infrastructures: HybridSGXDL, Piranha(2PC), and Piranha(4PC). The data for Piranha(3PC) was lost during data generation and tidying, thus it is not included in the table. Each value indicates the time taken to process and update the model for a single batch, highlighting the computational overhead of arithmetic and non-linear operations associated with different secure techniques.

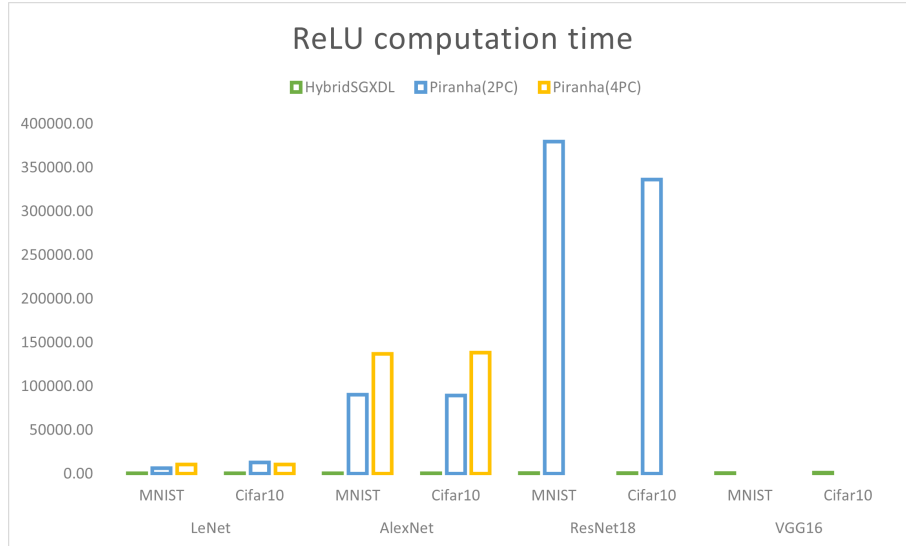


Figure 6.4.1: ReLU computation time comparison for HybridSGXDL(3PC) and Piranha during a single batch of training (batchsize = 100). The figure shows the ReLU computation time (ms) for the same workload in HybridSGXDL and Piranha (2PC and 4PC). Since the ReLU operation is computed inside the secure enclave in plain-text, the computation time remains the same in HybridSGXDL regardless of the number of parties (2PC, 3PC, 4PC, or more). The results indicate that Piranha incurs significantly higher ReLU computation overhead. Notably, data points for Piranha(4PC) are missing for ResNet18 and VGG16 as the computations caused memory overflows on the GPUs due to excessive overhead beyond the GPUs’s capacity. Similarly, Piranha(2PC) faced memory overflow issues with ResNet18 and VGG16, resulting in their absence in the figure. Although these overhead values are not shown, they are obviously much higher than HybridSGXDL. Additionally, Piranha(3PC) data points were lost during the data organization phase.

dramatically with larger model sizes, making it unsuitable for the secure training of large-scale models.

The substantial communication overhead in Piranha is primarily attributed to the frequent data share exchanges required for non-linear operations such as ReLU and Maxpool. Each of these operations necessitates multiple rounds of communication between parties, significantly increasing overhead.

In contrast, HybridSGXDL reduces communication overhead by performing secure plain-text floating-point computations with the assistance of SGX, requiring data share exchanges only as many times as the number of layers in CNNs. This single workflow communication strategy enables HybridSGXDL to maintain lower overhead and demonstrates its efficiency in handling larger computational workloads.

The significant communication overhead observed in Piranha protocols, particularly

CNNs model	Dataset	Secure computation	ReLU	Conv	Fc
LeNet	MNIST	HybridSGXDL	38.00	10,018.08	2,120.84
		Piranha(2PC)	6,488.14	807.35	24.03
		Piranha(4PC)	10445.78	1,986.38	115.88
	Cifar10	HybridSGXDL	34.75	9,991.44	2,436.28
		Piranha(2PC)	13020.74	794.57	24.09
		Piranha(4PC)	10,634.79	1,978.01	1,095.00
AlexNet	MNIST	HybridSGXDL	445.34	17,549.41	7,615.08
		Piranha(2PC)	90,160.70	8,169.61	1,240.08
		Piranha(4PC)	136,924.35	18,597.40	1,646.91
	Cifar10	HybridSGXDL	371.33	17,583.80	7,668.20
		Piranha(2PC)	89612.17	8,083.19	1,193.79
		Piranha(4PC)	138,401.64	18,334.15	2,694.90
ResNet18	MNIST	HybridSGXDL	581.09	62,805.01	3,298.77
		Piranha(2PC)	379,778.39	56,497.60	47.32
		Piranha(4PC)	N/A	N/A	N/A
	Cifar10	HybridSGXDL	648.20	60,432.96	3,729.67
		Piranha(2PC)	336402.31	50,944.13	48.21
		Piranha(4PC)	N/A	N/A	N/A
VGG16	MNIST	HybridSGXDL	982.80	67,135.34	12,960.23
		Piranha(2PC)	N/A	N/A	N/A
		Piranha(4PC)	N/A	N/A	N/A
	Cifar10	HybridSGXDL	1,061.00	55,810.51	10,145.53
		Piranha(2PC)	N/A	N/A	N/A
		Piranha(4PC)	N/A	N/A	N/A

Table 6.4.2: Computation time comparison of arithmetic and non-linear operations for different CNN models during the training of a single batch (**batch size = 100**) with model updates in HybridSGXDL(3PC). "N/A" indicates a GPUs memory overflow error, making execution unavailable. The table presents the same data and experimental setup with Figure 6.4.1 under a larger batchsize=100.

as the model size increases, suggests that these protocols may not be suitable for applications requiring efficient and scalable secure training of large models. On the other hand, HybridSGXDL's ability to maintain low and stable communication overhead makes it a more practical solution for such applications, offering better performance and scalability.

6.4.3 Exchange overhead between CPU and GPUs

We analyze the computation and exchange overhead in HybridSGXDL via Figures 6.4.3, 6.4.4, and 6.4.6.

From Figure 6.4.6, we observe that the computation overhead in HybridSGXDL, which includes both non-linear operations on the CPU with SGX and arithmetic operations on the GPUs, decreases as the batch size increases. This trend is consistent across different neural network models such as LeNet, AlexNet, ResNet18, and VGG16. This indicates that the system becomes more efficient in handling larger batch sizes, leading to a reduction in the relative overhead.

CNNs model	Dataset	Secure computation	ReLU	Conv	Fc
LeNet	MNIST	HybridSGXDL	57.28	10,601.25	2,003.70
		Piranha(2PC)	18,863.01	2,428.85	40.19
		Piranha(4PC)	29,661.99	5,508.42	121.90
	Cifar10	HybridSGXDL	46.43	10,118.56	2,489.78
		Piranha(2PC)	18,748.62	1,963.08	36.20
		Piranha(4PC)	30,268.89	5,702.77	2,979.77
AlexNet	MNIST	HybridSGXDL	1,056.59	18,145.94	7,518.71
		Piranha(2PC)	268,962.80	23,033.49	1,899.78
		Piranha(4PC)	N/A	N/A	N/A
	Cifar10	HybridSGXDL	1,013.14	17,663.05	8,572.35
		Piranha(2PC)	268,029.46	23,097.97	1,882.24
		Piranha(4PC)	N/A	N/A	N/A
ResNet18	MNIST	HybridSGXDL	1,704.54	64,257.77	3,279.16
		Piranha(2PC)	N/A	N/A	N/A
		Piranha(4PC)	N/A	N/A	N/A
	Cifar10	HybridSGXDL	581.50	62,191.56	3,379.08
		Piranha(2PC)	N/A	N/A	N/A
		Piranha(4PC)	N/A	N/A	N/A
VGG16	MNIST	HybridSGXDL	3,288.38	65,116.62	12,446.55
		Piranha(2PC)	N/A	N/A	N/A
		Piranha(4PC)	N/A	N/A	N/A
	Cifar10	HybridSGXDL	3,018.00	59,626.80	10,258.74
		Piranha(2PC)	N/A	N/A	N/A
		Piranha(4PC)	N/A	N/A	N/A

Table 6.4.3: Computation time comparison of arithmetic(convolution, full-connected layer) and non-linear operations such as ReLU for different CNN models during the training of a single batch (**batch size = 300**) with model updates in HybridSGXDL(3PC). "N/A" indicates GPUs memory overflow error, making execution unavailable. Conv is the convolution, and Fc is the full-connected layer. The table presents the same data and experimental setup with Figure 6.4.1 under a larger batchsize=300.

Batchsize	CNNs model	Dataset	HybridSGXDL	Piranah(2PC)	Piranha(3PC)	Piranha(4PC)
batchsize=100	LeNet	MNIST	97.00	5,567.12	253.76	163.25
		Cifar10	131	5,734.14	4,492.22	460.76
	AlexNet	MNIST	1,040.00	76,375.15	88,994.24	2,072.67
		Cifar10	1,385	75,660.33	60,740.58	3,954.54
	ResNet18	MNIST	2,821	307,534.48	N/A	N/A
		Cifar10	2,851.00	305,505.69	N/A	N/A
	VGG16	MNIST	4,406.00	N/A	N/A	N/A
		Cifar10	4,425.00	N/A	N/A	N/A
batchsize=300	LeNet	MNIST	345.00	17,141.27	13,499.26	936.01
		Cifar10	440	17,724.98	13,816.36	1,382.27
	AlexNet	MNIST	3,201.00	229,091.92	N/A	N/A
		Cifar10	4,206.00	237,010.01	N/A	N/A
	ResNet18	MNIST	8,477.00	N/A	N/A	N/A
		Cifar10	8,552.00	N/A	N/A	N/A
	VGG16	MNIST	13,214.00	N/A	N/A	N/A
		Cifar10	13,293.00	N/A	N/A	N/A
batchsize=1000	LeNet	MNIST	1,149.00	59,986.84	N/A	N/A
		Cifar10	1,510.00	59,942.62	N/A	N/A
	AlexNet	MNIST	10,637.00	N/A	N/A	N/A
		Cifar10	14,093.00	N/A	N/A	N/A
	ResNet18	MNIST	28,311.00	N/A	N/A	N/A
		Cifar10	28,774.00	N/A	N/A	N/A
	VGG16	MNIST	44,552.00	N/A	N/A	N/A
		Cifar10	44,342.00	N/A	N/A	N/A

Table 6.4.4: Communication time (ms) in HybridSGXDL(3PC) and Piranha during a single batch of training across different batch sizes (100, 300, 1000). Results are provided for four CNN models (LeNet, AlexNet, ResNet18, VGG16) using two datasets (MNIST and Cifar10). The secure computation methods include HybridSGXDL and Piranha with different numbers of parties (2PC, 3PC, and 4PC). "N/A" in Piranha indicates cases where GPUs memory overflow occurred, preventing the training of the corresponding CNN workloads.

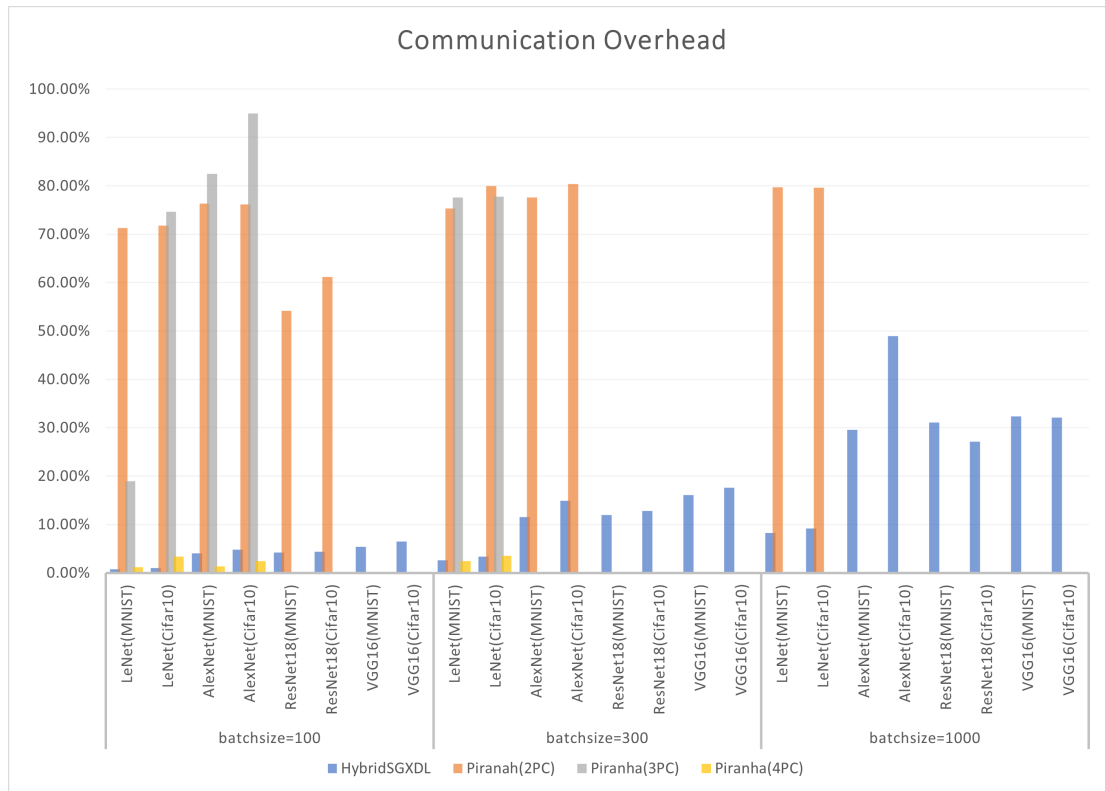


Figure 6.4.2: Communication Overhead Comparison across HybridSGXDL(3PC) and Piranha for a single batch of training. This figure compares the communication overhead for different neural network architectures (LeNet, AlexNet, ResNet18, and VGG16) across various batch sizes (100, 300, and 1000). The communication overhead is expressed as a percentage for HybridSGXDL and Piranha (2PC, 3PC, and 4PC). Notably, for batch sizes of 100, 300 and 1000, several data points are missing for Piranha(2PC, 3PC, 4PC) with corresponding CNNs models due to large memory requirements that caused GPUs overflow, making it impossible to execute these models to get the data. **The absence of these data points indicates extremely high communication overheads can not be handled by our equipment rather than zero values.** This suggests that when batch sizes reach such levels, Piranha incurs significant communication overhead, which negatively impacts its performance.

Figure 6.4.4 further breaks down the exchange overhead between the CPU and GPUs. It shows a potential downward trend in overhead as the computational workload increases with larger batch sizes. This suggests that the system's efficiency improves with increased workloads, likely due to better parallel processing and reduced relative communication time between the CPU and GPUs. Additionally, Figure 6.4.3 provides a detailed comparison of the exchange overhead between different models and batch sizes. It highlights that, although the exchange overhead constitutes a significant portion of the total computation overhead, this overhead decreases as the batch size increases. For instance, the overhead for LeNet, AlexNet, ResNet18, and VGG16 is substantially higher at smaller batch sizes but diminishes as the batch size grows,

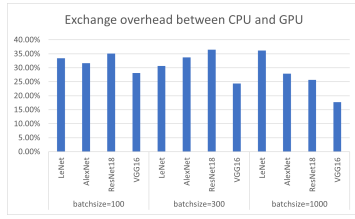


Figure 6.4.3: The figure shows the overhead incurred during a single batch of training in HybridSGXDL, which consists primarily of the exchange time between the CPU and GPUs. The exchange overhead reflects the proportion of time spent transferring data between the CPU and GPUs, which is a key component of the overall execution time.

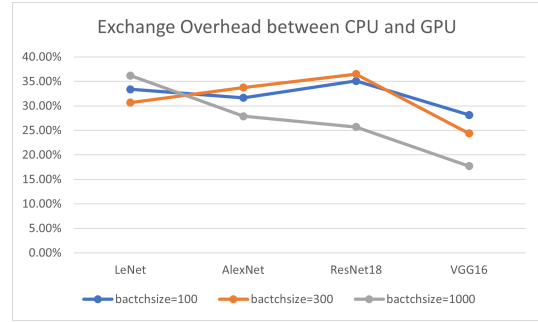


Figure 6.4.4: The figures indicate a potential downward trend in CPU and GPUs exchange overhead as the computational workload increases with larger model sizes for a single batch of training in HybridSGXDL.

Figure 6.4.5: Exchange overhead CPU and GPUs in HybridSGXDL(3PC, MNIST), the figure describes the exchange overhead between CPU and GPUs of HybridSGXDL among various CNNs models and batchsizes.

indicating improved efficiency and scalability in a single batch of training.

In summary, these figures collectively demonstrate that while the exchange overhead between the CPU and GPUs is a significant component of the total execution time, it exhibits a decreasing trend with increasing computational workloads and batch sizes. This also aligns with our analysis in Section 6.3.1, where we concluded that the exchange overhead between the CPU and SGX is minimal and negligible. The efficient handling of larger computational workloads by HybridSGXDL highlights its potential for scalability and performance optimization in secure DL applications.

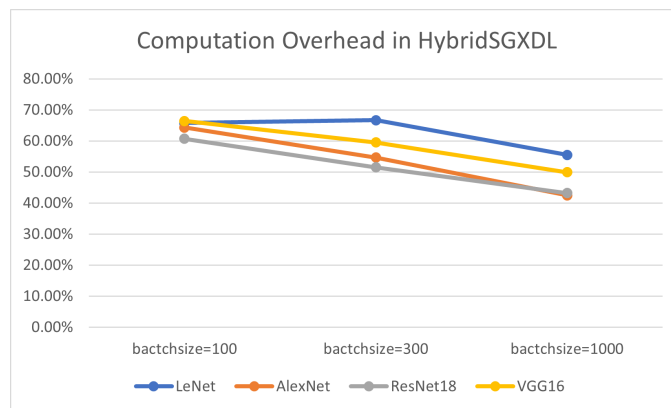


Figure 6.4.6: Computation Overhead of HybridSGXDL(3PC) on the MNIST dataset during a single batch of training. The figure shows the computation overhead as a percentage of the overall execution time across various CNN models (LeNet, AlexNet, ResNet18, and VGG16) and different batch sizes (100, 300, 1000). The overall execution time consists of three components: computation time, communication time, and exchange time. In this context, computation time refers to the time spent performing arithmetic and non-linear operations, such as convolutional layers and ReLU activations.

Batchsize	CNNs model	Exchange Time
batchsize=100	LeNet	4,108.15
	AlexNet	8,216.30
	ResNet18	23,528.87
	VGG16	22,939.80
batchsize=300	LeNet	4,008.90
	AlexNet	9,328.69
	ResNet18	25,774.83
	VGG16	20,071.21
batchsize=1000	LeNet	5,014.02
	AlexNet	10,028.05
	ResNet18	23,402.26
	VGG16	24,381.48

Table 6.4.5: Exchange time between the CPU and GPUs in HybridSGXDL(3PC) during a single batch of training using the MNIST dataset. The table presents the exchange time (ms) for different CNN models (LeNet, AlexNet, ResNet18, and VGG16) across varying batch sizes (100, 300, 1000). Exchange time refers to the duration required to transfer data between the CPU and GPUs during a batch of training process.

6.4.4 Accuracy

Figure 6.4.7 illustrates that HybridSGXDL maintains accuracy without drop during secure computation, which has also been proofed in Section 4.3.1. This is achieved by executing floating-point plain-text calculations securely, ensuring precise and accurate results. SGXDL also maintains plain-text computation without accuracy drop, highlighting its effectiveness in secure environments.

In contrast, Piranha encounters accuracy concerns due to its use of integer cryptography calculations. By approximating non-linear operations like ReLU with polynomials, Piranha inevitably suffers from precision loss. This comparison underscores the advantage of HybridSGXDL and SGXDL in preserving computational accuracy while ensuring security.

The training loss curves for different models such as LeNet, VGG, and ResNet18 on the CIFAR-10 dataset show that HybridSGXDL effectively maintains low training loss across epochs. This consistent performance across various models demonstrates the robustness and reliability of HybridSGXDL in secure computation scenarios.

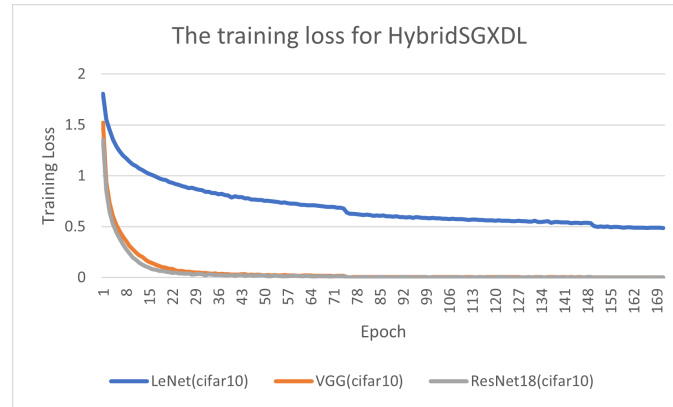


Figure 6.4.7: Training loss curves for different CNN models in HybridSGXDL(3PC). The figure illustrates the reduction in training loss over 170 epochs for three CNN models (LeNet, VGG, and ResNet18) on the whole CIFAR-10 dataset(60,000 samples). The y-axis represents the training loss, while the x-axis denotes the number of epochs.

6.5 Security Analysis

The section provides a primary security analysis for SGXDL and HybridSGXDL. The security concerns of data shares generation schemes in the thesis has been discussed in Section 2.3.4, so we eliminate it here. The secure computing infrastructure utilizes Intel SGX hardware, involving a server with the SGX enclave and several clients. All participants are assumed to be honest but curious (semi-honest), meaning they may attempt to retrieve useful information but will not maliciously attack the system by adding noise to the shares. The data are separated and distributed among participants in secret-sharing random data share formats. Secure channels are established to prevent the server from receiving all share values.

6.5.1 Side-Channel Attack for Intel SGX

A primary concern with Intel SGX is the potential for side-channel attacks, which can occur mainly through OCalls (calls from the enclave to execute functions outside the enclave) and interactions between the enclave and the external environment.

SGXDL

In SGXDL, the model is preserved within the enclave, and all computations are performed by SGX. The model remains within the enclave throughout the computing process. Interaction between the enclave and the outside world only occurs for dataset share exchanges, resulting in minimal interactions. There are no OCalls from the

enclave during computing, making side-channel attacks such as timing attacks, power attacks, and memory access pattern attacks unlikely. The entire training process is protected and executed within the enclave, with minimal observable interactions to outside environments. Since the data shares exchange operations are uniformly at the beginning of training, less useful information about the model or data can be inferred from observable behaviors. In the future, we can explore other TEEs techniques such as Intel TDX and TrustZone for strong security in the future, the thesis just provides an illustration and reference for the TEEs training methodologies.

HybridSGXDL

HybridSGXDL separates training into arithmetic and non-linear operations. This means the workflow and data process are not entirely oblivious to the participants, who can identify which parts of the model are convolution layers (arithmetic operations), and which parts are non-linear operations like MaxPool and ReLU. Participants can infer the structure of the CNN model by combining the data shares' structures and dimensions. However, the initial random parameters of the model are unknown to all parties outside the enclave, and only the updates in share format are transferred among participants, making the real values of model parameters unavailable to all parties.

In conclusion, HybridSGXDL may be susceptible to behavioral observation attacks to guess the structure of the CNN model, but the model parameter values remain secure. Under such consideration, SGXDL provides stronger security protection than HybridSGXDL, because SGXDL is an end-to-end black box with only communication at the beginning of training.

6.5.2 Gradients attack for Federated Learning

In traditional FL, attackers can obtain gradients transferred between the server and clients to retrieve information about data samples, such as membership and inference attacks. The SGXDL and HybridSGXDL using the wFL pattern, provides high security protection for the raw data and model to address the threat. The gradients and model in SGXDL do not leave the enclave because all computations are completed within the enclave. In HybridSGXDL, gradients are separated into random secret data shares, making the real values unavailable to all parties. Therefore, SGXDL and HybridSGXDL

protect gradients within the enclave or encrypt them into random shares, mitigating the common threats in traditional FL arising from gradient transfer processes.

Chapter 7

Conclusion

In this thesis, we explored the applications of SGX for enabling floating-point plain-text computation within the enclave to advance privacy-preserving DL computations. We proposed and implemented two secure plain-text weighted federated learning computing infrastructures, SGXDL and HybridSGXDL, leveraging the hardware security capabilities of Intel SGX. These infrastructures were applied to practical DL models, demonstrating significant performance improvements. Both SGXDL and HybridSGXDL surpass the efficiency of Piranha, a MPC protocol enhanced with GPUs acceleration, as reported in USENIX (2022). Our solutions achieve over 20× performance improvement and can train models that Piranha cannot due to memory overflow issues. SGXDL and HybridSGXDL make it feasible to securely train real-world neural networks with 100 million parameters of a floating data type (4 bytes), thus enabling secure training of LLMs.

Several factors contribute to the superior efficiency of SGXDL and HybridSGXDL over Piranha. These include the significant communication overhead for non-linear operations in Piranha, suboptimal GPUs utilization due to complex communication synchronization, and the high computation workload for cryptographic and integer-type computations, whereas GPUs are optimized for floating-point operations. SGXDL excels in moderate computing sessions, prevalent in many industrial scenarios, while HybridSGXDL outperforms for larger models and datasets. This advantage becomes more apparent as the execution workload grows. SGXDL’s superior performance in smaller and moderate sessions is attributed to lower communication overhead, with data communication required only at the beginning of training. Conversely,

HybridSGXDL’s higher time-consuming data exchange overhead between CPU and GPUs can take up to 2 seconds, compared to its 4 milliseconds of computing time on GPUs. However, for larger computation workloads, the GPUs’s superior matrix computation capabilities make HybridSGXDL the best performer.

7.1 Future Work

While SGXDL offers stronger security protection than HybridSGXDL, its efficiency in training large models with extensive datasets is currently limited due to its reliance on CPU-based computation. Future improvements could integrate GPUs support into SGX, significantly enhancing SGXDL’s efficiency and making its performance comparable to standard plain-text DL training without security measures. Achieving privacy-preserving training without sacrificing efficiency would then become feasible.

Moreover, the secure infrastructures have been implemented on models such as ResNet18 and VGG16 using double data types, demonstrating feasibility for LLMs with billions of float parameters. Future work can extend SGXDL and HybridSGXDL to larger models such as GPT and BERT. Additionally, the secure channel designed in this thesis is at a preliminary level. Future research can focus on developing a more detailed and secure channel mechanism to further enhance the security of these computing infrastructures.

With the integration of higher computational power offered by GPUs, the platform could potentially address certain current GPUs overflow scenarios marked as “N/A” in the performance evaluations. This improvement would allow for the calculation of previously missing data points, providing a more comprehensive demonstration of the computational efficiency of both HybridSGXDL and SGXDL. Additionally, the lost data points in the context of Piranha (3PC) could be recovered in future research, enabling a more thorough and accurate comparative analysis.

HybridSGXDL has been implemented and evaluated primarily in the 3PC scenario. Future work could explore other MPC protocols, such as 2PC and 4PC, to investigate their feasibility and performance in this framework. This would broaden the scope of HybridSGXDL and potentially optimize its efficiency and security under different multi-party configurations.

The experimental results in the thesis were obtained under a LAN network setup, characterized by low latency. However, in real-world scenarios, institutions and collaborating parties often operate in a WAN network environment. Such environments typically have higher latency and lower bandwidth, leading to significant communication overhead. This can further diminish the performance of Piranha, while highlighting the advantages of SGXDL and HybridSGXDL, which are less impacted by communication overhead.

Finally, this thesis employed Gaussian distribution to introduce randomness into the real-number secret shares. Future research could delve deeper into the security analysis of this secret-sharing approach or explore alternative, more secure secret-sharing schemes. Enhancing the robustness and security of the underlying secret-sharing mechanism would further solidify the platform's suitability for privacy-preserving ML applications.

Bibliography

- [1] Achiam, Josh, Adler, Steven, Agarwal, Sandhini, Ahmad, Lama, Akkaya, Ilge, Aleman, Florencia Leoni, Almeida, Diogo, Altschmidt, Janko, Altman, Sam, Anadkat, Shyamal, et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [2] Adams, Carlisle and Lloyd, Steve. *Understanding PKI: concepts, standards, and deployment considerations*. Addison-Wesley Professional, 2003.
- [3] Aono, Yoshinori, Hayashi, Takuya, Wang, Lihua, Moriai, Shiho, et al. “Privacy-preserving deep learning via additively homomorphic encryption”. In: *IEEE transactions on information forensics and security* 13.5 (2017), pp. 1333–1345.
- [4] Barak, Assi, Escudero, Daniel, Dalskov, Anders PK, and Keller, Marcel. “Secure Evaluation of Quantized Neural Networks.” In: *IACR Cryptol. ePrint Arch.* 2019 (2019), p. 131.
- [5] Beaver, D, Micali, S, and Rogaway, P. “The round complexity of secure protocols extended abstract”. In: *22nd ACM STOC*.
- [6] Beimel, Amos. “Secret-sharing schemes: A survey”. In: *International conference on coding and cryptology*. Springer. 2011, pp. 11–46.
- [7] Ben-Or, Michael, Goldwasser, Shafi, and Wigderson, Avi. “Completeness theorems for non-cryptographic fault-tolerant distributed computation”. In: *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*. 2019, pp. 351–371.

- [8] Bogdanov, Dan, Laur, Sven, and Willemson, Jan. “Sharemind: A framework for fast privacy-preserving computations”. In: *Computer Security-ESORICS 2008: 13th European Symposium on Research in Computer Security, Málaga, Spain, October 6-8, 2008. Proceedings 13*. Springer. 2008, pp. 192–206.
- [9] Bogdanov, Dan, Niitsoo, Margus, Toft, Tomas, and Willemson, Jan. “High-performance secure multi-party computation for data mining applications”. In: *International Journal of Information Security* 11 (2012), pp. 403–418.
- [10] Bonawitz, Keith, Ivanov, Vladimir, Kreuter, Ben, Marcedone, Antonio, McMahan, H Brendan, Patel, Sarvar, Ramage, Daniel, Segal, Aaron, and Seth, Karn. “Practical secure aggregation for privacy-preserving machine learning”. In: *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017, pp. 1175–1191.
- [11] Bouacida, Nader and Mohapatra, Prasant. “Vulnerabilities in federated learning”. In: *IEEE Access* 9 (2021), pp. 63229–63249.
- [12] Byrd, David and Polychroniadou, Antigoni. “Differentially private secure multi-party computation for federated learning in financial applications”. In: *Proceedings of the First ACM International Conference on AI in Finance*. 2020, pp. 1–9.
- [13] Chaudhari, Harsh, Choudhury, Ashish, Patra, Arpita, and Suresh, Ajith. “ASTRA: high throughput 3pc over rings with application to secure prediction”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*. 2019, pp. 81–92.
- [14] Chaudhari, Harsh, Rachuri, Rahul, and Suresh, Ajith. “Trident: Efficient 4pc framework for privacy preserving machine learning”. In: *arXiv preprint arXiv:1912.02631* (2019).
- [15] Chen, Yu, Luo, Fang, Li, Tong, Xiang, Tao, Liu, Zheli, and Li, Jin. “A training-integrity privacy-preserving federated learning scheme with trusted execution environment”. In: *Information Sciences* 522 (2020), pp. 69–79.
- [16] Cheng, Kewei, Fan, Tao, Jin, Yilun, Liu, Yang, Chen, Tianjian, Papadopoulos, Dimitrios, and Yang, Qiang. “Secureboost: A lossless federated learning framework”. In: *IEEE Intelligent Systems* 36.6 (2021), pp. 87–98.

- [17] Choi, Joseph I. and Butler, Kevin R. B. “Secure Multiparty Computation and Trusted Hardware: Examining Adoption Challenges and Opportunities”. In: *Security and Communication Networks* 2019 (Apr. 2, 2019), pp. 1–28. ISSN: 1939-0114, 1939-0122. DOI: 10.1155/2019/1368905. URL: <https://www.hindawi.com/journals/scn/2019/1368905/> (visited on 08/01/2023).
- [18] Costan, Victor and Devadas, Srinivas. “Intel SGX explained”. In: *Cryptology ePrint Archive* (2016).
- [19] Dalskov, Anders, Escudero, Daniel, and Keller, Marcel. “Fantastic four: {Honest-Majority} {Four-Party} secure computation with malicious security”. In: *30th USENIX Security Symposium (USENIX Security 21)*. 2021, pp. 2183–2200.
- [20] *Differential Privacy: How It Works, Benefits & Use Cases in 2024*. en. URL: <https://research.aimultiple.com/differential-privacy/> (visited on 05/04/2024).
- [21] Doganay, Mahir Can, Pedersen, Thomas B, Saygin, Yücel, Savaş, Erkan, and Levi, Albert. “Distributed privacy preserving k-means clustering with additive secret sharing”. In: *Proceedings of the 2008 international workshop on Privacy and anonymity in information society*. 2008, pp. 3–11.
- [22] Dong, Wentao and Wang, Cong. “Poster: Towards Lightweight TEE-Assisted MPC”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, pp. 3609–3611.
- [23] Dong, Ye, Chen, Xiaojun, Shen, Liyan, and Wang, Dakui. “Privacy-preserving distributed machine learning based on secret sharing”. In: *Information and Communications Security: 21st International Conference, ICICS 2019, Beijing, China, December 15–17, 2019, Revised Selected Papers 21*. Springer. 2020, pp. 684–702.
- [24] Du, Wenliang and Atallah, Mikhail J. “Secure multi-party computation problems and their applications: a review and open problems”. In: *Proceedings of the 2001 workshop on New security paradigms*. 2001, pp. 13–22.
- [25] El Ouadrhiri, Ahmed and Abdelhadi, Ahmed. “Differential privacy for deep and federated learning: A survey”. In: *IEEE access* 10 (2022), pp. 22359–22380.

- [26] Escudero, Daniel, Ghosh, Satrajit, Keller, Marcel, Rachuri, Rahul, and Scholl, Peter. “Improved primitives for MPC over mixed arithmetic-binary circuits”. In: *Advances in Cryptology—CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II 40*. Springer. 2020, pp. 823–852.
- [27] Evans, David, Kolesnikov, Vladimir, Rosulek, Mike, et al. “A pragmatic introduction to secure multi-party computation”. In: *Foundations and Trends® in Privacy and Security 2.2-3* (2018), pp. 70–246.
- [28] Fang, Haokun and Qian, Quan. “Privacy preserving machine learning with homomorphic encryption and federated learning”. In: *Future Internet 13.4* (2021), p. 94.
- [29] Fazli Khojir, Hamid, Alhadidi, Dima, Rouhani, Sara, and Mohammed, Noman. “FedShare: secure aggregation based on additive secret sharing in federated learning”. In: *Proceedings of the 27th International Database Engineered Applications Symposium*. 2023, pp. 25–33.
- [30] Felsen, Susanne, Kiss, Ágnes, Schneider, Thomas, and Weinert, Christian. “Secure and Private Function Evaluation with Intel SGX”. In: *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop. CCS ’19: 2019 ACM SIGSAC Conference on Computer and Communications Security*. London United Kingdom: ACM, Nov. 11, 2019, pp. 165–181. ISBN: 978-1-4503-6826-1. DOI: 10.1145/3338466.3358919. URL: <https://dl.acm.org/doi/10.1145/3338466.3358919> (visited on 07/20/2023).
- [31] El-Feshawy, Somaya A, Saad, Waleed, Shokair, Mona, and Dessouky, Moawad. “IoT framework for brain tumor detection based on optimized modified ResNet 18 (OMRES)”. In: *The Journal of Supercomputing 79.1* (2023), pp. 1081–1110.
- [32] Gascón, Adrià, Schoppmann, Phillipp, Balle, Borja, Raykova, Mariana, Doerner, Jack, Zahur, Samee, and Evans, David. “Privacy-preserving distributed linear regression on high-dimensional data”. In: *Cryptology ePrint Archive* (2016).
- [33] Gehlhar, Till, Marx, Felix, Schneider, Thomas, Suresh, Ajith, Wehrle, Tobias, and Yalame, Hossein. “SafeFl: Mpc-friendly framework for private and robust federated learning”. In: *2023 IEEE Security and Privacy Workshops (SPW)*. IEEE. 2023, pp. 69–76.

- [34] Ghodsi, Zahra, Veldanda, Akshaj Kumar, Reagen, Brandon, and Garg, Siddharth. "Cryptonas: Private inference on a relu budget". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 16961–16971.
- [35] Goldreich, Oded. *Foundations of Cryptography, Volume 2*. Cambridge university press Cambridge, 2004.
- [36] Goldreich, Oded. "Secure Multi-Party Computation". In: ().
- [37] Goldreich, Oded, Micali, Silvio, and Wigderson, Avi. "How to play any mental game, or a completeness theorem for protocols with honest majority". In: *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. 2019, pp. 307–328.
- [38] Gupta, Prajjwal, Yadav, Krishna, Gupta, Brij B, Alazab, Mamoun, and Gadekallu, Thippa Reddy. "A novel data poisoning attack in federated learning based on inverted loss function". In: *Computers & Security* 130 (2023), p. 103270.
- [39] Hamza, Aljaafari and Kumar, Basant. "A review paper on DES, AES, RSA encryption standards". In: *2020 9th International Conference System Modeling and Advancement in Research Trends (SMART)*. IEEE. 2020, pp. 333–338.
- [40] Hardy, Stephen, Henecka, Wilko, Ivey-Law, Hamish, Nock, Richard, Patrini, Giorgio, Smith, Guillaume, and Thorne, Brian. "Private federated learning on vertically partitioned data via entity resolution and additively homomorphic encryption". In: *arXiv preprint arXiv:1711.10677* (2017).
- [41] He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, and Sun, Jian. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [42] Hitaj, Briland, Ateniese, Giuseppe, and Perez-Cruz, Fernando. *Deep Models Under the GAN: Information Leakage from Collaborative Deep Learning*. Sept. 14, 2017. arXiv: 1702.07464[cs, stat]. URL: <http://arxiv.org/abs/1702.07464> (visited on 08/01/2023).
- [43] Huang, Gao, Liu, Zhuang, Pleiss, Geoff, Van Der Maaten, Laurens, and Weinberger, Kilian Q. "Convolutional networks with dense connectivity". In: *IEEE transactions on pattern analysis and machine intelligence* 44.12 (2019), pp. 8704–8716.

- [44] Huang, Yimin, Wang, Wanwan, Zhao, Xingying, Wang, Yukun, Feng, Xinyu, He, Hao, and Yao, Ming. “EFMVFL: an efficient and flexible multi-party vertical federated learning without a third party”. In: *ACM Transactions on Knowledge Discovery from Data* 18.3 (2023), pp. 1–20.
- [45] Hui, Siyuan, Zhang, Yuqiu, Hu, Albert, and Song, Edmund. “Horizontal Federated Learning and Secure Distributed Training for Recommendation System with Intel SGX”. In: *arXiv preprint arXiv:2207.05079* (2022).
- [46] Jiang, Mingyang, Liang, Yanchun, Feng, Xiaoyue, Fan, Xiaojing, Pei, Zhili, Xue, Yu, and Guan, Renchu. “Text classification based on deep belief network and softmax regression”. In: *Neural Computing and Applications* 29 (2018), pp. 61–70.
- [47] Kadam, Kalyani Ganesh and Khairnar, Vaishali. “Hybrid rsa-aes encryption for web services”. In: *International Journal of Technical Research and Applications”, Special* 31 (2015), pp. 51–56.
- [48] Kaminaga, Hiroki, Awaysheh, Feras M, Alawadi, Sadi, and Kamm, Liina. “MPCFL: Towards Multi-party Computation for Secure Federated Learning Aggregation”. In: *Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing*. 2023, pp. 1–10.
- [49] Kanagavelu, Renuga, Li, Zengxiang, Samsudin, Juniarto, Yang, Yechao, Yang, Feng, Goh, Rick Siow Mong, Cheah, Mervyn, Wiwatphonthana, Praewpiraya, Akkarajitsakul, Khajonpong, and Wang, Shangguang. “Two-phase multi-party computation enabled privacy-preserving federated learning”. In: *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. IEEE. 2020, pp. 410–419.
- [50] Keller, Marcel and Scholl, Peter. “Efficient, oblivious data structures for MPC”. In: *Advances in Cryptology–ASIACRYPT 2014: 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, ROC, December 7–11, 2014, Proceedings, Part II* 20. Springer. 2014, pp. 506–525.
- [51] Kim, Muah, Günlü, Onur, and Schaefer, Rafael F. “Federated learning with local differential privacy: Trade-offs between privacy, utility, and communication”. In: *ICASSP 2021–2021 IEEE International Conference on*

- Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2021, pp. 2650–2654.
- [52] Kolesnikov, Vladimir. *Secure two-party computation and communication*. Citeseer, 2006.
- [53] Koti, Nishat, Pancholi, Mahak, Patra, Arpita, and Suresh, Ajith. “{SWIFT}: Super-fast and robust {Privacy-Preserving} machine learning”. In: *30th USENIX Security Symposium (USENIX Security 21)*. 2021, pp. 2651–2668.
- [54] Krizhevsky, Alex, Hinton, Geoffrey, et al. “Learning multiple layers of features from tiny images”. In: (2009).
- [55] Krizhevsky, Alex, Sutskever, Ilya, and Hinton, G. “ImageNet classification with deep convolutional neural networks (AlexNet) ImageNet classification with deep convolutional neural networks (AlexNet)”. In: *Actorsfit. Com* (2023).
- [56] Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. “Imagenet classification with deep convolutional neural networks”. In: *Advances in neural information processing systems* 25 (2012).
- [57] Kuswaha, Shashikant, Waghmare, Sachin, and Choudhary, P. “Data Transmission using AES-RSA Based Hybrid Security Algorithms”. In: *International Journal on Recent and Innovation Trends in Computing and Communication* 3.4 (2015), pp. 1964–1969.
- [58] Kuutti, Sampo, Bowden, Richard, Jin, Yaochu, Barber, Phil, and Fallah, Saber. “A survey of deep learning applications to autonomous vehicle control”. In: *IEEE Transactions on Intelligent Transportation Systems* 22.2 (2020), pp. 712–733.
- [59] LeCun, Yann et al. “LeNet-5, convolutional neural networks”. In: *URL: <http://yann.lecun.com/exdb/lenet>* 20.5 (2015), p. 14.
- [60] LeCun, Yann, Bottou, Léon, Bengio, Yoshua, and Haffner, Patrick. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [61] Lee, JDMCK and Toutanova, K. “Pre-training of deep bidirectional transformers for language understanding”. In: *arXiv preprint arXiv:1810.04805* 3 (2018), p. 8.

- [62] Li, Audeliano Wolian and Bastos, Guilherme Sousa. “Stock market forecasting using deep learning and technical analysis: a systematic review”. In: *IEEE access* 8 (2020), pp. 185232–185242.
- [63] Li, Qiongxiu, Cascudo, Ignacio, and Christensen, Mads Græsbøll. “Privacy-preserving distributed average consensus based on additive secret sharing”. In: *2019 27th European Signal Processing Conference (EUSIPCO)*. IEEE. 2019, pp. 1–5.
- [64] Lindell, Yehuda. “Secure multiparty computation”. In: *Communications of the ACM* 64.1 (2020), pp. 86–96.
- [65] Liu, Changchang, Chakraborty, Supriyo, and Verma, Dinesh. “Secure model fusion for distributed learning using partial homomorphic encryption”. In: *Policy-Based Autonomic Data Governance* (2019), pp. 154–179.
- [66] Liu, Yang, Kang, Yan, Xing, Chaoping, Chen, Tianjian, and Yang, Qiang. “A secure federated transfer learning framework”. In: *IEEE Intelligent Systems* 35.4 (2020), pp. 70–82.
- [67] Liu, Ye, Gong, Wei, and Fan, Wenqing. “Application of AES and RSA Hybrid Algorithm in E-mail”. In: *2018 IEEE/ACIS 17th International Conference on Computer and Information Science (ICIS)*. IEEE. 2018, pp. 701–703.
- [68] Lyu, Lingjuan, Yu, Han, Ma, Xingjun, Chen, Chen, Sun, Lichao, Zhao, Jun, Yang, Qiang, and Philip, S Yu. “Privacy and robustness in federated learning: Attacks and defenses”. In: *IEEE transactions on neural networks and learning systems* (2022).
- [69] Lyu, Lingjuan, Yu, Han, and Yang, Qiang. “Threats to federated learning: A survey”. In: *arXiv preprint arXiv:2003.02133* (2020).
- [70] Ma, Chuan, Li, Jun, Ding, Ming, Yang, Howard H, Shu, Feng, Quek, Tony QS, and Poor, H Vincent. “On safeguarding privacy and security in the framework of federated learning”. In: *IEEE network* 34.4 (2020), pp. 242–248.
- [71] Macedonia, Michael. “The GPU enters computing’s mainstream”. In: *Computer* 36.10 (2003), pp. 106–108.

- [72] Makri, Eleftheria, Rotaru, Dragos, Vercauteren, Frederik, and Wagh, Sameer. “Rabbit: Efficient comparison for secure multi-party computation”. In: *International Conference on Financial Cryptography and Data Security*. Springer. 2021, pp. 249–270.
- [73] McCulloch, Warren S and Pitts, Walter. “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 115–133.
- [74] McKeen, Frank, Alexandrovich, Ilya, Berenzon, Alex, Rozas, Carlos V, Shafi, Hisham, Shanbhogue, Vedvyas, and Savagaonkar, Uday R. “Innovative instructions and software model for isolated execution.” In: *Hasp@ isca 10.1* (2013).
- [75] McMahan, Brendan and Ramage, Daniel. “Federated learning: Collaborative machine learning without centralized training data”. In: *Google Research Blog* (Mar. 2017).
- [76] Mo, Fan and Haddadi, Hamed. “Efficient and private federated learning using tee”. In: *Proc. EuroSys Conf., Dresden, Germany*. 2019.
- [77] Mo, Fan, Haddadi, Hamed, Katevas, Kleomenis, Marin, Eduard, Perino, Diego, and Kourtellis, Nicolas. “PPFL: privacy-preserving federated learning with trusted execution environments”. In: *Proceedings of the 19th annual international conference on mobile systems, applications, and services*. 2021, pp. 94–108.
- [78] Mohassel, Payman and Zhang, Yupeng. “Secureml: A system for scalable privacy-preserving machine learning”. In: *2017 IEEE symposium on security and privacy (SP)*. IEEE. 2017, pp. 19–38.
- [79] Mothukuri, Virraji, Parizi, Reza M, Pouriyeh, Seyedamin, Huang, Yan, Dehghantanha, Ali, and Srivastava, Gautam. “A survey on security and privacy of federated learning”. In: *Future Generation Computer Systems* 115 (2021), pp. 619–640.
- [80] Muazu, Tasiu, Mao, Yingchi, Muhammad, Abdullahi Uwaisu, Ibrahim, Muhammad, Kumshe, Umar Muhammad Mustapha, and Samuel, Omaji. “A federated learning system with data fusion for healthcare using multi-party computation and additive secret sharing”. In: *Computer Communications* 216 (2024), pp. 168–182.

- [81] Naehrig, Michael, Lauter, Kristin, and Vaikuntanathan, Vinod. “Can homomorphic encryption be practical?” In: *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*. 2011, pp. 113–124.
- [82] Naik, Ds Bhupal and Dondeti, Venkatesulu. “Pothole Detection and Classification in Vehicular Networks using ResNet-18”. In: *2023 IEEE 7th Conference on Information and Communication Technology (CICT)*. IEEE. 2023, pp. 1–6.
- [83] Nair, Akarsh K, Raj, Ebin Deni, and Sahoo, Jayakrushna. “A robust analysis of adversarial attacks on federated learning environments”. In: *Computer Standards & Interfaces* 86 (2023), p. 103723.
- [84] Nikolaenko, Valeria, Weinsberg, Udi, Ioannidis, Stratis, Joye, Marc, Boneh, Dan, and Taft, Nina. “Privacy-preserving ridge regression on hundreds of millions of records”. In: *2013 IEEE symposium on security and privacy*. IEEE. 2013, pp. 334–348.
- [85] Ozbayoglu, Ahmet Murat, Gudelek, Mehmet Ugur, and Sezer, Omer Berat. “Deep learning for financial applications: A survey”. In: *Applied soft computing* 93 (2020), p. 106384.
- [86] Rosenblatt, Frank. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological review* 65.6 (1958), p. 386.
- [87] Shamir, Adi. “How to share a secret”. In: *Communications of the ACM* 22.11 (1979), pp. 612–613.
- [88] Shamshirband, Shahab, Fathi, Mahdis, Dehzangi, Abdollah, Chronopoulos, Anthony Theodore, and Alinejad-Rokny, Hamid. “A review on deep learning approaches in healthcare systems: Taxonomies, challenges, and open issues”. In: *Journal of Biomedical Informatics* 113 (2021), p. 103627.
- [89] Shi, Zhaosen, Yang, Zeyu, Hassan, Alzubair, Li, Fagen, and Ding, Xuyang. “A privacy preserving federated learning scheme using homomorphic encryption and secret sharing”. In: *Telecommunication Systems* 82.3 (2023), pp. 419–433.
- [90] Simonyan, Karen and Zisserman, Andrew. “Very deep convolutional networks for large-scale image recognition”. In: *arXiv preprint arXiv:1409.1556* (2014).

- [91] So, Jinhyun, Güler, Başak, and Avestimehr, A Salman. “Byzantine-resilient secure federated learning”. In: *IEEE Journal on Selected Areas in Communications* 39.7 (2020), pp. 2168–2181.
- [92] So, Jinhyun, Güler, Başak, and Avestimehr, A Salman. “Turbo-aggregate: Breaking the quadratic aggregation barrier in secure federated learning”. In: *IEEE Journal on Selected Areas in Information Theory* 2.1 (2021), pp. 479–489.
- [93] Sotthiwat, Ekanut, Zhen, Liangli, Li, Zengxiang, and Zhang, Chi. “Partially encrypted multi-party computation for federated learning”. In: *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE. 2021, pp. 828–835.
- [94] Stephen, Ancy, Punitha, A, and Chandrasekar, A. “Designing self attention-based ResNet architecture for rice leaf disease classification”. In: *Neural Computing and Applications* 35.9 (2023), pp. 6737–6751.
- [95] Taud, Hind and Mas, Jean-Franccois. “Multilayer perceptron (MLP)”. In: *Geomatic approaches for modeling land change scenarios* (2018), pp. 451–455.
- [96] Tjell, Katrine and Wisniewski, Rafael. “Privacy in distributed computations based on real number secret sharing”. In: *arXiv preprint arXiv:2107.00911* (2021).
- [97] Tolpegin, Vale, Truex, Stacey, Gursoy, Mehmet Emre, and Liu, Ling. “Data poisoning attacks against federated learning systems”. In: *Computer Security–ESORICS 2020: 25th European Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, September 14–18, 2020, Proceedings, Part I* 25. Springer. 2020, pp. 480–501.
- [98] Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N, Kaiser, Łukasz, and Polosukhin, Illia. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [99] Wagh, Sameer, Gupta, Divya, and Chandran, Nishanth. “SecureNN: 3-party secure computation for neural network training”. In: *Proceedings on Privacy Enhancing Technologies* (2019).

- [100] Wagh, Sameer, Tople, Shruti, Benhamouda, Fabrice, Kushilevitz, Eyal, Mittal, Prateek, and Rabin, Tal. “Falcon: Honest-majority maliciously secure framework for private deep learning”. In: *arXiv preprint arXiv:2004.02229* (2020).
- [101] Wang, Qi, Ma, Yue, Zhao, Kun, and Tian, Yingjie. “A comprehensive survey of loss functions in machine learning”. In: *Annals of Data Science* (2020), pp. 1–26.
- [102] Wang, Yongqin, Suh, G Edward, Xiong, Wenjie, Lefaudeux, Benjamin, Knott, Brian, Annavaram, Murali, and Lee, Hsien-Hsin S. “Characterization of mpc-based private inference for transformer-based models”. In: *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE. 2022, pp. 187–197.
- [103] Watson, Jean-Luc, Wagh, Sameer, and Popa, Raluca Ada. “Piranha: A {GPU} platform for secure computation”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 827–844.
- [104] Wei, Wenqi, Liu, Ling, Loper, Margaret, Chow, Ka-Ho, Gursoy, Mehmet Emre, Truex, Stacey, and Wu, Yanzhao. “A framework for evaluating gradient leakage attacks in federated learning”. In: *arXiv preprint arXiv:2004.10397* (2020).
- [105] Wei, Wenqi, Liu, Ling, Wut, Yanzhao, Su, Gong, and Iyengar, Arun. “Gradient-leakage resilient federated learning”. In: *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*. IEEE. 2021, pp. 797–807.
- [106] Werbos, Paul. “Beyond regression: New tools for prediction and analysis in the behavioral sciences”. In: *PhD thesis, Committee on Applied Mathematics, Harvard University, Cambridge, MA* (1974).
- [107] Wu, Pengfei, Ning, Jianting, Shen, Jiamin, Wang, Hongbing, and Chang, Ee-Chien. “Hybrid Trust Multi-party Computation with Trusted Execution Environment.” In: *NDSS*. 2022.
- [108] Wu, Xiaolong, Tian, Dave Jing, and Kim, Chung Hwan. “Building GPU TEEs using CPU Secure Enclaves with GEVisor”. In: *Proceedings of the 2023 ACM Symposium on Cloud Computing* (2023). DOI: 10.1145/3620678.3624659. URL: <https://doi.org/10.1145/3620678.3624659>.

- [109] Wu, Yuncheng, Xing, Naili, Chen, Gang, Dinh, Tien Tuan Anh, Luo, Zhaojing, Ooi, Beng Chin, Xiao, Xiaokui, and Zhang, Meihui. “Falcon: A Privacy-Preserving and Interpretable Vertical Federated Learning System”. In: *Proceedings of the VLDB Endowment* 16.10 (2023), pp. 2471–2484.
- [110] Xiong, Lizhi, Zhou, Wenhao, Xia, Zhihua, Gu, Qi, and Weng, Jian. “Efficient privacy-preserving computation based on additive secret sharing”. In: *arXiv preprint arXiv:2009.05356* (2020).
- [111] Yao, Andrew C. “Protocols for secure computations”. In: *23rd annual symposium on foundations of computer science (sfcs 1982)*. IEEE. 1982, pp. 160–164.
- [112] Zhang, Chengliang, Li, Suyi, Xia, Junzhe, Wang, Wei, Yan, Feng, and Liu, Yang. “{BatchCrypt}: Efficient homomorphic encryption for {Cross-Silo} federated learning”. In: *2020 USENIX annual technical conference (USENIX ATC 20)*. 2020, pp. 493–506.
- [113] Zhang, Chi, Ekanut, Sotthiwat, Zhen, Liangli, and Li, Zengxiang. “Augmented multi-party computation against gradient leakage in federated learning”. In: *IEEE Transactions on Big Data* (2022).
- [114] Zhang, Xinyang, Ji, Shouling, Wang, Hui, and Wang, Ting. “Private, yet practical, multiparty deep learning”. In: *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. IEEE. 2017, pp. 1442–1452.
- [115] Zhang, Yahui, Zhao, Min, Li, Tingquan, and Han, Huan. “Survey of Attacks and Defenses against SGX”. In: *2020 IEEE 5th Information Technology and Mechatronics Engineering Conference (ITOEC)*. 2020, pp. 1492–1496. DOI: 10.1109/ITOEC49072.2020.9141835.
- [116] Zhao, Ying, Chen, Junjun, Zhang, Jiale, Wu, Di, Blumenstein, Michael, and Yu, Shui. “Detecting and mitigating poisoning attacks in federated learning using generative adversarial networks”. In: *Concurrency and Computation: Practice and Experience* 34.7 (2022), e5906.
- [117] Zhu, Huafei, Goh, Rick Siow Mong, and Ng, Wee-Keong. “Privacy-preserving weighted federated learning within the secret sharing framework”. In: *IEEE Access* 8 (2020), pp. 198275–198284.

Appendix - Contents

A First Appendix	99
A.1 Original Code for Pseudocode and model structure	99
A.1.1 Conv	99
A.2 Original Code for Pseudocode	99
A.2.1 LeNet5(MNIST)	99
A.2.2 AlexNet(MNIST)	100
A.2.3 ResNet18(MNIST)	102
A.2.4 VGG16(MNIST)	105

Appendix A

First Appendix

A.1 Original Code for Pseudocode and model structure

A.1.1 Conv

```
1  #define CONVOLUTION_FORWARD(input,output,weight,bias,action)
2  {
3  for(int x=0;x<GEILENGTH(weight);++x)
4  for(int y=0; y<GEILENGTH(*weight);++y)
5  CONVOLUTE_VALID(input[x], output[y], weight[x][y]);
6  FOREACH(j, GEILENGTH(output))
7  FOREACH(i, GEICOUNT(output[j]))
8  (((double *)output[j])[i]=action((((double *)output[j])[i]
9  +bias[j]);
10 }
11
12 #define CONVOLUTE_VALID(input,output,weight)
13 {
14 FOREACH(o0,GEILENGTH(output))
15 FOREACH(o1,GEILENGTH(*output))
16 FOREACH(w0,GEILENGTH(weight))
17 FOREACH(w1,GEILENGTH(*weight))
18 (output)[o0][o1]+=(input)[o0+w0][o1+w1](weight)[w0][w1];
19 }
```

A.2 Original Code for Pseudocode

A.2.1 LeNet5(MNIST)

```
1
2 #pragma once
3 #define LENGTH_KERNEL 5
4
5 #define LENGTH_FEATURE0 32
6 #define LENGTH_FEATURE1 (LENGTH_FEATURE0 - LENGTH_KERNEL + 1)
7 #define LENGTH_FEATURE2 (LENGTH_FEATURE1 >> 1)
```

```

8  #define LENGTH_FEATURE3 (LENGTH_FEATURE2 - LENGTH_KERNEL + 1)
9  #define LENGTH_FEATURE4 (LENGTH_FEATURE3 >> 1)
10 #define LENGTH_FEATURE5 (LENGTH_FEATURE4 - LENGTH_KERNEL + 1)
11
12 #define INPUT          1
13 #define LAYER1         6
14 #define LAYER2         6
15 #define LAYER3        16
16 #define LAYER4        16
17 #define LAYER5       120
18 #define OUTPUT        10
19
20 #define ALPHA 0.5
21 #define PADDING 2
22
23 typedef unsigned char uint8;
24 typedef uint8 image[28][28];
25
26
27 typedef struct LeNet5
28 {
29     double weight0_1[INPUT][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
30     double weight2_3[LAYER2][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
31     double weight4_5[LAYER4][LAYER5][LENGTH_KERNEL][LENGTH_KERNEL];
32     double weight5_6[LAYER5*LENGTH_FEATURE5*LENGTH_FEATURE5][OUTPUT];
33
34     double bias0_1[LAYER1];
35     double bias2_3[LAYER3];
36     double bias4_5[LAYER5];
37     double bias5_6[OUTPUT];
38
39 }LeNet5;
40
41 typedef struct Feature
42 {
43     double input[INPUT][LENGTH_FEATURE0][LENGTH_FEATURE0];
44     double layer1[LAYER1][LENGTH_FEATURE1][LENGTH_FEATURE1];
45     double layer2[LAYER2][LENGTH_FEATURE2][LENGTH_FEATURE2];
46     double layer3[LAYER3][LENGTH_FEATURE3][LENGTH_FEATURE3];
47     double layer4[LAYER4][LENGTH_FEATURE4][LENGTH_FEATURE4];
48     double layer5[LAYER5][LENGTH_FEATURE5][LENGTH_FEATURE5];
49     double output[OUTPUT];
50 }Feature;
51
52 void TrainBatch(LeNet5 *lenet, image *inputs, uint8 *labels, int batchSize);
53
54 void Train(LeNet5 *lenet, image input, uint8 label);
55
56 uint8 Predict(LeNet5 *lenet, image input, uint8 count);
57
58 void Initial(LeNet5 *lenet);
59 double relu(double x);

```

A.2.2 AlexNet(MNIST)

```

1
2 #pragma once
3
4 #define LENGTH_KERNEL    3
5 #define MAXPOOL_SIZE    2
6
7 #define PADDING 1
8 #define LENGTH_FEATURE0 28
9 #define LENGTH_FEATURE1_1 (LENGTH_FEATURE0 - LENGTH_KERNEL + 2*PADDING+1)
10 #define LENGTH_FEATURE1_2 (LENGTH_FEATURE1_1/2)
11
12 #define LENGTH_FEATURE2_1 (LENGTH_FEATURE1_2 - LENGTH_KERNEL + 2*PADDING + 1)
13 #define LENGTH_FEATURE2_2 (LENGTH_FEATURE2_1/2)

```

```

14
15 #define LENGTH_FEATURE3_1      (LENGTH_FEATURE2_2 - LENGTH_KERNEL +2*PADDING+ 1)
16
17 #define LENGTH_FEATURE4_1      (LENGTH_FEATURE3_1 - LENGTH_KERNEL +2*PADDING+ 1)
18
19 #define MAXPOOL5_2_KERNEL 3
20 #define MAXPOOL5_2_STRIDE 2
21
22 #define LENGTH_FEATURE5_1      (LENGTH_FEATURE4_1 - LENGTH_KERNEL+2*PADDING+ 1)
23 #define LENGTH_FEATURE5_2      ((LENGTH_FEATURE5_1 - MAXPOOL5_2_KERNEL)/MAXPOOL5_2_STRIDE+ 1)
24
25 #define FC1_OUTPUT      1024
26 #define FC2_OUTPUT      512
27 #define FC3_OUTPUT      10
28
29
30 #define INPUT      1
31 #define LAYER1      32
32 #define LAYER2      64
33 #define LAYER3      128
34 #define LAYER4      256
35 #define LAYER5      256
36
37 #define OUTPUT      10
38
39 #define ALPHA 0.1
40
41
42 typedef unsigned char uint8;
43 typedef uint8 image[28][28];
44
45
46
47 typedef struct AlexNet
48 {
49     double weight1[INPUT][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL]; //Layer1 kernel
50     double weight2[LAYER1][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL]; //Layer2 Kernel
51     double weight3[LAYER2][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL]; //Layer3 Kernel
52     //double weight5_6[LAYER5 * LENGTH_FEATURE5 * LENGTH_FEATURE5][OUTPUT];
53     double weight4[LAYER3][LAYER4][LENGTH_KERNEL][LENGTH_KERNEL]; //Layer4 Kernel
54     double weight5[LAYER4][LAYER5][LENGTH_KERNEL][LENGTH_KERNEL]; //Layer5 Kernel
55
56
57     double fc1[LAYER5*LENGTH_FEATURE5_2*LENGTH_FEATURE5_2][FC1_OUTPUT];
58     double fc2[FC1_OUTPUT][FC2_OUTPUT];
59     double fc3[FC2_OUTPUT][FC3_OUTPUT];
60
61
62
63     double bias1[LAYER1];
64     double bias2[LAYER2];
65     double bias3[LAYER3];
66     double bias4[LAYER4];
67     double bias5[LAYER5];
68
69     double bias_fc1[FC1_OUTPUT];
70     double bias_fc2[FC2_OUTPUT];
71     double bias_fc3[FC3_OUTPUT];
72
73 }AlexNet;
74
75 typedef struct Feature
76 {
77     double input[INPUT][LENGTH_FEATURE0][LENGTH_FEATURE0];
78
79     double layer1_conv[LAYER1][LENGTH_FEATURE1_1][LENGTH_FEATURE1_1];
80     double layer1_pool[LAYER1][LENGTH_FEATURE1_2][LENGTH_FEATURE1_2];
81
82     double layer2_conv[LAYER2][LENGTH_FEATURE2_1][LENGTH_FEATURE2_1];
83     double layer2_pool[LAYER2][LENGTH_FEATURE2_2][LENGTH_FEATURE2_2];
84

```

```

85     double layer3_conv [LAYER3] [LENGTH_FEATURE3_1] [LENGTH_FEATURE3_1];
86
87     double layer4_conv [LAYER4] [LENGTH_FEATURE4_1] [LENGTH_FEATURE4_1];
88
89
90     double layer5_conv [LAYER5] [LENGTH_FEATURE5_1] [LENGTH_FEATURE5_1]; // 256*7*7
91     double layer5_pool [LAYER5] [LENGTH_FEATURE5_2] [LENGTH_FEATURE5_2]; // 256*3*3
92
93     double fc1 [FC1_OUTPUT];
94     double fc2 [FC2_OUTPUT];
95     double output [FC3_OUTPUT];
96
97 }Feature;
98
99
100 typedef struct Feature_Pad
101 {
102     double input [INPUT] [LENGTH_FEATURE0+2*PADDING] [LENGTH_FEATURE0+2*PADDING];
103
104     double layer1_conv [LAYER1] [LENGTH_FEATURE1_1] [LENGTH_FEATURE1_1];
105     double layer1_pool [LAYER1] [LENGTH_FEATURE1_2+2*PADDING] [LENGTH_FEATURE1_2+2*PADDING];
106
107     double layer2_conv [LAYER2] [LENGTH_FEATURE2_1] [LENGTH_FEATURE2_1];
108     double layer2_pool [LAYER2] [LENGTH_FEATURE2_2+2*PADDING] [LENGTH_FEATURE2_2+2*PADDING];
109
110     double layer3_conv [LAYER3] [LENGTH_FEATURE3_1+2*PADDING] [LENGTH_FEATURE3_1+2*PADDING];
111
112     double layer4_conv [LAYER4] [LENGTH_FEATURE4_1+2*PADDING] [LENGTH_FEATURE4_1+2*PADDING];
113
114
115     double layer5_conv [LAYER5] [LENGTH_FEATURE5_1] [LENGTH_FEATURE5_1]; // 256*7*7
116     double layer5_pool [LAYER5] [LENGTH_FEATURE5_2] [LENGTH_FEATURE5_2]; // 256*3*3
117
118     double fc1 [FC1_OUTPUT];
119     double fc2 [FC2_OUTPUT];
120     double output [FC3_OUTPUT];
121
122 }Feature_Pad;
123
124 void TrainBatch(AlexNet *alexnet, image *inputs, uint8 *labels, int batchSize);
125
126 void Train(AlexNet *alexnet, image input, uint8 label);
127
128 uint8 Predict(AlexNet *alexnet, image input, uint8 count);
129
130 void Initial(AlexNet *alexnet);

```

A.2.3 ResNet18(MNIST)

```

1
2
3 #pragma once
4
5 #define LENGTH_KERNEL 3
6
7 #define LENGTH_FEATURE0 32
8 #define LENGTH_FEATURE1 (LENGTH_FEATURE0 - LENGTH_KERNEL + 1)
9 #define LENGTH_FEATURE2 (LENGTH_FEATURE1 >> 1)
10 #define LENGTH_FEATURE3 (LENGTH_FEATURE2 - LENGTH_KERNEL + 1)
11 #define LENGTH_FEATURE4 (LENGTH_FEATURE3 >> 1)
12 #define LENGTH_FEATURE5 (LENGTH_FEATURE4 - LENGTH_KERNEL + 1)
13
14 #define INPUT 1
15 #define LAYER0 64
16 #define LAYER1 64
17 #define LAYER2 128
18 #define LAYER3 256
19 #define LAYER4 512

```

```

20 #define OUTPUT          10
21
22 #define Res1 16
23 #define Res2 8
24 #define Res3 4
25 #define Res4 2
26 #define pool2_size 1
27
28 #define ALPHA 0.5
29 #define PADDING 1
30
31 typedef unsigned char uint8;
32 typedef uint8 image[28][28];
33
34 typedef struct ResBlock1
35 {
36     double weight1_1[LAYER0][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
37     double weight1_2[LAYER1][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
38
39     double weight2_1[LAYER1][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
40     double weight2_2[LAYER1][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
41
42
43     double bias1_1[LAYER1];
44     double bias1_2[LAYER1];
45     double bias2_1[LAYER1];
46     double bias2_2[LAYER1];
47 }ResBlock1;
48 typedef struct ResBlock2
49 {
50     double weight1_1[LAYER1][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL];
51     double weight1_2[LAYER2][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL];
52     double conv1[LAYER1][LAYER2][1][1];
53     double weight2_1[LAYER2][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL];
54     double weight2_2[LAYER2][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL];
55
56     double bias1_1[LAYER2];
57     double bias1_2[LAYER2];
58     double bias2_1[LAYER2];
59     double bias2_2[LAYER2];
60 }ResBlock2;
61 typedef struct ResBlock3
62 {
63     double weight1_1[LAYER2][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
64     double weight1_2[LAYER3][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
65     double conv1[LAYER2][LAYER3][1][1];
66     double weight2_1[LAYER3][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
67     double weight2_2[LAYER3][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
68
69     double bias1_1[LAYER3];
70     double bias1_2[LAYER3];
71     double bias2_1[LAYER3];
72     double bias2_2[LAYER3];
73 }ResBlock3;
74 typedef struct ResBlock4
75 {
76     double weight1_1[LAYER3][LAYER4][LENGTH_KERNEL][LENGTH_KERNEL];
77     double weight1_2[LAYER4][LAYER4][LENGTH_KERNEL][LENGTH_KERNEL];
78     double conv1[LAYER3][LAYER4][1][1];
79     double weight2_1[LAYER4][LAYER4][LENGTH_KERNEL][LENGTH_KERNEL];
80     double weight2_2[LAYER4][LAYER4][LENGTH_KERNEL][LENGTH_KERNEL];
81
82     double bias1_1[LAYER4];
83     double bias1_2[LAYER4];
84     double bias2_1[LAYER4];
85     double bias2_2[LAYER4];
86 }ResBlock4;
87
88
89 typedef struct ResNet18
90 {

```

```

91     double weight1 [INPUT] [LAYER1] [LENGTH_KERNEL] [LENGTH_KERNEL]; //Layer1 kernel
92     ResBlock1 res_block1;
93     ResBlock2 res_block2;
94     ResBlock3 res_block3;
95     ResBlock4 res_block4;
96
97     double fc [LAYER4*pool2_size*pool2_size] [OUTPUT];
98     double bias1 [LAYER1];
99     double bias_fc [OUTPUT];
100 }ResNet18;
101
102
103 typedef struct Res1_Feature
104 {
105     double conv1_1 [LAYER1] [Res1] [Res1];
106     double conv1_2 [LAYER1] [Res1] [Res1];
107
108     double conv2_1 [LAYER1] [Res1] [Res1];
109     double conv2_2 [LAYER1] [Res1] [Res1];
110 }Res1_Feature;
111
112
113 typedef struct Res2_Feature
114 {
115     double conv1_1 [LAYER2] [Res2] [Res2];
116     double conv1_2 [LAYER2] [Res2] [Res2];
117
118     double conv_res [LAYER2] [Res2] [Res2];
119
120     double conv2_1 [LAYER2] [Res2] [Res2];
121     double conv2_2 [LAYER2] [Res2] [Res2];
122 }Res2_Feature;
123
124 typedef struct Res3_Feature
125 {
126     double conv1_1 [LAYER3] [Res3] [Res3];
127     double conv1_2 [LAYER3] [Res3] [Res3];
128
129     double conv_res [LAYER3] [Res3] [Res3];
130
131     double conv2_1 [LAYER3] [Res3] [Res3];
132     double conv2_2 [LAYER3] [Res3] [Res3];
133 }Res3_Feature;
134
135 typedef struct Res4_Feature
136 {
137     double conv1_1 [LAYER4] [Res4] [Res4];
138     double conv1_2 [LAYER4] [Res4] [Res4];
139
140     double conv_res [LAYER4] [Res4] [Res4];
141
142     double conv2_1 [LAYER4] [Res4] [Res4];
143     double conv2_2 [LAYER4] [Res4] [Res4];
144 }Res4_Feature;
145
146
147
148
149
150 typedef struct Feature
151 {
152     double input [INPUT] [LENGTH_FEATURE0] [LENGTH_FEATURE0];
153     double conv1 [LAYER0] [LENGTH_FEATURE0] [LENGTH_FEATURE0];
154     double pool1 [LAYER0] [16] [16];
155     Res1_Feature res1f;
156     Res2_Feature res2f;
157     Res3_Feature res3f;
158     Res4_Feature res4f;
159
160     double pool2 [LAYER4] [pool2_size] [pool2_size];
161     double output [OUTPUT];

```

```

162
163
164
165 }Feature;
166
167 void TrainBatch(ResNet18 *resnet, image *inputs, uint8 *labels, int batchSize);
168
169 void Train(ResNet18 *resnet, image input, uint8 label);
170
171 uint8 Predict(ResNet18 *resnet, image input, uint8 count);
172
173 void Initial(ResNet18 *resnet);
174 double relu(double x);

```

A.2.4 VGG16(MNIST)

```

1  #pragma once
2
3  // #include <charconv>
4  #define LENGTH_KERNEL 3
5
6  #define LENGTH_FEATURE0 32
7  #define LENGTH_FEATURE1 LENGTH_FEATURE0/2 //32
8  #define LENGTH_FEATURE2 LENGTH_FEATURE1/2 //16
9  #define LENGTH_FEATURE3 LENGTH_FEATURE2/2 //8
10 #define LENGTH_FEATURE4 LENGTH_FEATURE3/2 //4
11 #define LENGTH_FEATURE5 LENGTH_FEATURE4/2 //2
12
13
14
15 #define INPUT 1
16 #define LAYER1 64
17
18
19 #define LAYER2 128
20
21 #define LAYER3 256
22
23
24 #define LAYER4 512
25
26 #define LAYER5 512
27 #define FC1 4096
28 #define FC2 4096
29
30 #define OUTPUT 10
31
32 #define ALPHA 0.5
33 #define PADDING 1
34
35 typedef unsigned char uint8;
36 typedef uint8 image[28][28];
37
38
39 typedef struct VGG
40 {
41     double weight1_1[INPUT][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
42     double weight1_2[LAYER1][LAYER1][LENGTH_KERNEL][LENGTH_KERNEL];
43     double weight2_1[LAYER1][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL];
44     double weight2_2[LAYER2][LAYER2][LENGTH_KERNEL][LENGTH_KERNEL];
45
46
47     double weight3_1[LAYER2][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
48     double weight3_2[LAYER3][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
49     double weight3_3[LAYER3][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
50     double weight3_4[LAYER3][LAYER3][LENGTH_KERNEL][LENGTH_KERNEL];
51
52

```

```

53     double weight4_1 [LAYER3] [LAYER4] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
54     double weight4_2 [LAYER4] [LAYER4] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
55     double weight4_3 [LAYER4] [LAYER4] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
56     double weight4_4 [LAYER4] [LAYER4] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
57
58     double weight5_1 [LAYER4] [LAYER5] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
59     double weight5_2 [LAYER5] [LAYER5] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
60     double weight5_3 [LAYER5] [LAYER5] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
61     double weight5_4 [LAYER5] [LAYER5] [LENGTH_KERNEL] [LENGTH_KERNEL] ;
62
63     double fc1 [LAYER5*LENGTH_FEATURE5*LENGTH_FEATURE5] [FC1] ;
64     double fc2 [FC1] [FC2] ;
65     double fc3 [FC2] [OUTPUT] ;
66
67     double bias1_1 [LAYER1] ;
68     double bias1_2 [LAYER1] ;
69
70     double bias2_1 [LAYER2] ;
71     double bias2_2 [LAYER2] ;
72
73     double bias3_1 [LAYER3] ;
74     double bias3_2 [LAYER3] ;
75     double bias3_3 [LAYER3] ;
76     double bias3_4 [LAYER3] ;
77
78     double bias4_1 [LAYER4] ;
79     double bias4_2 [LAYER4] ;
80     double bias4_3 [LAYER4] ;
81     double bias4_4 [LAYER4] ;
82
83
84     double bias5_1 [LAYER5] ;
85     double bias5_2 [LAYER5] ;
86     double bias5_3 [LAYER5] ;
87     double bias5_4 [LAYER5] ;
88
89     double bias_fc1 [FC1] ;
90     double bias_fc2 [FC2] ;
91     double bias_fc3 [OUTPUT] ;
92 }VGG;
93
94 typedef struct Feature
95 {
96     double input [INPUT] [LENGTH_FEATURE0] [LENGTH_FEATURE0] ;
97
98     double layer1_conv1 [LAYER1] [LENGTH_FEATURE0] [LENGTH_FEATURE0] ;
99     double layer1_conv2 [LAYER1] [LENGTH_FEATURE0] [LENGTH_FEATURE0] ;
100    double layer1_pool [LAYER1] [LENGTH_FEATURE1] [LENGTH_FEATURE1] ;
101
102    double layer2_conv1 [LAYER2] [LENGTH_FEATURE1] [LENGTH_FEATURE1] ;
103    double layer2_conv2 [LAYER2] [LENGTH_FEATURE1] [LENGTH_FEATURE1] ;
104    double layer2_pool [LAYER2] [LENGTH_FEATURE2] [LENGTH_FEATURE2] ;
105
106    double layer3_conv1 [LAYER3] [LENGTH_FEATURE2] [LENGTH_FEATURE2] ;
107    double layer3_conv2 [LAYER3] [LENGTH_FEATURE2] [LENGTH_FEATURE2] ;
108    double layer3_conv3 [LAYER3] [LENGTH_FEATURE2] [LENGTH_FEATURE2] ;
109    double layer3_conv4 [LAYER3] [LENGTH_FEATURE2] [LENGTH_FEATURE2] ;
110    double layer3_pool [LAYER3] [LENGTH_FEATURE3] [LENGTH_FEATURE3] ;
111
112    double layer4_conv1 [LAYER4] [LENGTH_FEATURE3] [LENGTH_FEATURE3] ;
113    double layer4_conv2 [LAYER4] [LENGTH_FEATURE3] [LENGTH_FEATURE3] ;
114    double layer4_conv3 [LAYER4] [LENGTH_FEATURE3] [LENGTH_FEATURE3] ;
115    double layer4_conv4 [LAYER4] [LENGTH_FEATURE3] [LENGTH_FEATURE3] ;
116    double layer4_pool [LAYER4] [LENGTH_FEATURE4] [LENGTH_FEATURE4] ;
117
118    double layer5_conv1 [LAYER5] [LENGTH_FEATURE4] [LENGTH_FEATURE4] ;
119    double layer5_conv2 [LAYER5] [LENGTH_FEATURE4] [LENGTH_FEATURE4] ;
120    double layer5_conv3 [LAYER5] [LENGTH_FEATURE4] [LENGTH_FEATURE4] ;
121    double layer5_conv4 [LAYER5] [LENGTH_FEATURE4] [LENGTH_FEATURE4] ;
122    double layer5_pool [LAYER5] [LENGTH_FEATURE5] [LENGTH_FEATURE5] ;
123

```

```
124     double fc1[FC1];
125     double fc2[FC2];
126     double output[OUTPUT];
127
128 }Feature;
129
130 void TrainBatch(VGG *vggnet, image *inputs, uint8 *labels, int batchSize);
131
132 void Train(VGG *vggnet, image input, uint8 label);
133
134 uint8 Predict(VGG *vggnet, image input, uint8 count);
135
136 void Initial(VGG *vggnet);
137 double relu(double x);
```